

Budapesti Műszaki SZC Egressy Gábor Két Tanítási Nyelvű Technikum

NamonaProject Dokumentáció

Abd-El Zaher Amir, Kovács Kristóf

Namona Project Dokumentáció

Projekt: Namona Web Áruház

Backend: ASP.NET Core Web API + EF Core + PostgreSQL

Frontend: HTML/CSS/TypeScript

Desktop kliens: Avalonia

Tesztelés: xUnit + integrációs tesztek

Csapattagok feladatai

Frontend: Kovács Kristóf

Backend: Abd-El Zaher Amir (Kovács Kristóf)

Mobil(Avalonia): Abd-El Zaher Amir

Adatbázis: Abd-El Zaher Amir (Varga Szabolcs)

Tesztelés:

- Manuális tesztelés: Abd-El Zaher Amir
- Automata (unit) tesztelés: Abd-El Zaher Amir

Dokumentáció készítése és frissítése: Abd-El Zaher Amir, Kovács Kristóf

Tartalomjegyzék

Tartalomjegyzék

Csapattagok feladatai	2
1. Bevezető.....	8
1.1 Bemutató, miről szól?	8
1.2 Célok és célkitűzések	9
2. Felhasználói dokumentáció	9
2.1 Ismertető	9
2.1.1 Felhasználói szerepkörök	9
2.1.2 Főbb funkciók	10
2.1.2.1 Ruhák kezelése.....	11
2.1.2.2 Rendelési rendszer	11
2.1.2.3 Kosárkezelés	11
2.1.2.4 Felhasználókezelés.....	11
2.1.2.5 Kategória- és attribútumkezelés	12
2.1.2.5 Adminisztrációs modul	12
2.1.2.6 Mobilalkalmazás támogatás	12
2.2 Célközönség	12
2.3 Megvalósítás eszközei.....	13
2.3.1 Backend fejlesztés	13
2.3.2 Adatbázis	13
2.3.3 Webes felület.....	13
2.3.4 Mobilalkalmazás	14
2.3.5 Fejlesztői környezet és eszközök	14
2.4. Rendszerkövetelmények	14
2.5 Futtatás	15
2.6 Program használata képernyőképpen	17
2.6.1 Webes felhasználói felület	17
2.6.2 Webes admin felület.....	26
2.6.3 Avalonia alkalmazás használata képernyőképpen.....	30
3. Fejlesztői dokumentáció	40
3.1 Használt fejlesztési eszközök	40
3.1.1 Szoftvertechnológiák.....	40
3.1.2 Konkrét szoftverek és eszközök	41

	04
3.1.3 Fejlesztési megközelítés és architekturális szemlélet.....	41
3.1.4 Moduláris felépítés összefoglalása.....	42
3.2 Adatbázis.....	43
3.2.1 Az adatbázis elérésének módja	44
ER Diagram	45
3.3 REST API / Backend	46
3.3.1 A backend réteges felépítése	49
Backend osztálydiagram	50
3.3.2 Főbb backend model osztályok	52
3.3.3 UserController – Felhasználókezelés	53
Login.....	53
Admin login	53
Regisztráció	54
Összes felhasználó (Admin).....	54
Felhasználó törlése	55
Jelszó módosítás	55
User szerkesztés	55
Promote user	56
Saját profil	56
Logout	57
3.3.4 ClothesController – Ruhák kezelése.....	58
Összes ruha	58
Ruha hozzáadás.....	58
Ruha módosítás.....	59
Ruha törlés	59
Szűrés	60
Keresés	60
3.3.5 CategoryController – Kategóriák.....	61
Összes kategória	61
Kategória hozzáadás	61
Kategória módosítás	62
Kategória törlés.....	62
3.3.6 GenderController – Nemek.....	63
Összes gender	63

	05
Gender hozzáadás.....	63
Gender módosítás.....	63
Gender törlés	64
3.3.7 CartController – Kosár.....	65
Kosár tartalma.....	65
Kosár módosítás.....	65
Kosárhoz adás	66
Kosár elem törlés	66
3.3.8 OrdersController – Rendelések.....	67
Összes rendelés.....	67
Saját rendelések.....	67
Checkout	67
Rendelés módosítás	68
Bevétel	68
3.3.9 Rendelésekhez kapcsolódó funkciók és végpontok.....	69
Rendelések lekérdezése (User)	69
Checkout	70
Összes rendelés (Admin).....	70
Rendelés lekérdezése azonosító alapján (Admin)	71
Rendelés hozzáadása (Admin)	71
Rendelés módosítása (Admin)	72
Rendelés törlése.....	72
Rendelés lemondása (User)	73
Rendelés teljesítése (Admin)	73
Státusz módosítása (Admin)	74
Bevétel lekérdezése (Admin)	74
3.4 Webes frontend fejlesztői leírása - Namona E-commerce Platform	75
3.4.1. Oldalszerkezet és főbb nézetek	75
3.4.2 Közös vizuális megjelenés	76
3.4.3 A JavaScript klienslogika szerepe	76
3.4.4 DTO interfészek és adatstruktúrák a frontendben	77
3.4.5 Termékek megjelenítése és szűrése	78
3.4.6 Kategória oldalak.....	79
3.4.7 Termék detail oldalak.....	79

	06
3.4.8 Kosárkezelés.....	80
3.4.9 Rendeléskezelés.....	81
3.4.10 Mentett termékek.....	82
3.4.11 Bejelentkezés és regisztráció	82
3.4.12 Profil kezelés	85
3.4.13 Admin frontend.....	86
3.4.14 Globális keresési és szűrési rendszer	89
3.4.15 Event handling és inicializáció.....	90
3.5 Avalonia fejlesztői leírása	91
3.5.1 Alkalmazott technológiák	91
3.5.2 Az alkalmazás belépési pontja	92
3.5.3 Az App.axaml.cs működése.....	92
3.5.4 MVVM architektúra.....	92
Models	93
ViewModels.....	93
Views	94
3.5.5 A ClientModel szerepe.....	94
3.5.5.1 API kommunikáció	94
Autentikációs függvények.....	95
Ruházati termék kezelő függvények	95
Kategóriakezelő függvények.....	95
Felhasználókezelő függvények	95
Rendeléskezelő függvények.....	95
Kosárkezelő függvények.....	95
3.5.6 Az AdminPanelVM szerepe.....	96
3.5.6.1 Admin panel működése.....	96
3.5.6.2 Az AdminPanelVM.....	97
3.5.7 Inicializáció bejelentkezés után.....	97
3.5.8 Adatok kezelése az alkalmazásban	97
3.5.8.1 Ruházati termékek kezelése	98
3.5.8.2 Kategóriakezelés	99
3.5.8.3 Felhasználókezelés.....	100
3.5.8.4 Rendeléskezelés	101
3.5.8.5 Dashboard rendszer.....	102

3.5.8.6 UI állapotkezelés.....	102
Avalonia use-case diagram	103
Avalonia állapotdiagram	103
Avalonia szekvencia diagram	104
Avalonia osztálydiagram.....	108
4. Tesztelés.....	108
4.1 Tesztkörnyezet	108
4.2 UnitTest projekt	111
4.3 Tesztadatbázis és seedelés.....	111
4.4 Integrációs tesztelés :	111
4.5 Tesztesetek táblázata.....	115
5. Összegzés.....	122
6. Köszönetnyilvánítás.....	122

1. Bevezető

1.1 Bemutató, miről szól?

A **Nanoma** egy online ruházati webshop, amely a divat iránt érdeklődő vásárlók számára kínál modern és stílusos termékeket. A rendszer támogatja a webshop működéséhez szükséges legfontosabb folyamatokat, beleértve a termékek, kategóriák, készletek és árak kezelését, valamint az online rendelések lebonyolítását.

A felhasználók könnyedén böngészhetnek az aktuálisan elérhető ruhák között, kiválaszthatják a számukra megfelelő méretet, színt és stílust, majd egyszerűen leadhatják rendelésüket. Emellett a rendszer adminisztrációs felülettel is rendelkezik, ahol kezelhetők a termékek, készletek, árak, rendelések és vásárlói adatok.

Az alkalmazás elsősorban az online kereskedelem és a divat értékesítés területén használható. Különösen hasznos olyan környezetben, ahol fontos a termékek központi nyilvántartása, a készletkezelés, a rendelések nyomon követése, valamint az adminisztratív és vásárlói folyamatok hatékony és elkülönített kezelése.

1.2 Célok és célkitűzések

A projekt fő célja egy korszerű, több platformon is használható webshop rendszer létrehozása, amely egységes backend logikára épül. A rendszernek ki kell szolgálnia a vásárlói rendelési folyamatokat, valamint az adminisztratív adatkezelési feladatokat is.

Rövid távon a cél a webshop alapvető működésének megvalósítása volt. Ide tartozik a termékek, kategóriák, készletek, kosarak és rendelések adatmodelljének kialakítása, egy működő backend API elkészítése, valamint a webes felület létrehozása. Fontos cél volt továbbá, hogy a felhasználók képesek legyenek termékeket keresni, kiválasztani, kosárba helyezni, majd rendelésüket véglegesíteni.

Hosszabb távon a rendszer továbbfejleszthető további kényelmi és adminisztratív funkciókkal. Ilyen lehet például az online fizetési rendszerek integrálása, részletesebb jogosultságkezelés, statisztikai admin felület, fejlettebb naplózás, valamint egy modernebb és egységesebb frontend architektúra kialakítása.

2. Felhasználói dokumentáció

2.1 Ismertető

2.1.1 Felhasználói szerepkörök

A projektben több szerepkör is megkülönböztethető. A szerepkörök eltérő jogosultságokkal rendelkeznek, ezért más-más funkciók érhetők el számukra.¹²²

2.1.1.1 Vendég Felhasználó

A vendég felhasználó olyan látogató, aki még nem jelentkezett be a rendszerbe. Számára a webshop nyilvános tartalmai érhetők el.

Jogosultságai:

- ruhák böngészése
- termékek keresése név vagy egyéb szűrők alapján
- kategóriák megtekintése
- termékadatok megtekintése
- regisztráció és bejelentkezés indítása

2.1.1.2 Regisztrált felhasználó

A regisztrált felhasználó bejelentkezést követően további, személyre szabott funkciókhoz is hozzáfér.

Jogosultságai:

- ruhák böngészése és keresése
- termékek kiválasztása
- kosár használata
- rendelés leadása
- korábbi rendelések megtekintése
- profiladatok kezelése
- jelszó módosítása
- kijelentkezés

2.1.1.3 Adminisztrátor

Az adminisztrátor emelt jogosultságokkal rendelkezik, és a rendszer adatainak karbantartásáért, valamint az operatív működés irányításáért felel.

Jogosultságai:

- ruhák (termékek) kezelése
- kategóriák kezelése
- nemek (gender) kezelése
- felhasználók kezelése
- rendelések kezelése
- rendelési státuszok módosítása (pl. folyamatban, teljesítve)
- admin jogosultságok kiosztása
- bevételi adatok megtekintése

2.1.2 Főbb funkciók

Az alkalmazás több egymásra épülő modulból áll, amelyek együtt biztosítják az online ruhavásárlási folyamat teljes működését.

2.1.2.1 Ruhák kezelése

Lehetővé teszi a ruhák adatainak tárolását, megjelenítését és kezelését. A felhasználók megtekinthetik az elérhető termékeket és azok részleteit.

Főbb lehetőségek:

- termékek listázása
- termékek keresése
- termékek szűrése (pl. kategória alapján)
- termékadatok megjelenítése

2.1.2.2 Rendelési rendszer

Biztosítja a teljes vásárlási folyamatot a termék kiválasztásától a rendelés véglegesítéséig

Főbb lehetőségek:

- termék kiválasztása
- kosárba helyezés
- rendelés leadása (checkout)
- rendelési visszaigazolás
- rendelési előzmények megtekintése

2.1.2.3 Kosárkezelés

A kiválasztott termékek ideiglenesen a kosárba kerülnek, ahol a felhasználó módosíthatja azokat.

Főbb lehetőségek:

- kosár tartalmának megtekintése
- mennyiség módosítása
- termékek eltávolítása
- végösszeg megjelenítése

2.1.2.4 Felhasználókezelés

Biztosítja a felhasználói fiókok kezelését és a biztonságos bejelentkezést.

Főbb lehetőségek:

- regisztráció
- bejelentkezés (felhasználó és admin)
- kijelentkezés
- profil adatok kezelése
- jelszó módosítása
- aktuális felhasználó adatainak lekérdezése

2.1.2.5 Kategória- és attribútumkezelés

Lehetővé teszi a termékek rendszerezését és szűrését különböző szempontok alapján.

Főbb lehetőségek:

- kategóriák kezelése
- nemek (gender) kezelése
- szűrési feltételek biztosítása a felhasználók számára

2.1.2.5 Adminisztrációs modul

Külön adminisztrációs felületet biztosít az adatok karbantartásához.

Főbb lehetőségek:

- termékek létrehozása, módosítása, törlése
- kategóriák kezelése
- felhasználók kezelése
- rendelések kezelése
- rendelési státuszok frissítése
- bevételi statisztikák lekérdezése

2.1.2.6 Mobilalkalmazás támogatás

Avalonia alapú mobilalkalmazás is tartozik a rendszerhez, amely adminisztrációs feladatok végrehajtására használható.

Főbb lehetőségek:

- backend adatok kezelése
- admin műveletek végrehajtása
- gyorsabb kezelői munkavégzés
- többplatformos használat

2.2 Célközönség

Az alkalmazás elsődleges célközönségét azok a felhasználók alkotják, akik online szeretnének ruhákat vásárolni, böngészni az aktuális kínálatot, valamint kiválasztani a számukra megfelelő termékeket méret, stílus vagy kategória alapján.

A rendszer emellett a webshopot üzemeltető adminisztrátorok számára is készült, akik a termékek, kategóriák, felhasználók és rendelések kezelését végzik. Számukra egy adminisztrációs felület áll rendelkezésre, amely lehetővé teszi az adatok gyors és hatékony kezelését, valamint a rendelések nyomon követését és feldolgozását.

2.3 Megvalósítás eszközei

A rendszer fejlesztése során több korszerű programozási nyelv és technológia került felhasználásra, amelyek együtt biztosítják az alkalmazás teljes működését.

2.3.1 Backend fejlesztés

A szerveroldali rész C# programozási nyelven, .NET 8 keretrendszer használatával készült. A backend ASP.NET Core Web API alapokra épül, amely a kliensalkalmazások számára biztosítja az adatok elérését és az üzleti logikát.

Felhasznált technológiák:

- C# 12
- .NET 8
- ASP.NET Core Web API
- Entity Framework Core 9
- Swagger / OpenAPI

2.3.2 Adatbázis

Az adatok tárolására PostgreSQL relációs adatbázis-kezelő rendszer szolgál.

Felhasznált verzió:

- PostgreSQL 16 / 17 (telepített környezettől függően)

2.3.3 Webes felület

A felhasználói webes felület szabványos webes technológiákkal készült.

Felhasznált nyelvek és eszközök:

- HTML5
- CSS3
- JavaScript
- TypeScript
- Bootstrap 5.3.8

A Bootstrap biztosítja a reszponzív, modern megjelenést, így az oldal különböző képernyőméreteken is megfelelően használható.

2.3.4 Mobilalkalmazás

A projekt mobilalkalmazása Avalonia UI technológiával készült. Az Avalonia lehetőséget biztosít arra, hogy .NET alapokon, modern felhasználói felületű alkalmazás készüljön több platform támogatásával.

Felhasznált technológiák:

- C#
- .NET 8
- Avalonia UI 11.3.9
- MVVM architektúra

2.3.5 Fejlesztői környezet és eszközök

A projekt fejlesztése során az alábbi eszközök kerültek használatra:

- Visual Studio 2022
- Git
- GitHub
- pgAdmin 4
- xUnit

2.4. Rendszerkövetelmények

Komponens	Minimum követelmény	Ajánlott követelmény
Backend szerver	Windows 10/11, Linux vagy macOS; .NET 8 SDK; PostgreSQL	Windows 11 vagy Linux szerver; .NET 8 SDK; PostgreSQL 15+
Webes frontend	Modern böngésző és internetkapcsolat a CDN-ekhez	Chrome, Edge vagy Firefox aktuális verzió
Avalonia kliens	Windows 10/11, Linux vagy macOS; .NET futtatókörnyezet	Windows 11 + .NET 8 Desktop Runtime
Processzor	2 magos CPU	4 magos CPU vagy jobb
Memória	4 GB RAM	8 GB RAM vagy több
Tárhely	1 GB szabad hely a projektnek és csomagoknak	2-5 GB szabad hely adatbázissal és tesztekkel együtt

2.5 Futtatás

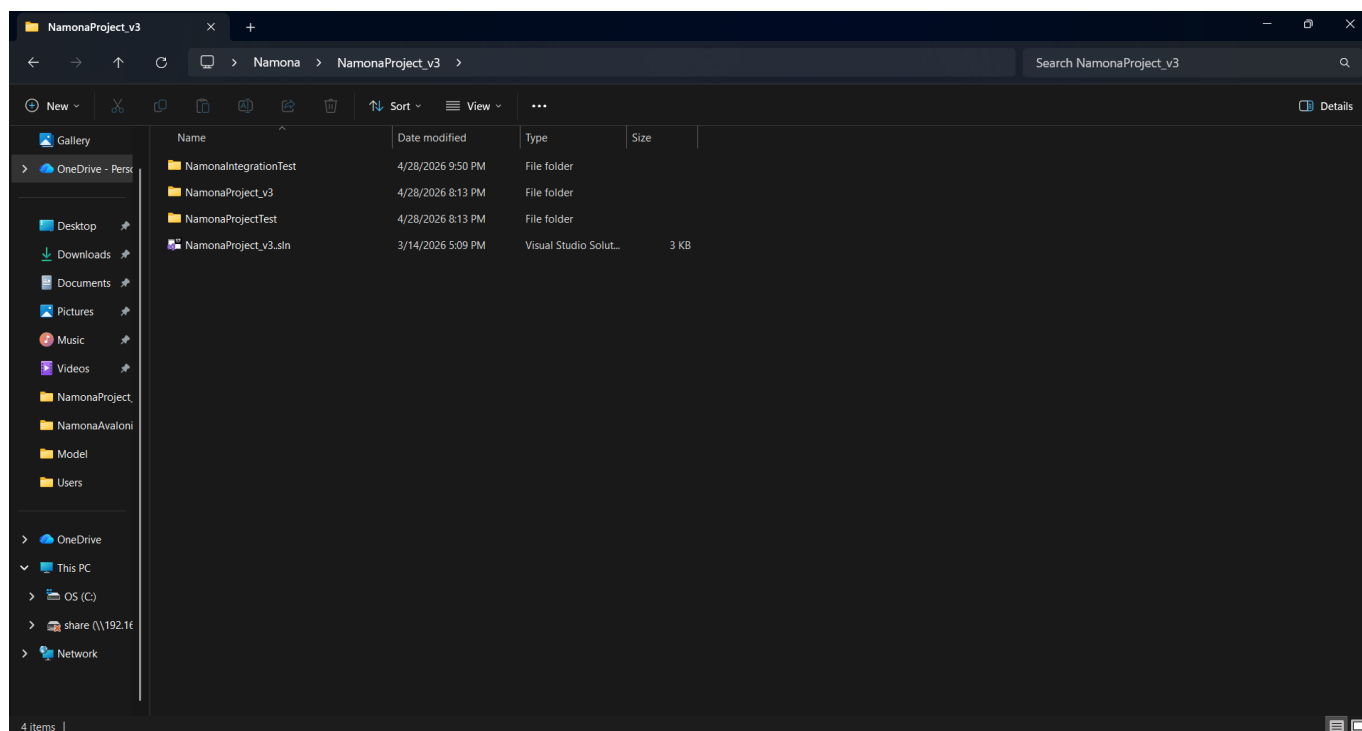
A Namona projekt működéséhez elengedhetetlen a háttérrendszer, vagyis a Web API megfelelő futtatása. Az alkalmazás egy ASP.NET Core alapú Web API-ra épül, amely a rendszer központi eleme — ez felelős az összes adat kezeléséért, a businesslogika végrehajtásáért, valamint a kommunikáció biztosításáért a különböző kliensek között.

A rendszer két különböző felületen érhető el: egy böngészőben futó weboldalon, valamint egy Avalonia alapú asztali alkalmazáson keresztül. Mindkét felület kizárólag a Web API-n keresztül kommunikál az adatbázissal és végzi el a szükséges műveleteket. Ez azt jelenti, hogy amennyiben a Web API nem fut, sem a weboldal, sem az asztali alkalmazás nem lesz képes adatokat lekérni, módosítani vagy bármilyen funkciót elvégezni.

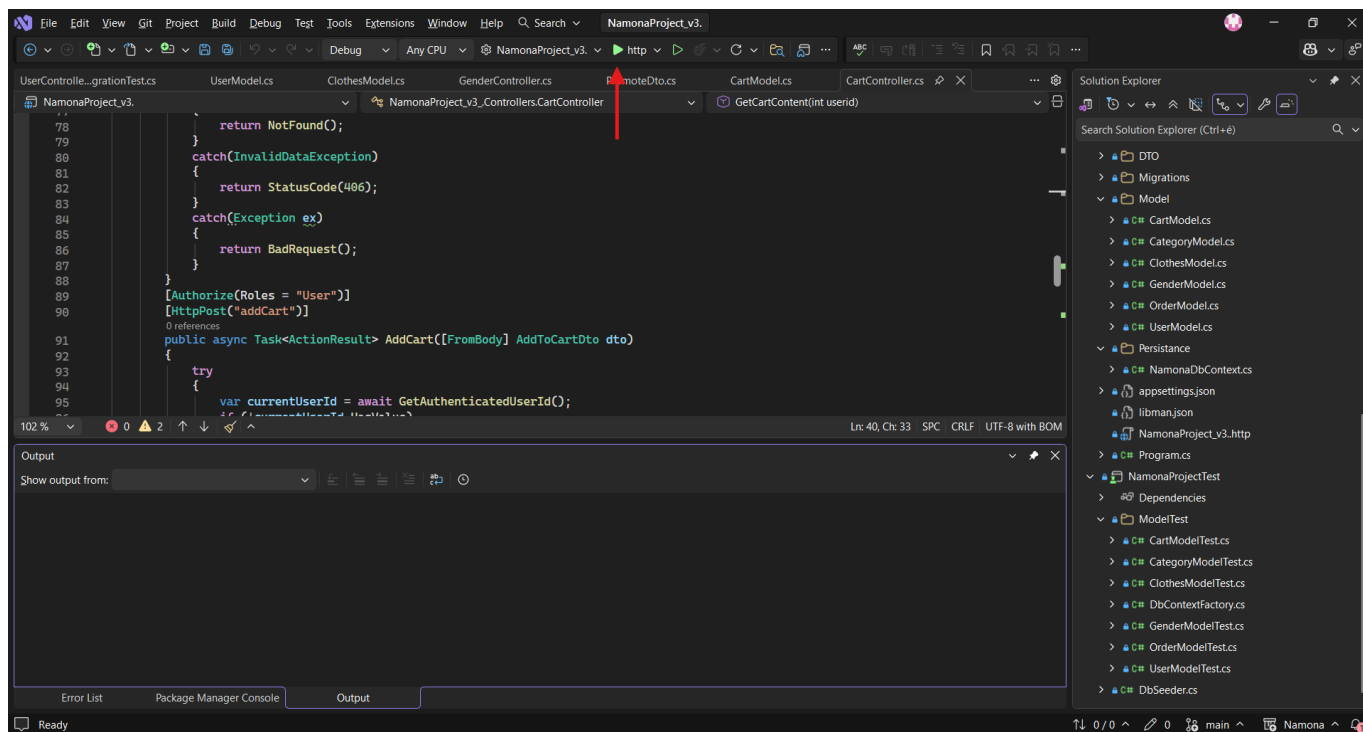
A Web API elindításához szükséges a háttérrendszer futtatása, amely alapértelmezetten a helyi gépen, a megadott porton érhető el. Fontos, hogy az alkalmazás használata előtt mindig győződjünk meg arról, hogy a szerver elindult és megfelelően működik. Enélkül a felhasználói felületek nem tudnak csatlakozni a rendszerhez, és hibaüzeneteket fognak megjeleníteni.

Röviden összefoglalva: a Web API a projekt szíve — nélküle az alkalmazás egyetlen része sem működik.

A Web Api Futtatásához meg kell nyitnunk a NamonaProject_v3 nevű mappát amiben megtalálható a NamonaProject_v3 nevű .sln file.



Ha megnyitottuk a File-t a VisualStudio-ban az alkalmazás tetejében lévő fejlécben megtaláljuk a futtatás gombot ami egy zöld egy “http” szöveggel mellette.

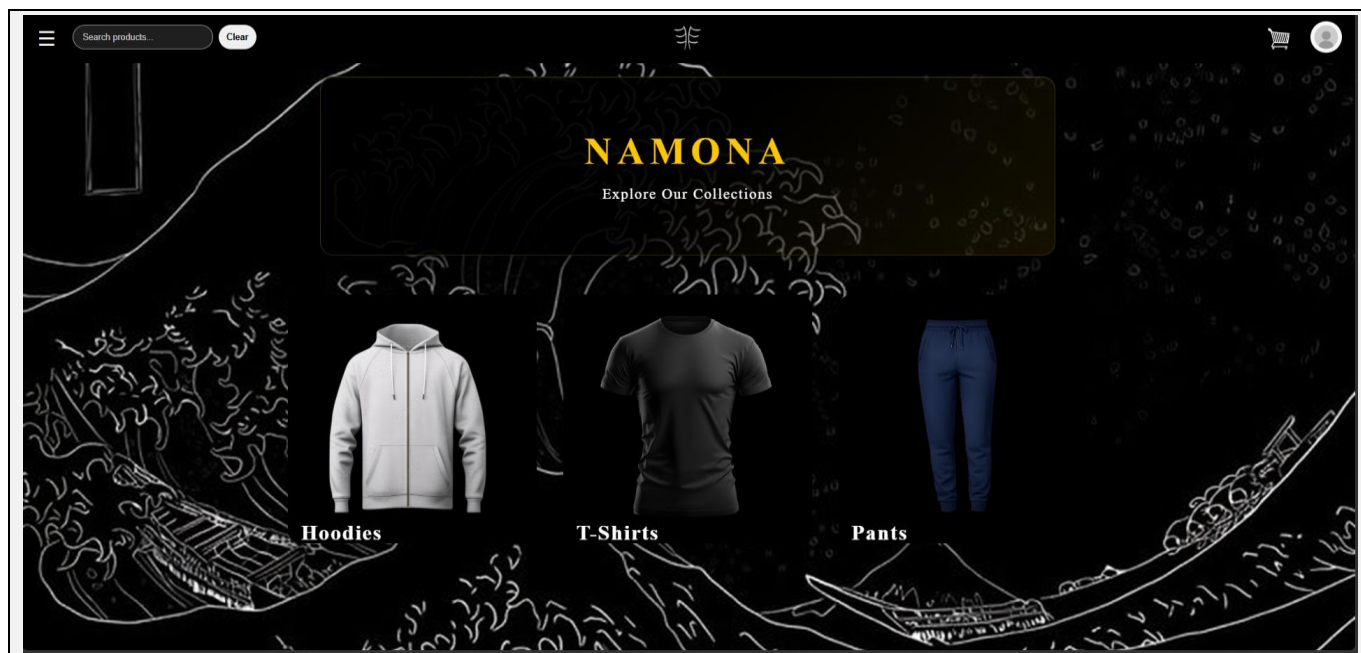


Ha az alábbi műveletet elvégeztük, a WEB API sikeresen elindul, így elérhetővé és funkcionálissá válik mind a felhasználók részére, mind az adminisztrátorok részére a szolgáltatások amiket a projekt nyújt.

2.6 Program használata képernyőképenként

2.6.1 Webes felhasználói felület

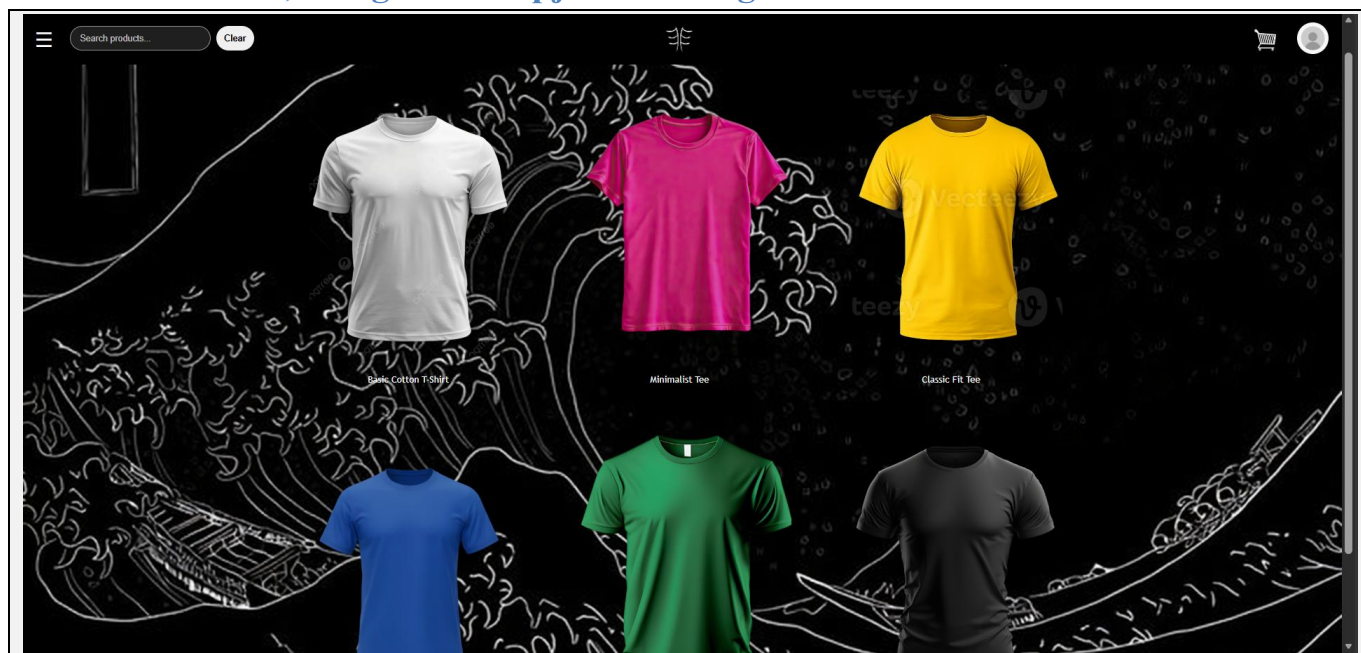
2.6.1.1 Kezdőoldal



Leírás: A felhasználó ezen az oldalon megtekinti a kínálatban lévő ruhákat kategória alapján.

Elérhető műveletek: Ruhák keresése (akár jellemzők alapján), Navigáció, Profil-menü.

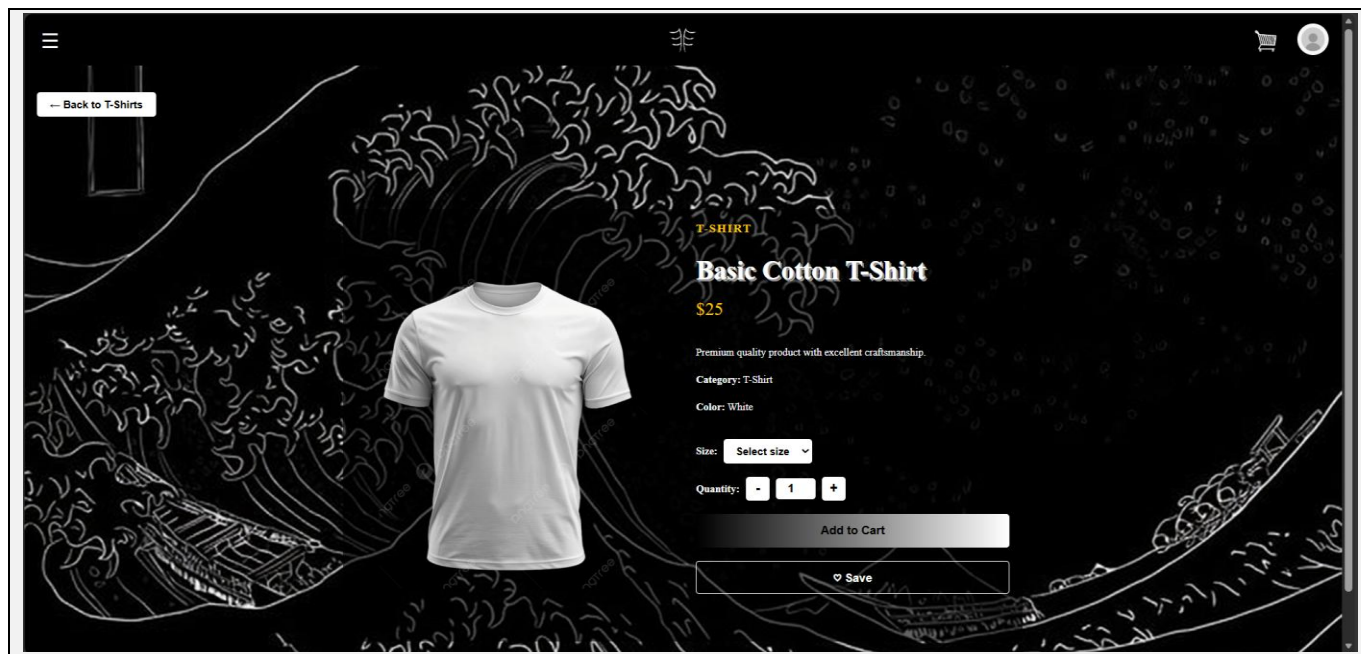
2.6.1.2 Termékek, kategóriák alapján való megtekintése



Leírás: A felhasználó megtekintheti kategóriákban tárolt termékeket.

Elérhető műveletek: Ruhák keresése (akár jellemzők alapján), Navigáció, Profil-menü, termékek megtekintése.

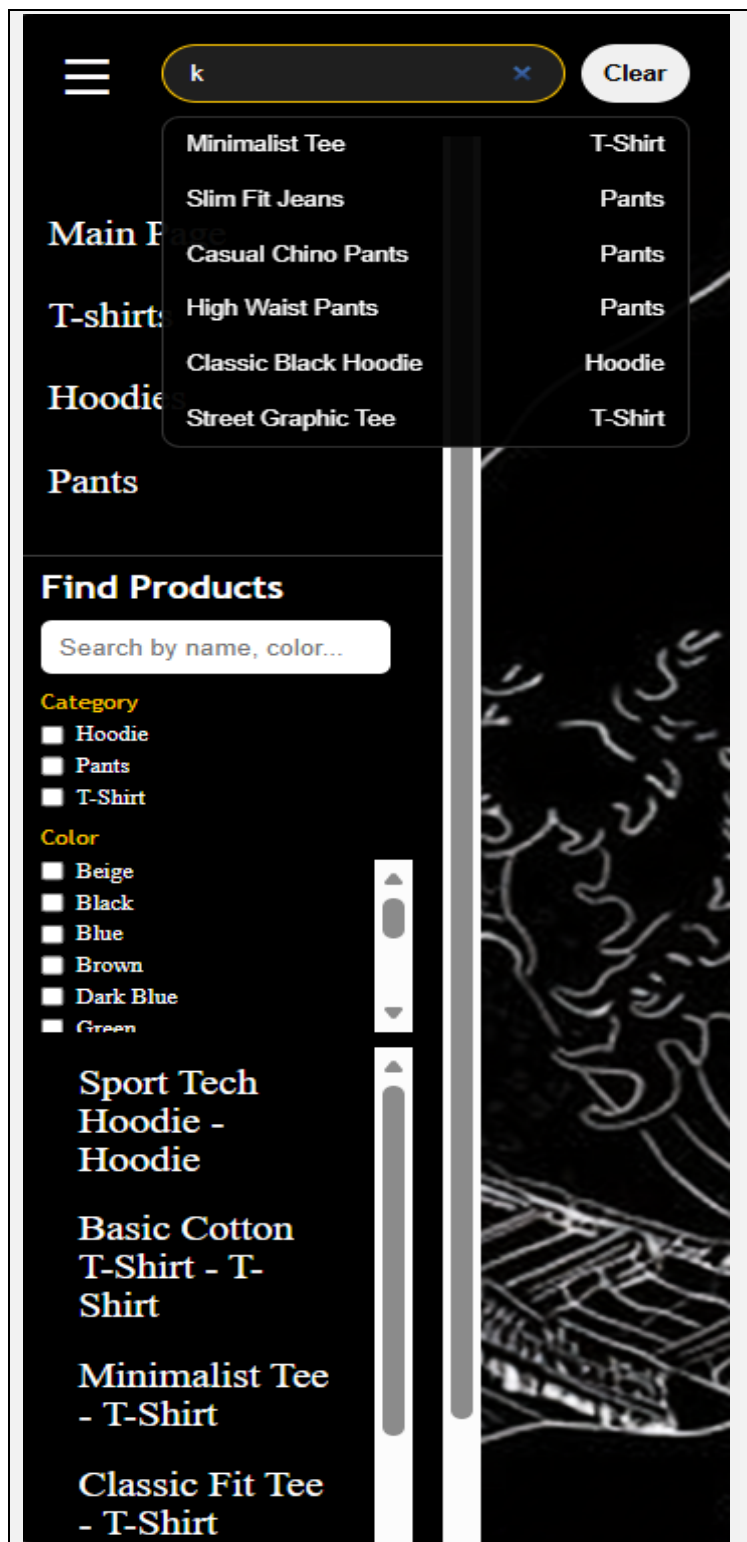
2.6.1.3 Termékek



Leírás: A felhasználó megtekintheti a termékeket egy külön, önálló oldalon, ha be van jelentkezve akár vásárolhat is.

Elérhető műveletek: Ruhák keresése (akár jellemzők alapján), Navigáció, Profil-menü, termék mentése, kosárba helyezése, mennyiség, méret kiválasztása, vissza gomb a kategóriák alapján megjelenítős oldalra.

2.6.1.4 Keresés és szűrés



Leírás: A felhasználó szín, név, kategória lapján tud szűrést, vagy keresést végrehajtani.

Elérhető műveletek: Keresőmező használata, szűrés checkboxokkal.

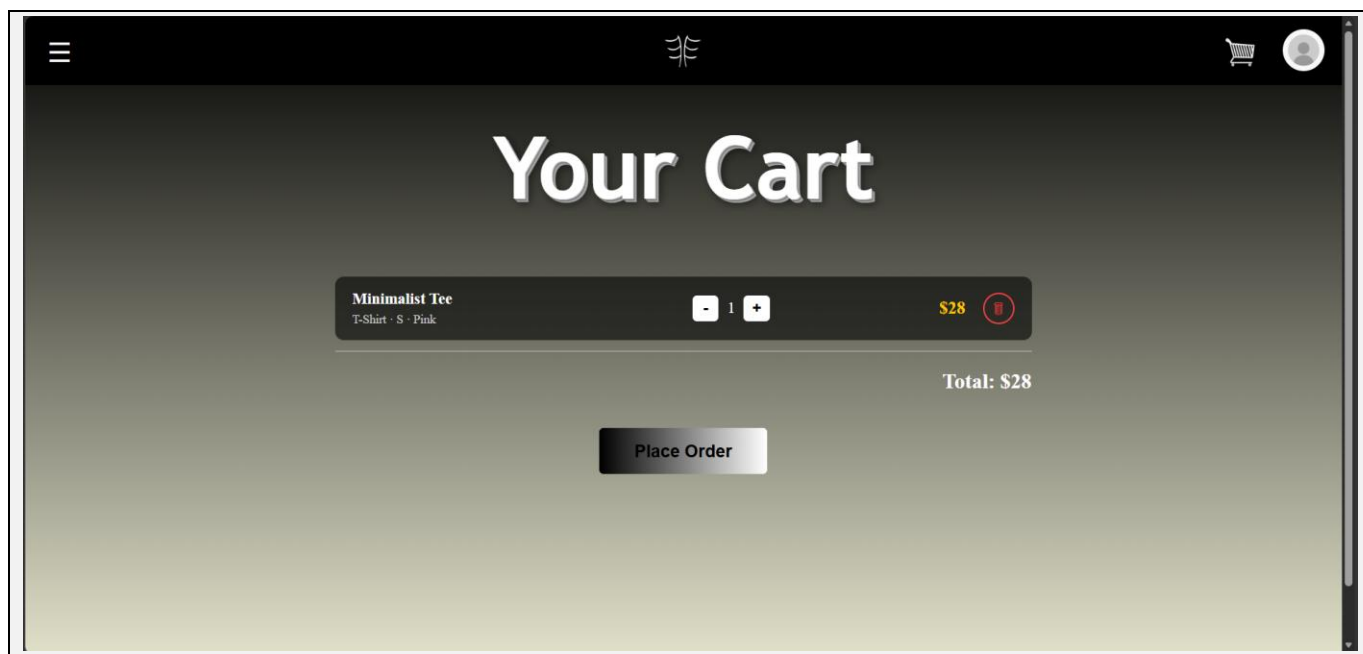
2.6.1.5 Rólunk oldal



Leírás: Hozzáférhető a profil-menüből.

Elérhető műveletek: A cég Instagram oldalára való navigálása, ha az instagram ikonra kattintunk.

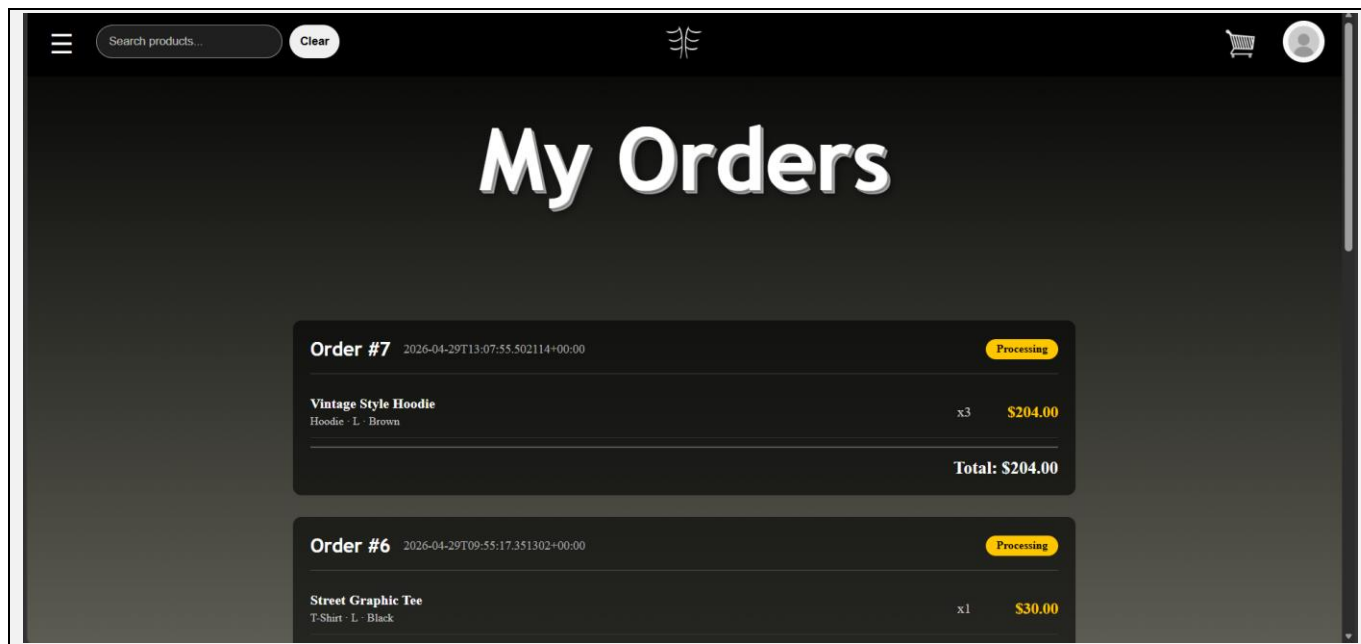
2.6.1.6 Kosár



Leírás: A felhasználó a kosárba rakott termékeit tekintheti meg, rendelheti meg, illetve törölheti azt.

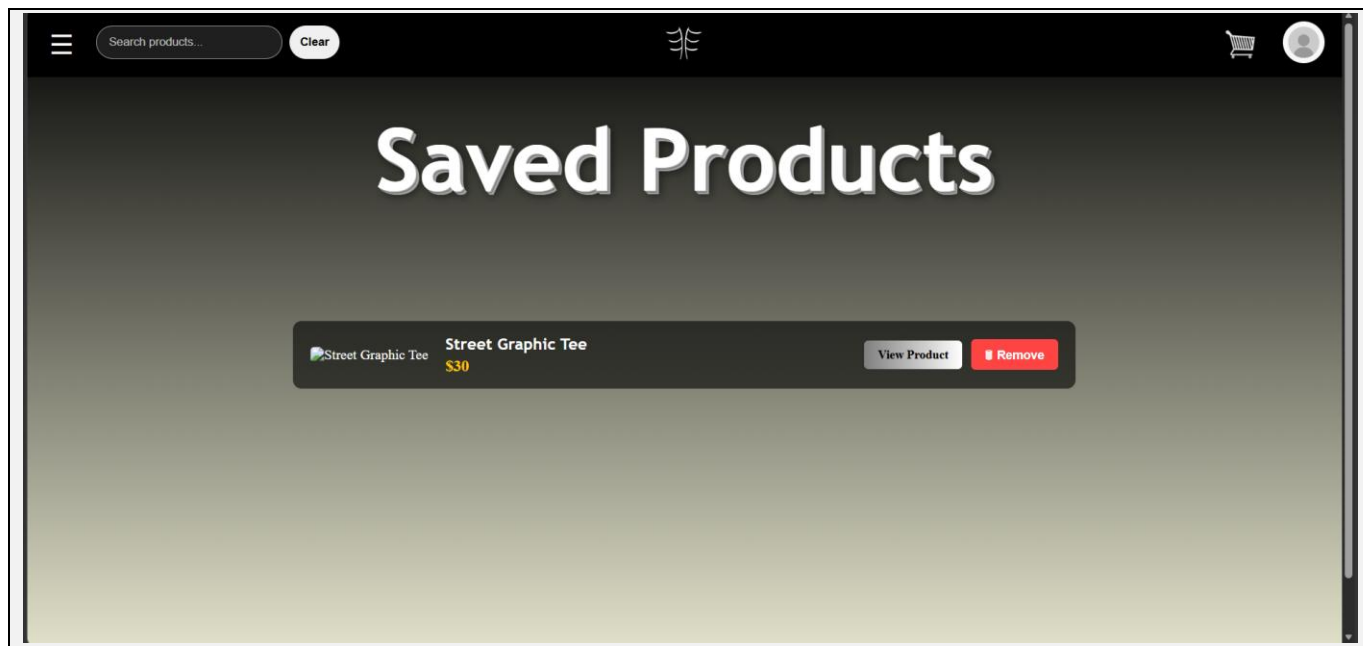
Elérhető műveletek: Mennyiség megváltoztatása, termék törlése a kosárból, végösszeg, rendelés.

2.6.1.7 Rendeléseim



Leírás: Korábbi rendelések megtekintése, függőben lévő megrendelések nyomonkövetése.

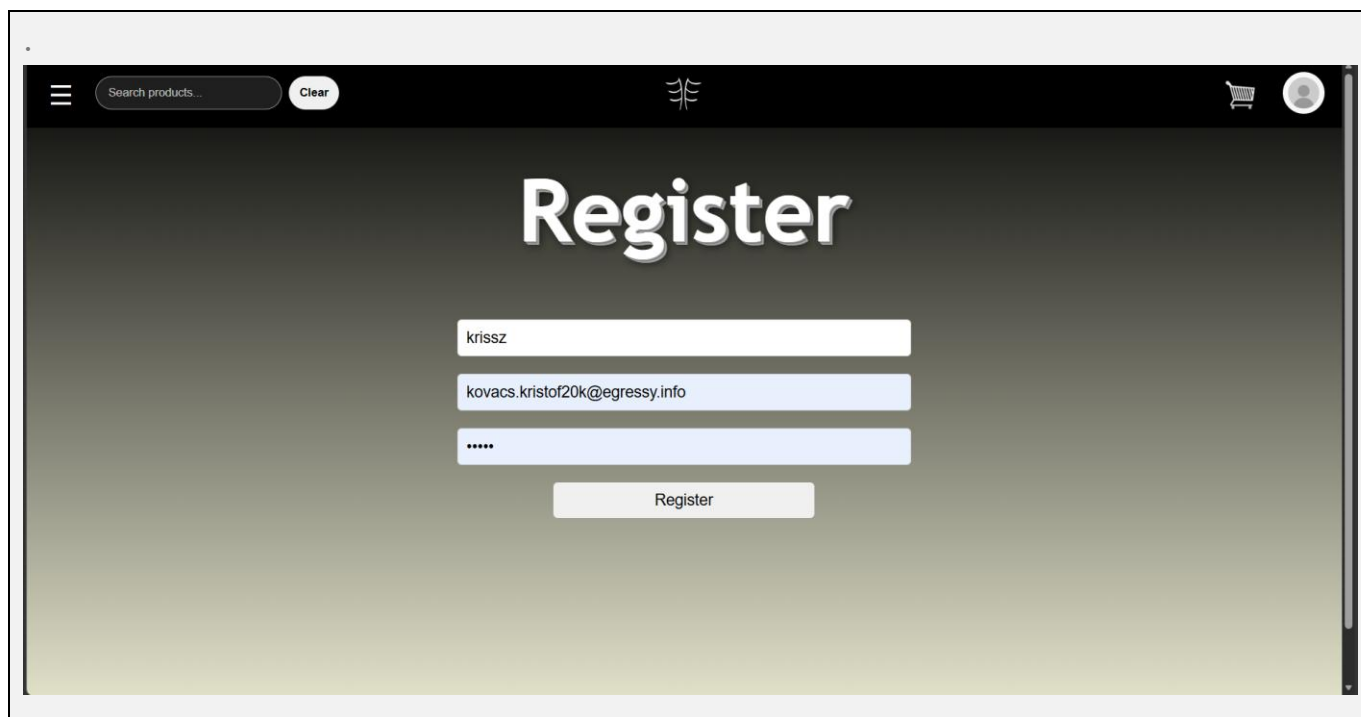
2.6.1.8 Mentett termékek



Leírás: Mentett termékeink megtekintése.

Elérhető műveletek: Mentett termékek megtekintése, a termékoldalra való navigálás, törlés a mentettek közül.

2.6.1.9 Regisztráció

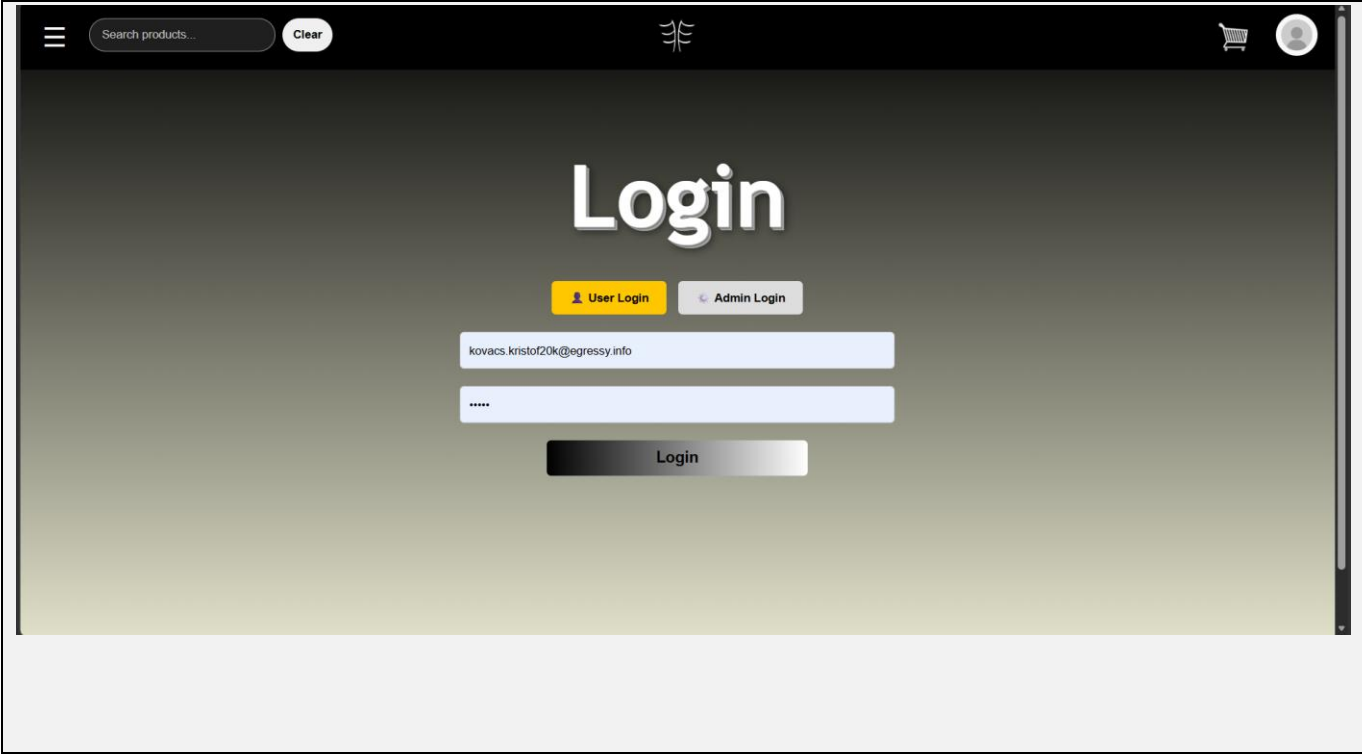


The screenshot shows a web browser window displaying the 'Register' page. The page has a dark background with a light green gradient at the bottom. At the top, there is a navigation bar with a menu icon, a search bar containing 'Search products...', a 'Clear' button, a logo, a shopping cart icon, and a user profile icon. The main content area features the word 'Register' in large, white, bold letters. Below this, there are three input fields: the first contains 'krissz', the second contains 'kovacs.kristof20k@egressy.info', and the third contains masked characters '*****'. A 'Register' button is positioned below the input fields.

Leírás: Felhasználói profil regisztrálása.

Elérhető műveletek: Felhasználói profil regisztrálása.

2.6.1.10 Felhasználói Bejelentkezés



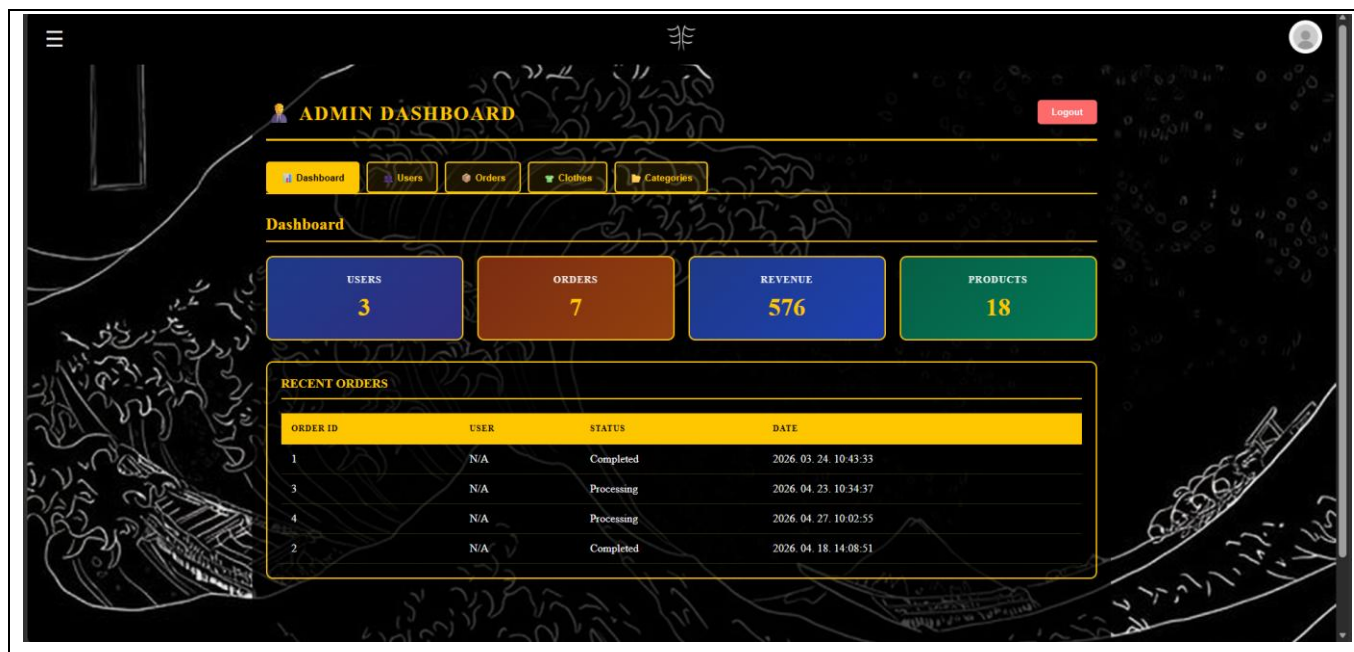
The screenshot shows a web application's login interface. At the top, a dark navigation bar contains a hamburger menu, a search bar with the text 'Search products...', a 'Clear' button, a logo, and icons for a shopping cart and a user profile. The main area has a dark background with the word 'Login' in a large, white, serif font. Below this, there are two buttons: 'User Login' in yellow and 'Admin Login' in gray. Underneath these are two white input fields. The first field contains the email address 'kovacs.kristof20k@egressy.info'. The second field contains masked characters '*****'. At the bottom of the form is a dark 'Login' button.

Leírás: Regisztrált fiókunk adataival való bejelentkezés, ezzel hozzáférhető lesz a termék kosárhoz adásának funkciója, a rendelés, a termék mentése funkció és a rendelések megtekintése.

Elérhető műveletek: Bejelentkezés a profilunkba.

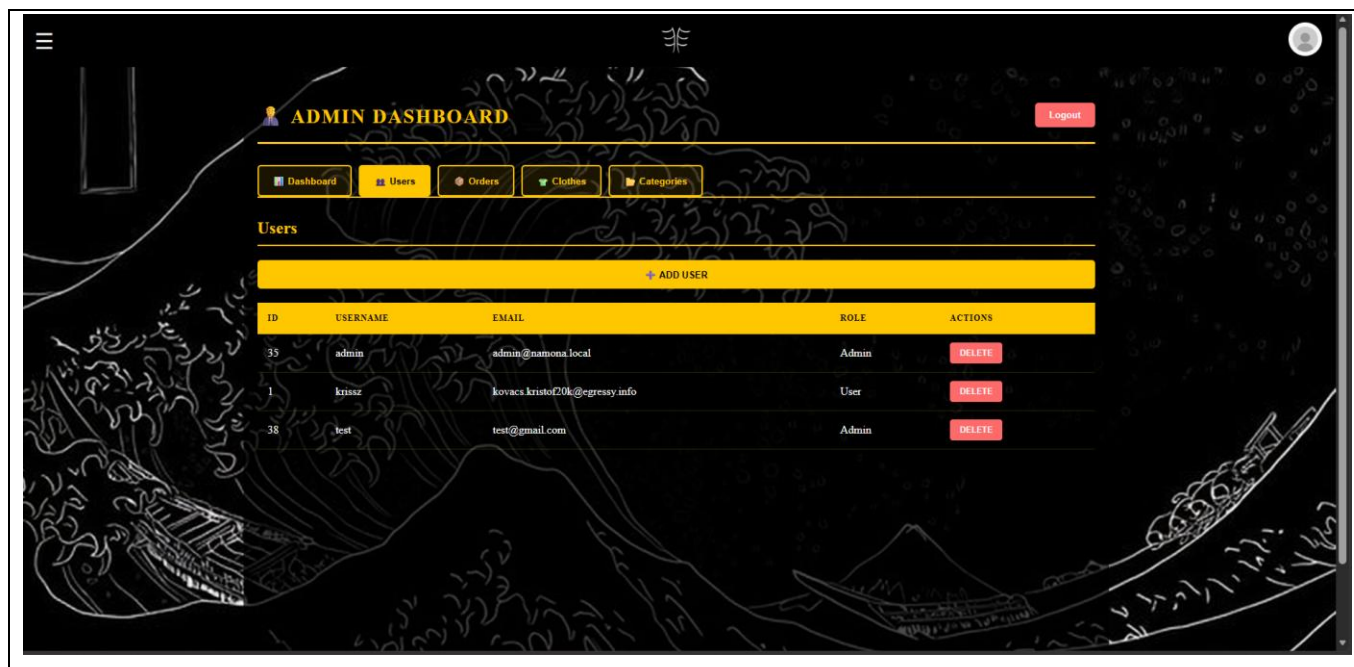
2.6.2 Webes admin felület

2.6.2.1 Áttekintési oldal



Leírás: Az admin megtekintheti a főbb adatokat felhasználóinkról.

2.6.2.2 Felhasználók kezelése



Leírás: Az admin itt tudja megtekinteni a regisztrált felhasználók listáját, szerepét és törölheti ezeket.

2.6.2.3 Rendelések áttekintése

ORDER ID	USER	STATUS	TOTAL PRICE	DATE	ACTIONS
1	N/A	Completed	\$0	2026. 03. 24. 10:43:33	COMPLETE DELETE
3	N/A	Processing	\$0	2026. 04. 23. 10:34:37	COMPLETE DELETE
4	N/A	Processing	\$0	2026. 04. 27. 10:02:55	COMPLETE DELETE
2	N/A	Completed	\$0	2026. 04. 18. 14:08:51	COMPLETE DELETE
5	N/A	Processing	\$0	2026. 04. 29. 11:54:59	COMPLETE DELETE
6	N/A	Processing	\$0	2026. 04. 29. 11:55:17	COMPLETE DELETE
7	N/A	Processing	\$0	2026. 04. 29. 15:07:55	COMPLETE DELETE

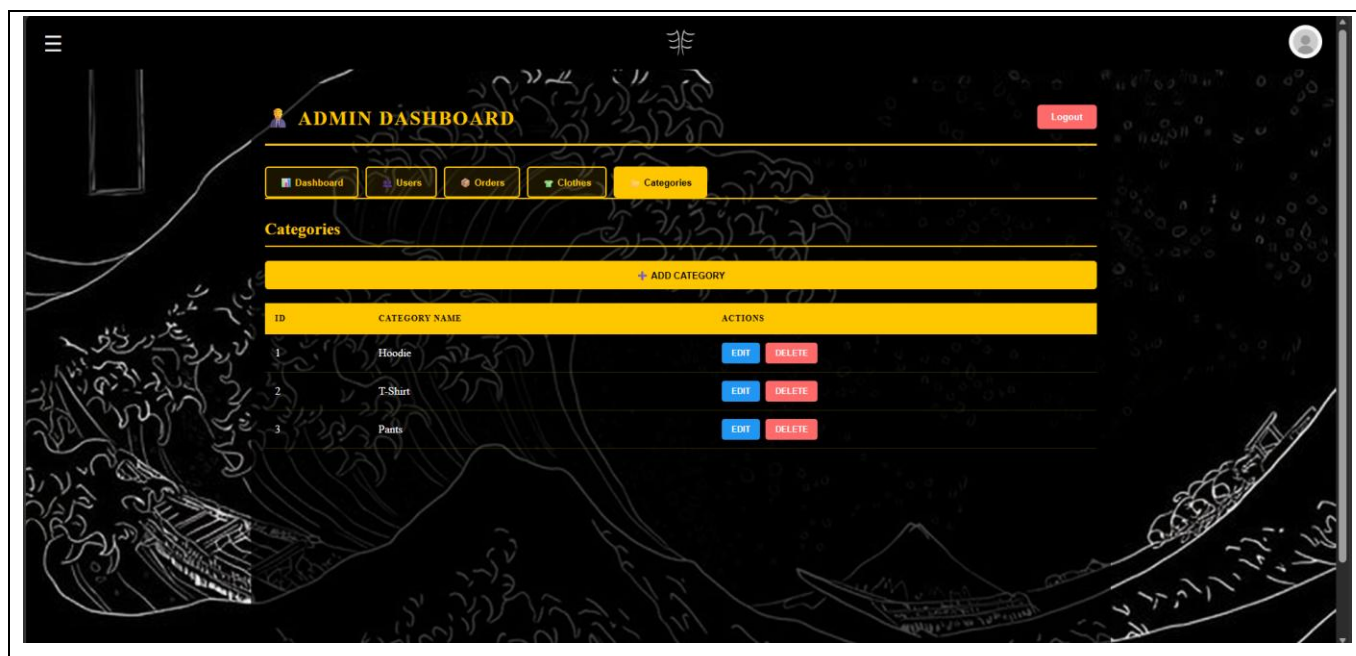
Leírás: Az admin áttekintheti a rendeléseket, törölheti őket, vagy lezártnak jelölheti.

2.6.2.4 Termékek kezelése

ID	NAME	CATEGORY	COLOR	SIZE	STOCK	PRICE	ACTIONS
5	Sport Tech Hoodie	Hoodie	Blue	M	50	\$80	EDIT DELETE
7	Basic Cotton T-Shirt	T-Shirt	White	M	100	\$25	EDIT DELETE
9	Minimalist Tee	T-Shirt	Pink	S	80	\$28	EDIT DELETE
10	Classic Fit Tee	T-Shirt	Yellow	M	70	\$26	EDIT DELETE
11	Sport Performance Tee	T-Shirt	Blue	XL	60	\$32	EDIT DELETE
12	Vintage Logo Tee	T-Shirt	Green	L	75	\$29	EDIT DELETE
13	Slim Fit Jeans	Pants	Dark Blue	M	45	\$90	EDIT DELETE
14	Casual Chino Pants	Pants	Khaki	L	50	\$85	EDIT DELETE

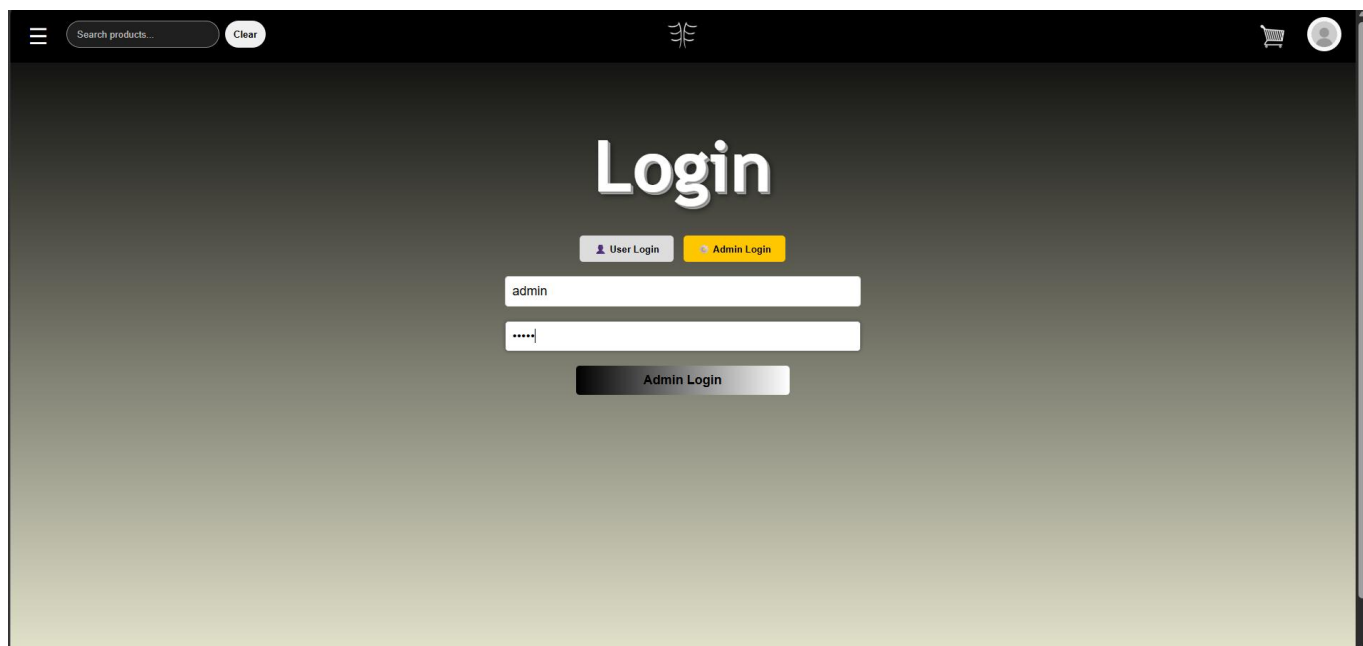
Leírás: A ruhák adatai itt kezelhetők, módosítani lehet minden egyes adatukat vagy törölni a termékeket.

2.6.2.5 Kategóriák kezelése



Leírás: A kategóriákat itt lehet hozzáadni, módosítani vagy törölni.

2.6.2.6 Admin bejelentkezés

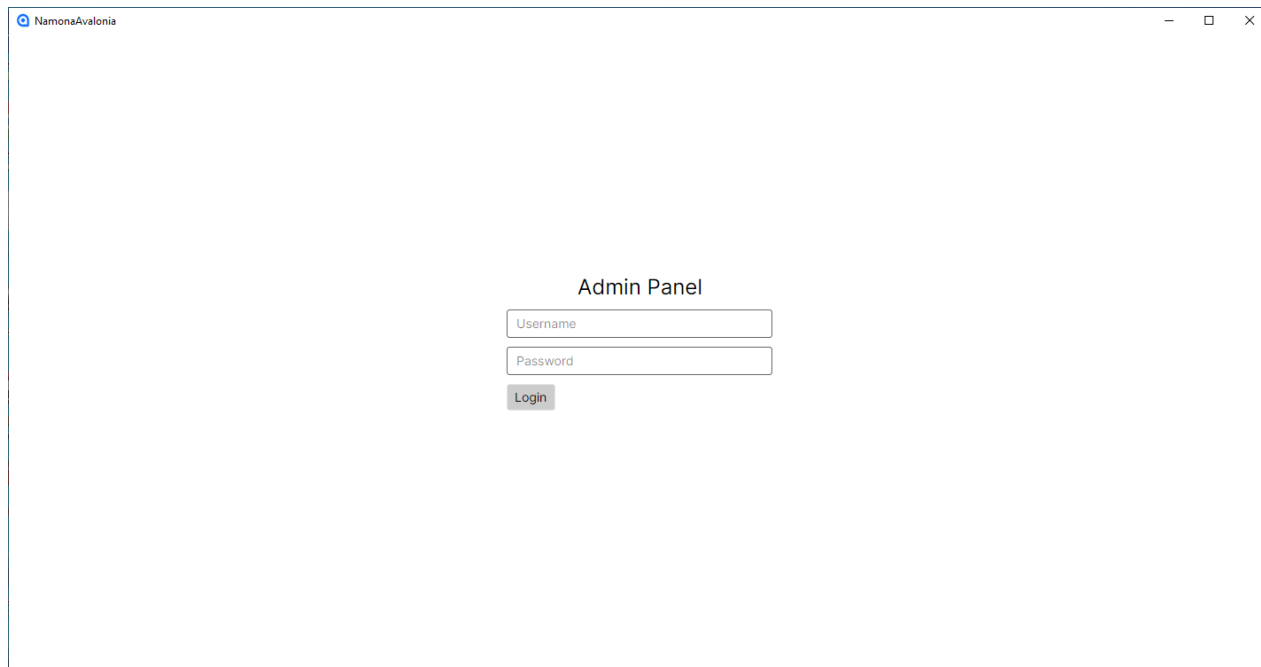


Leírás: Az adatbázisban előre létrehozott és tárolt admin felhasználónév és jelszó alapján való bejelentkezés a kezelőfelületbe.

2.6.3 Avalonia alkalmazás használata képernyőképenként

1.5.1 Avalonia bejelentkezés

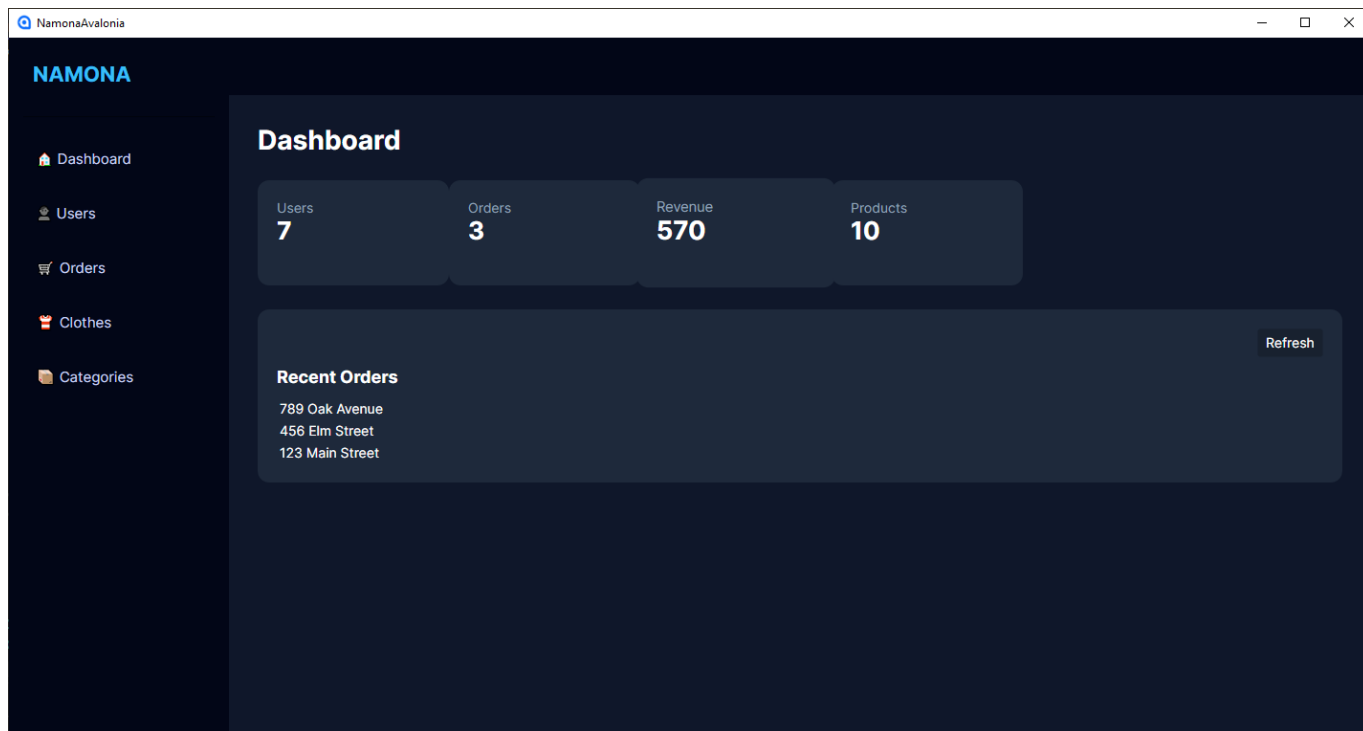
Leírás: Az admin bejelentkezik a backend API-n keresztül.



The screenshot shows a web application window titled "NamonaAvalonia". In the center of the window is a login form titled "Admin Panel". The form contains two input fields: "Username" and "Password", both with placeholder text. Below these fields is a "Login" button. The window has standard Windows-style controls (minimize, maximize, close) in the top right corner.

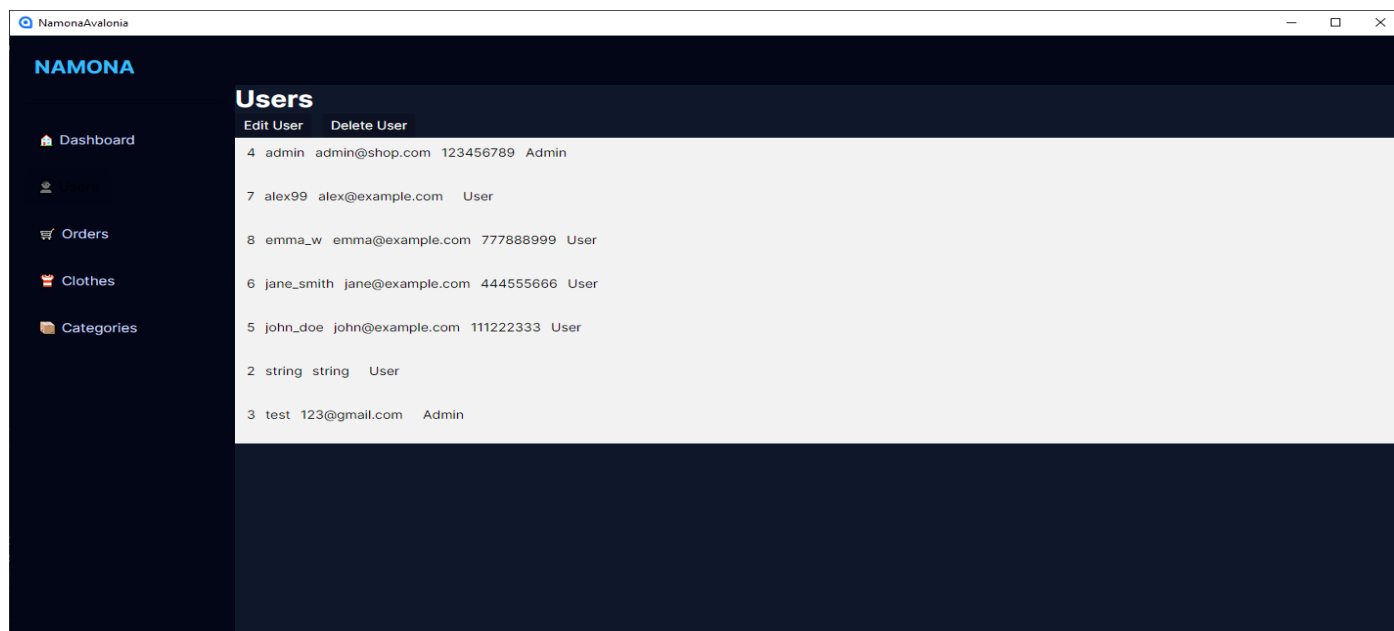
1.5.2 Főablak / navigáció

Leírás: A főablakból érhetőek el az adminisztratív modulok valamint a felhasználók száma, a rendelések száma, az eddigi bevétel, az összes termék és az 5 legújabb rendelés.



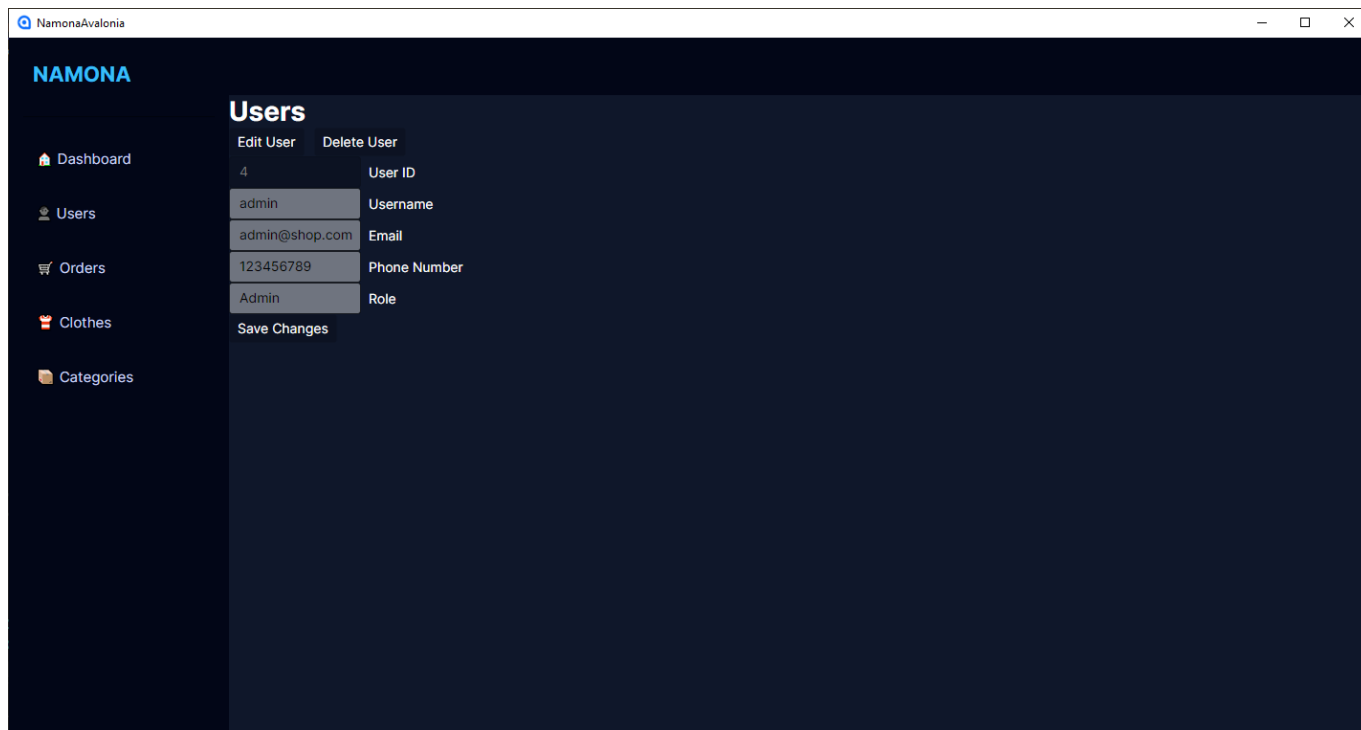
1.5.3 Felhasználó Kezelés

Leírás: A felhasználók listázása



1.5.3 Felhasználó Kezelés → Módosítás

Leírás: Felhasználók módosítása.



NamonaAvalonia

NAMONA

Users

Edit User Delete User

4 User ID

admin Username

admin@shop.com Email

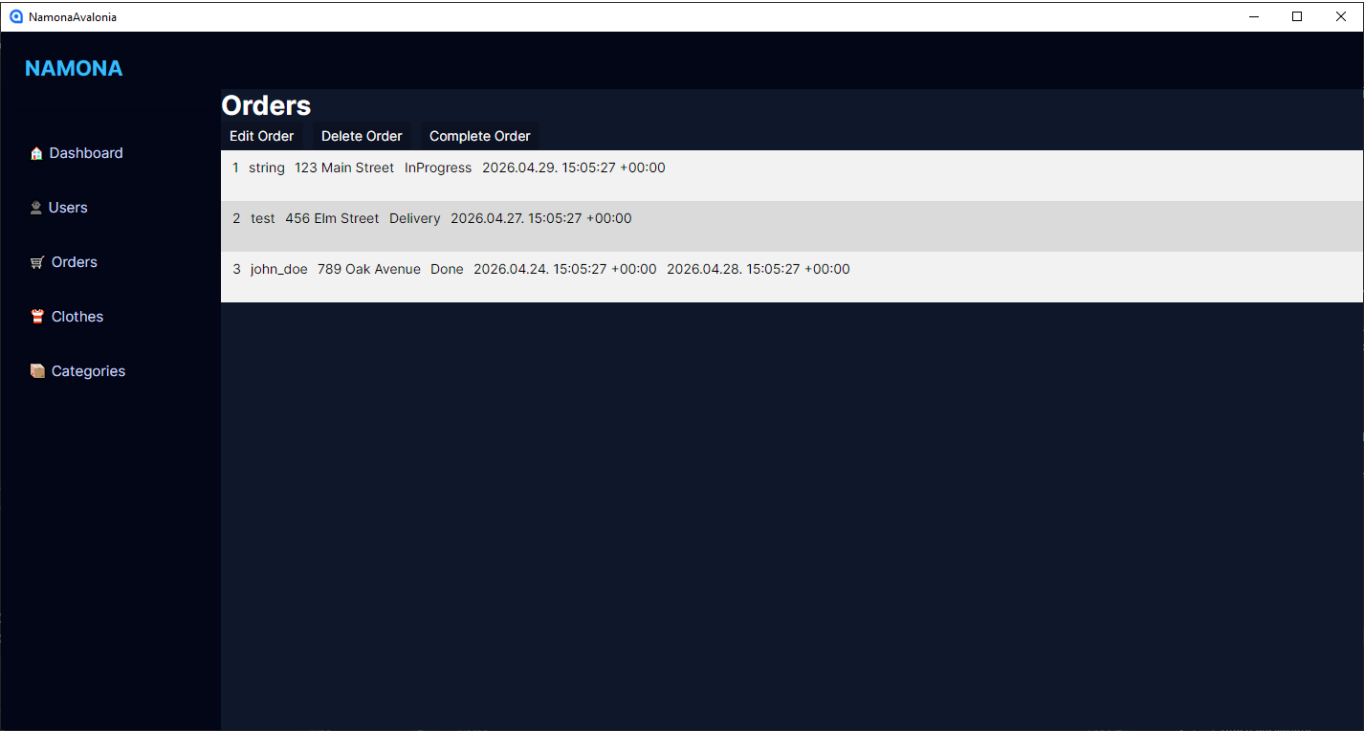
123456789 Phone Number

Admin Role

Save Changes

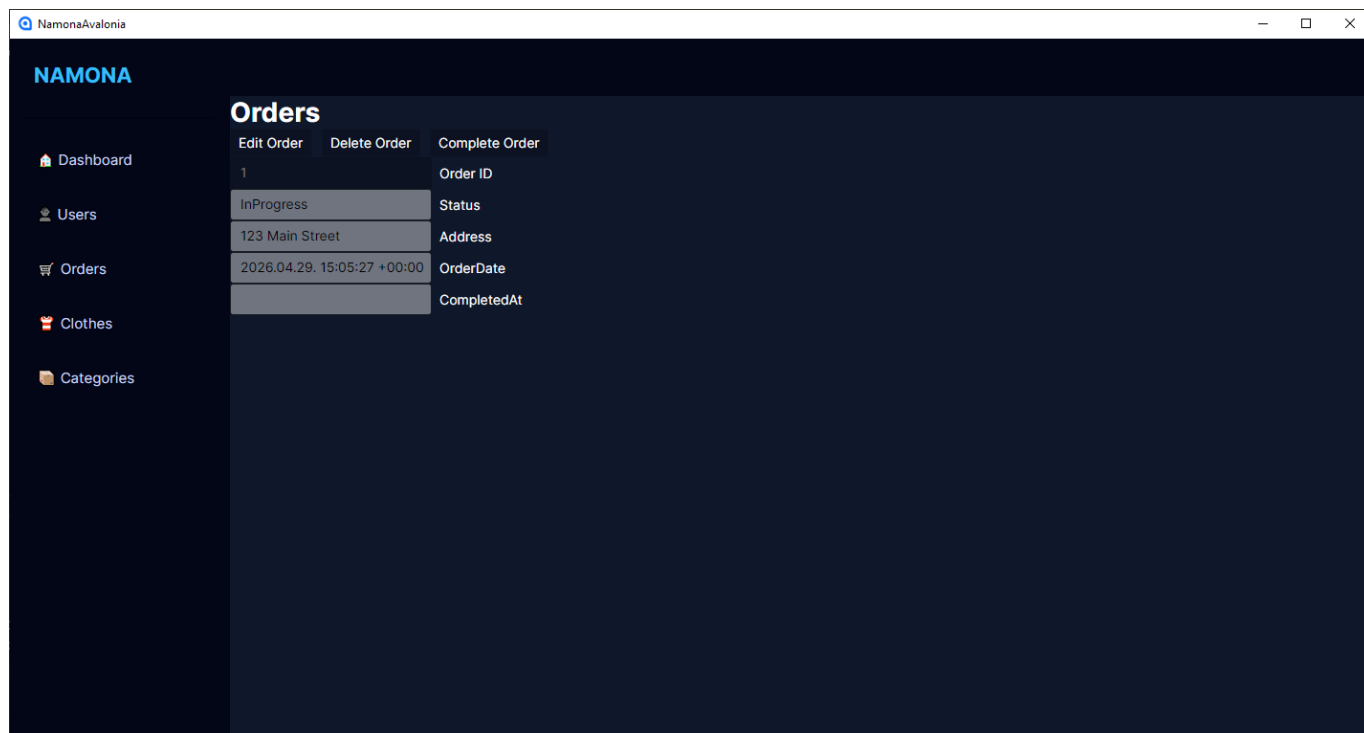
1.5.4 Rendelés Kezelés

Leírás: Rendelések listázása, törlése.



1.5.4 Rendelés Kezelés → Módosítás

Leírás: Rendelések módosítása.



1.5.5 Termék kezelés

Leírás: Termékek listázása, törlése.

The screenshot displays the 'NamonaAvalonia' web application. On the left is a dark sidebar with the 'NAMONA' logo and a menu containing 'Dashboard', 'Users', 'Orders', 'Clothes' (highlighted), and 'Categories'. The main content area is titled 'Clothes' and features three buttons: 'Add Clothes', 'Edit Clothes', and 'Delete Clothes'. Below these buttons is a table listing 10 clothing items. Each row contains an index number, a product name, a season, a year, a size, a quantity, a color, a count, a category, and a gender. The table has a light gray background with alternating row colors.

	Product Name	Season	Year	Size	Quantity	Color	Count	Category	Gender
1	Basic White T-Shirt	Summer 2025	M	100	White	20	T-Shirts	Unisex	
2	Slim Fit Jeans	Winter 2024	L	50	Blue	60	Jeans	Male	
3	Leather Jacket	Autumn 2025	M	20	Black	150	Jackets	Male	
4	Running Shoes	Spring 2025	42	75	Black	80	Shoes	Unisex	
5	Baseball Cap	Summer 2025	One Size	200	Red	15	Accessories	Unisex	
6	Women Dress	Summer 2025	S	40	Yellow	70	T-Shirts	Female	
7	High Heels	Spring 2025	38	30	Black	90	Shoes	Female	
8	Denim Jacket	Winter 2024	M	25	Blue	110	Jackets	Unisex	
9	Unisex Hoodie	Autumn 2025	L	60	Gray	55	T-Shirts	Unisex	
10	Sport Shorts	Summer 2025	M	90	Black	25	T-Shirts	Male	

1.5.5 Termék kezelés → Hozzáadás

Leírás: Termékek hozzáadása.

The screenshot shows a web application window titled 'NamonaAvalonia'. The main content area is titled 'Clothes' and features three buttons: 'Add Clothes', 'Edit Clothes', and 'Delete Clothes'. Below these buttons is a form with several input fields and labels:

Field	Label
	Clothing Name
	Collection
	Size
0	Stock
	Color
0	Price
	Gender Id
	Category Id

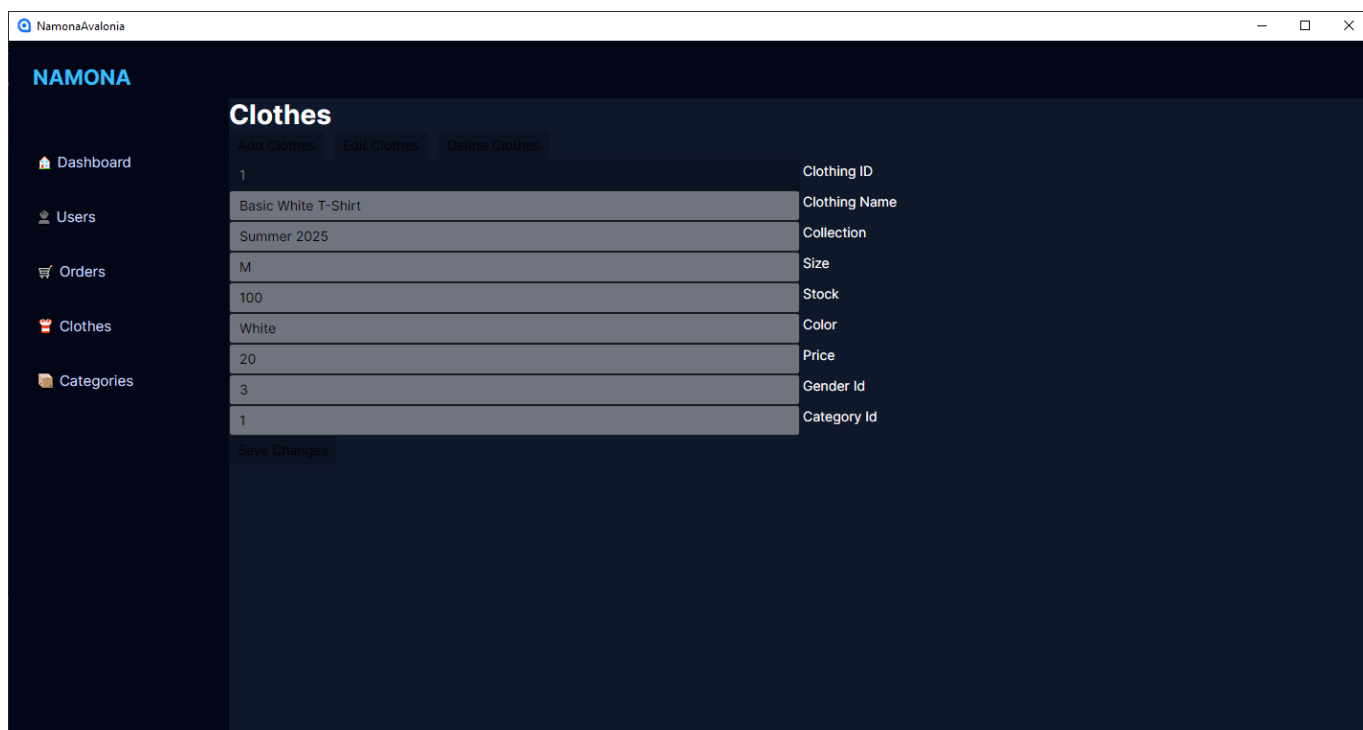
Below the form is a 'Save Changes' button.

The left sidebar contains the following menu items:

- Dashboard
- Users
- Orders
- Clothes
- Categories

1.5.5 Termék kezelés → Módosítás

Leírás: Termékek módosítása.



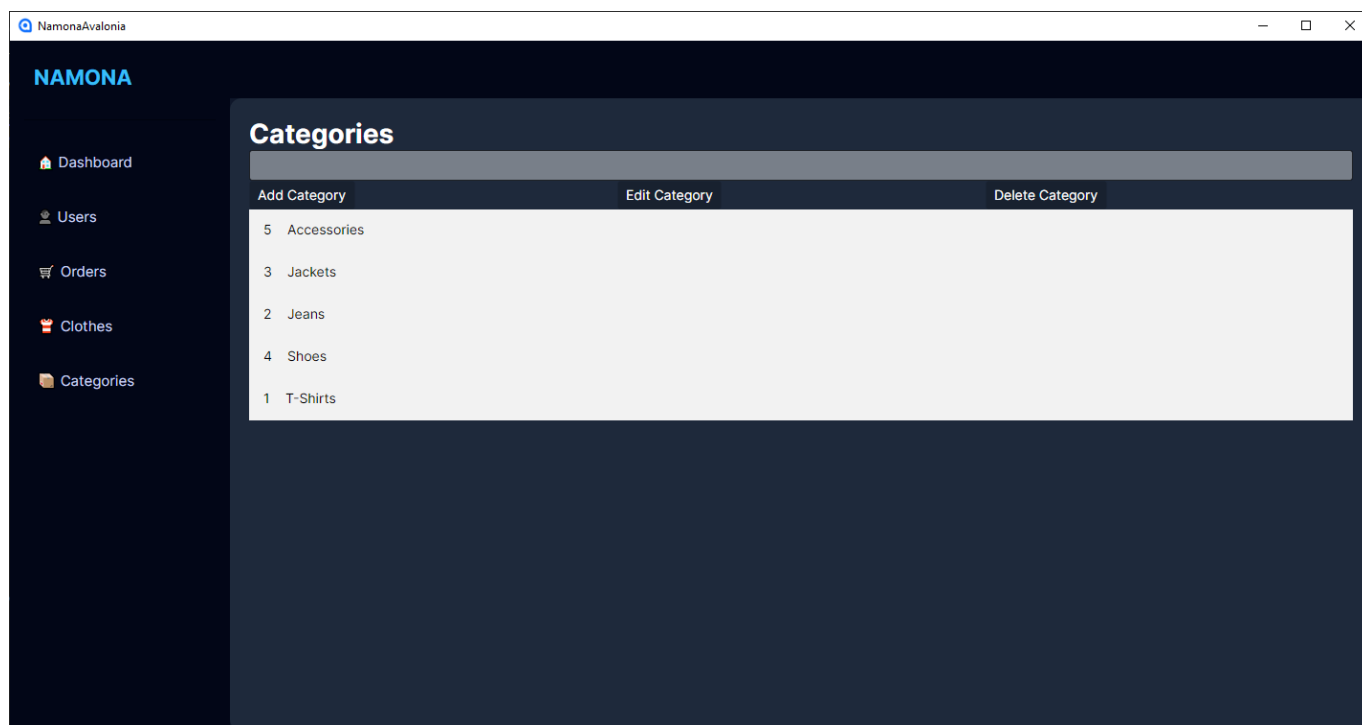
The screenshot shows the 'Clothes' edit form in the Namona WebShop admin interface. The interface has a dark blue sidebar with the 'NAMONA' logo and navigation links: Dashboard, Users, Orders, Clothes, and Categories. The main content area is titled 'Clothes' and contains three buttons: 'Add Clothes', 'Edit Clothes', and 'Delete Clothes'. Below these buttons is a form for editing a specific item. The form has a table-like structure with labels on the right and input fields on the left. The labels are: Clothing ID, Clothing Name, Collection, Size, Stock, Color, Price, Gender Id, and Category Id. The input fields contain the following values: 1, Basic White T-Shirt, Summer 2025, M, 100, White, 20, 3, and 1. At the bottom of the form is a 'Save Changes' button.

1	Clothing ID
Basic White T-Shirt	Clothing Name
Summer 2025	Collection
M	Size
100	Stock
White	Color
20	Price
3	Gender Id
1	Category Id

Save Changes

2.6.3.6 Kategória Kezelés

Leírás: Kategóriák listázása, hozzáadása, módosítása és törlése.



3. Fejlesztői dokumentáció

3.1 Használt fejlesztési eszközök

3.1.1 Szoftverttechnológiák

A projekt megvalósítása során több különböző technológia került felhasználásra, amelyek együttesen biztosítják a rendszer működését.

- **Backend:** ASP.NET Core Web API

A backend réteg REST-szerű API végpontokon keresztül biztosítja a kliensalkalmazások számára az üzleti logika és az adatelérés elérését. A controllerek feladata a kérések fogadása, validálása és a megfelelő model rétegbeli műveletek meghívása.

- **Adatkezelés:** Entity Framework Core

Az adatbázis-kezelés objektum-relációs leképezéssel történik. Az alkalmazás a perzisztens entitásokat a CinemaDbContext osztályon keresztül éri el. A rendszerben külön entitások reprezentálják a filmeket, vetítéseket, termeket, székeket, kosarakat, felhasználókat, foglalásokat és egyéb kapcsolódó objektumokat.

- **Adatbázis:** PostgreSQL

Az éles vagy fejlesztői adatkezelés alapját relációs adatbázis biztosítja. A backend a UseNpgsql(...) konfiguráción keresztül kapcsolódik a PostgreSQL adatbázishoz, ami jól illeszkedik az EF Core használatához.

- **Frontend:** HTML, CSS, Bootstrap, JavaScript, TypeScript

A webes felület statikus oldalakból, egy közös stílusállományból, valamint TypeScript/JavaScript logikából épül fel. A Bootstrap segíti a responzív és egységes megjelenést. A frontend több nézetet tartalmaz, például kezdőoldalt, profiloldalt, kosár oldalt, foglalási oldalt, kategória- és ároldalt, illetve admin felületeket.

Fejlesztői környezetben Swagger felület is elérhető, amely támogatja az API végpontok áttekintését és tesztelését.

- **Tesztelés:** xUnit, EF Core InMemory, ASP.NET Core integration testing

A projekt külön unit és integrációs tesztprojekteket is tartalmaz. A tesztek között megtalálhatók a model réteg és a controller réteg működését vizsgáló esetek.

3.1.2 Konkrét szoftverek és eszközök

- Visual Studio / .NET fejlesztői környezet
 - A solution és a projektfájlok alapján a rendszer .NET alapú fejlesztői környezetben készült.
- GitHub
 - A projekt verziókezelése Git alapokon történik, a forráskód GitHub repository-ban található.
- Swagger UI
 - A backendben fejlesztői környezetben elérhető, az API-k áttekintésére.
- Bootstrap CDN
 - A webes frontend HTML oldalai közvetlen CDN hivatkozással használják a Bootstrap stílusokat és komponenseket.
- xUnit test framework
 - Az automata tesztelés xUnit keretrendszerrel történt.

3.1.3 Fejlesztési megközelítés és architekturális szemlélet

- Controller réteg
- Business logic/model réteg
- Persistence réteg
- DTO-k
- Különálló kliensek
- Tesztprojektek

3.1.4 Moduláris felépítés összefoglalása

A rendszer főbb moduljai a következők:

- Backend
 - A központi backend projekt, amely API-t, üzleti logikát és adatkezelést biztosít.
- FrontEnd
 - Webes kliensoldali nézetek és kapcsolódó stílus- illetve scriptállományok.
- FrontEnd/AdminFrontEnd
 - Adminisztrációs célú webes felületek.
- Avalonia
 - Mobilalkalmazás, amely a backend API-t használja.
- Test
 - Unit teszt projekt a model réteg logikájának ellenőrzésére.
- IntegrationTest
 - Integrációs teszt projekt a controller réteg és az alkalmazás összműködésének ellenőrzésére.

A rendszer egy többretegű kliens–szerver architektúrát valósít meg, amely négy fő komponensből áll: a PostgreSQL adatbázisból, az ASP.NET Core alapú backendből, egy webes kliensalkalmazásból, valamint egy Avalonia alapú desktop alkalmazásból.

Az adatbázis réteg a rendszer perzisztens adattárolását biztosítja. Itt kerülnek eltárolásra a felhasználói adatok, termékek, kosár tartalma, rendelések, valamint a kapcsolódó törzsadatok.

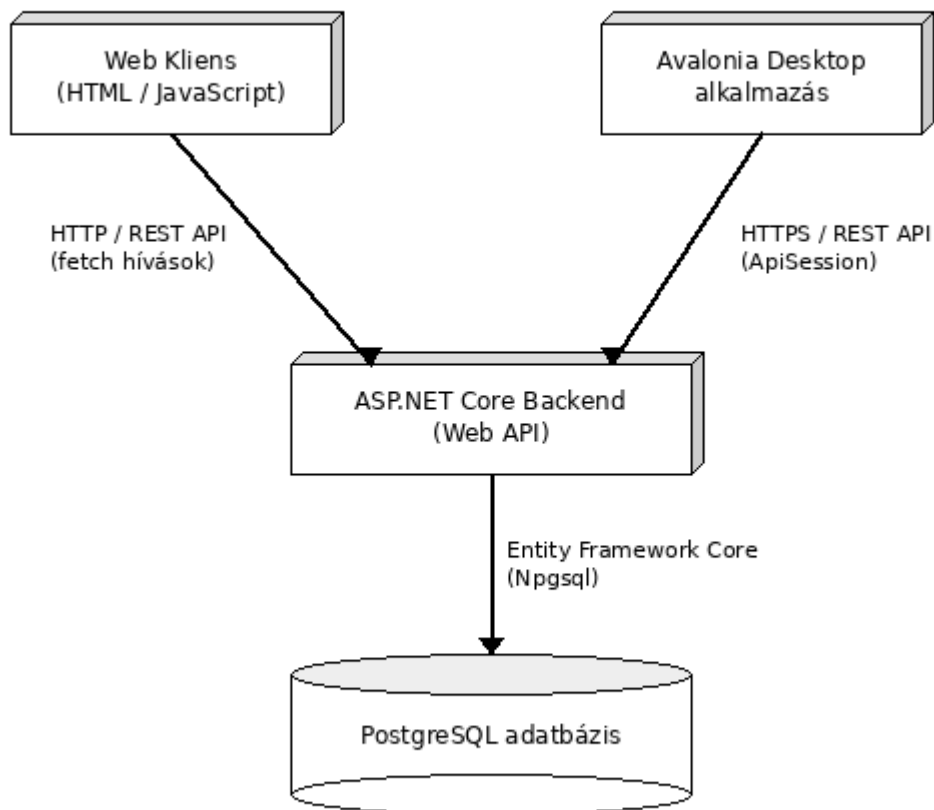
A backend réteg (ASP.NET Core Web API) biztosítja az üzleti logikát és az adatelérést. Ez a réteg felelős az adatok feldolgozásáért, validálásáért, valamint az adatbázissal való kommunikációért Entity Framework Core (Npgsql) segítségével. A backend REST API végpontokon keresztül szolgálja ki a klienseket.

A rendszer két különböző klienssel rendelkezik. A webes kliens egy böngészőben futó frontend alkalmazás, amely HTTP kérésekkel (fetch API) kommunikál a backenddel. Ez biztosítja a felhasználói felületet és a dinamikus adatmegjelenítést.

Az Avalonia alapú desktop alkalmazás egy natív asztali kliens, amely szintén a backend REST API-ját használja HTTPS kapcsolaton keresztül. Ez az alkalmazás elsősorban a rendszer adminisztrációs és felhasználói funkcióinak elérésére szolgál.

A rendszer felépítése lehetővé teszi, hogy több különböző kliens (web és desktop) ugyanazt a backend szolgáltatást használja, ezáltal biztosítva az egységes üzleti logikát és adatkezelést.

Rendszer architektúra - Namona projekt



3.2 Adatbázis

Az alkalmazás PostgreSQL adatbázist használ, amelyhez az Entity Framework Core biztosít hozzáférést. A központi adatkezelő osztály egy DbContext, amely tartalmazza a rendszerhez tartozó DbSet-eket, például rendelések, kosarak, felhasználók, termékek, kategóriák, képek és egyéb kapcsolódó entitások kezelésére.

Az adatmodell a webshop működéséhez igazodik: a Product entitás a ruházati termékeket írja le, a Category a különböző termékkategóriákat kezeli, a Cart a felhasználók aktuális kosarát reprezentálja, míg az Order a leadott rendeléseket tartalmazza. Emellett a rendszer kezeli a termékekhez tartozó képeket, valamint a felhasználói adatokat is, biztosítva a teljes vásárlási folyamat lefedését.

Entitás	Szerep	Főbb mezők	Kapcsolatok
Users	Felhasználók (vásárlók és adminok) adatai.	UserId, UserName, Email, Password, PhoneNumber, Role	Cart
Clothes	Ruházati termékek adatai.	ClothingId, ClothingName, Collection, Size, Color, Price, Stock, GenderId, CategoryId	Category, Gender, Cart
Category	Termékkategóriák (pl. póló, nadrág).	CategoryId, CategoryName	Clothes
Gender	Célcsoport (pl. férfi, női, unisex).	GenderId, GenderType	Clothes
Cart	Felhasználó kosara, kiválasztott termékekkel.	CartId, ClothingId, UserId, Amount, PriceSum, OrderId	Users, Clothes, Orders
Orders	Leadott rendelések és állapotuk.	OrderId, OrderDate, Address, Status, CompletedAt	Cart

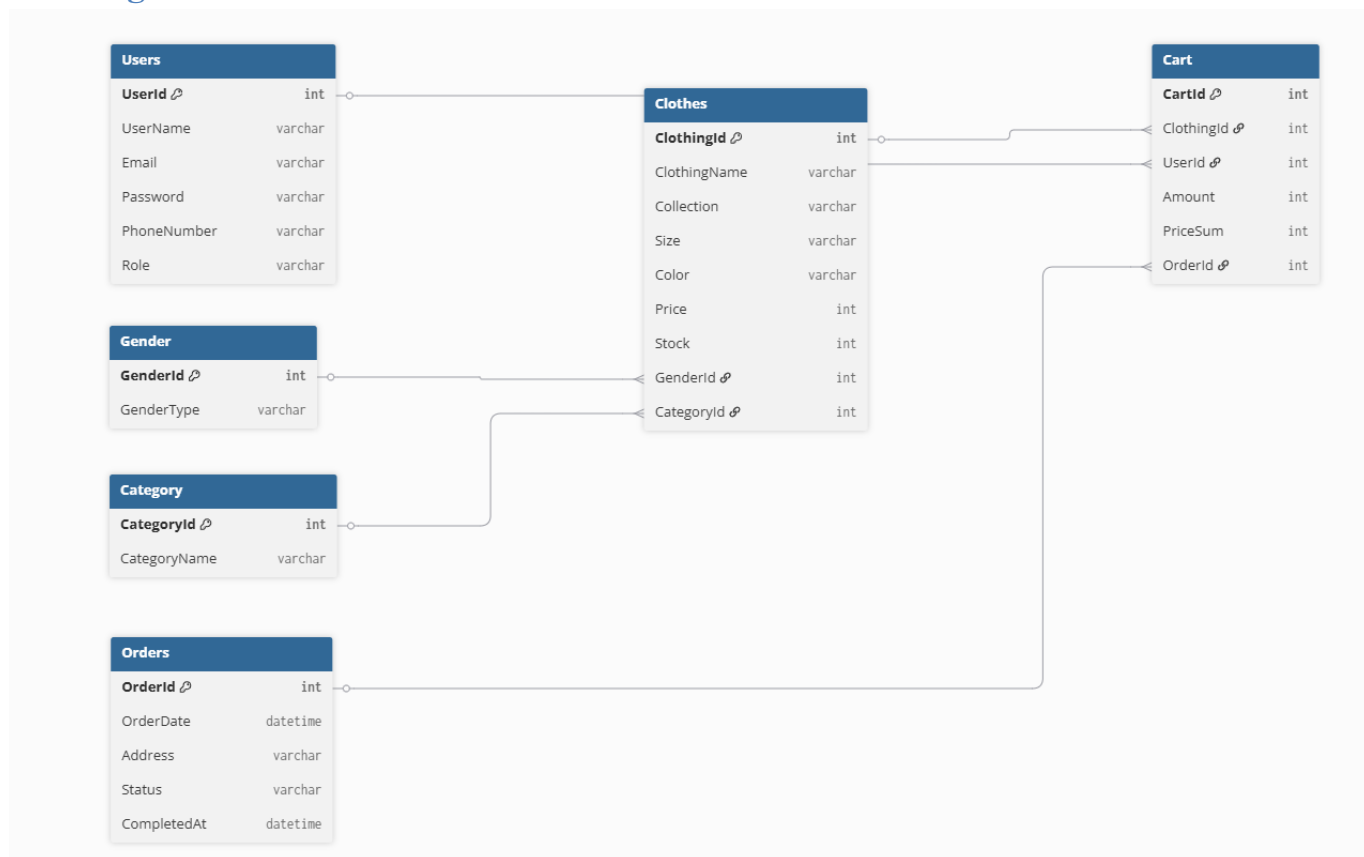
3.2.1 Az adatbázis elérésének módja

Az adatbázishoz való kapcsolódás a backend indulásakor történik meg. A rendszer ehhez a `NamonaDbContext` osztályt használja, amely a PostgreSQL adatbázishoz kapcsolódik az `Npgsql` provider segítségével.

A kapcsolat regisztrálása az alkalmazás indulásakor történik, ahol a rendszer a `DbContext`-et poolozott formában hozza létre. Ez gyorsabb és hatékonyabb működést biztosít.

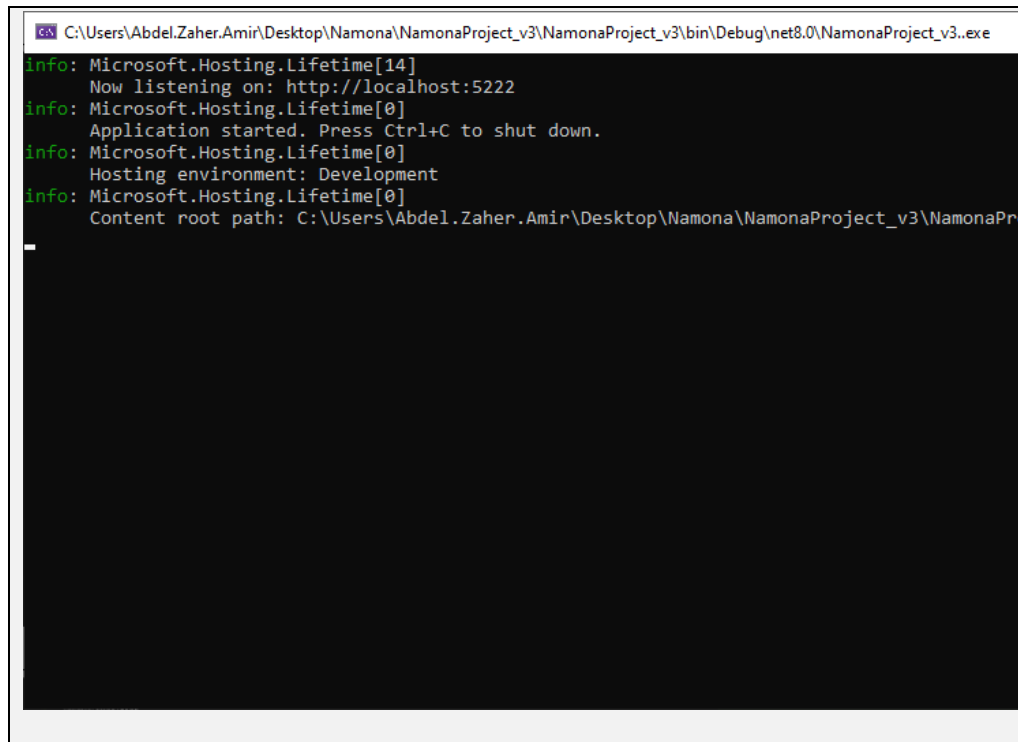
A projektben a `DbContext` példányait a model réteg osztályai kapják meg `dependency injection` segítségével. Ennek köszönhetően az adatbázis-kezelési műveletek nem a controller rétegben, hanem a business logic rétegben találhatók.

ER Diagram



3.3 REST API / Backend

A backend ASP.NET Core Web API projekt. A Program.cs konfigurálja az adatbázis-kapcsolatot, a CORS szabályokat, a Cookie Authentication hitelesítést, az authorization szolgáltatást, a Swagger fejlesztői felületet és a model osztályok dependency injection regisztrációját.



```
C:\Users\Abdel.Zaher.Amir\Desktop\Namona\NamonaProject_v3\NamonaProject_v3\bin\Debug\net8.0\NamonaProject_v3.exe
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5222
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\Abdel.Zaher.Amir\Desktop\Namona\NamonaProject_v3\NamonaPr
```



Swagger

OpenAPI Specification

Select a definition

NamonaProject_v3.v1

NamonaProject_v3.

1.0

OK

1.0

<http://localhost:8080/swagger-ui/index.html>

Cart

GET

/api/cart/cartcontent

PUT

/api/cart/editcart

POST

/api/cart/addcart

DELETE

/api/cart/deletecartitem

Category

GET

/api/category/detailcategories

POST

/api/category/addcategory

PUT

/api/category/editcategory

DELETE

/api/category/deletecategory

Clothes

GET

/api/clothes/detailclothes

POST

/api/clothes/addclothes

PUT

/api/clothes/modify

DELETE

/api/clothes/remove

GET

/api/clothes/filterclothes

GET

/api/clothes/searchclothes

Gender

GET

/api/gender/allgender

POST

/api/gender/addgender

PUT

/api/gender/modifygender

DELETE

/api/gender/deletegender

Orders

GET

/api/orders/allorders

GET

/api/orders/getorderbyid

GET

/api/orders/orders

POST

/api/orders/checkout

POST

/api/orders/addorder

PUT

/api/orders/updateorder

DELETE

/api/orders/deleteorder

DELETE

/api/orders/cancel

PUT

/api/orders/complete

PUT

/api/orders/status

GET

/api/orders/getrevenue

User

POST

/api/user/login

POST

/api/user/admin/login

POST

/api/user/register

GET

/api/user/showusers

DELETE

/api/user/deleteuser

PUT

/api/user/password

PUT

/api/user/edituser

PUT

/api/user/promote

GET

/api/user/me

POST

/api/user/logout

Controller	Model	Fő feladat
UserController	UserModel	Felhasználók kezelése: regisztráció, bejelentkezés (user és admin), kijelentkezés, aktuális felhasználó lekérdezése, jelszómódosítás, valamint adminisztrációs műveletek (felhasználók listázása, törlése, szerkesztése, jogosultságkezelés).
ClothesController	ClothesModel	Ruházati termékek kezelése: termékek listázása, keresése és szűrése, valamint adminisztrátori jogosultsággal új termék hozzáadása, módosítása és törlése.
CategoryController	CategoryModel	Termékkategóriák kezelése: kategóriák lekérdezése, illetve adminisztrációs műveletek (létrehozás, módosítás, törlés).
GenderController	GenderModel	Termékekhez tartozó célcsoportok (pl. férfi, női) kezelése: lekérdezés, valamint adminisztrátori CRUD műveletek.
OrdersController	OrderModel	Rendelések kezelése: felhasználói rendelések lekérdezése és leadása (checkout), rendelés törlése, valamint admin oldalon rendelések kezelése (listázás, módosítás, státusz frissítése, teljesítés, bevétel lekérdezés).
CartController	CartModel	A kosár kezelése: A felhasználó kosarának lekérése amire csak maga a felhasználó jogosult. Emellett a felhasználó képes új kosarat létrehozni, a kosarat módosítani, valamint törölni.

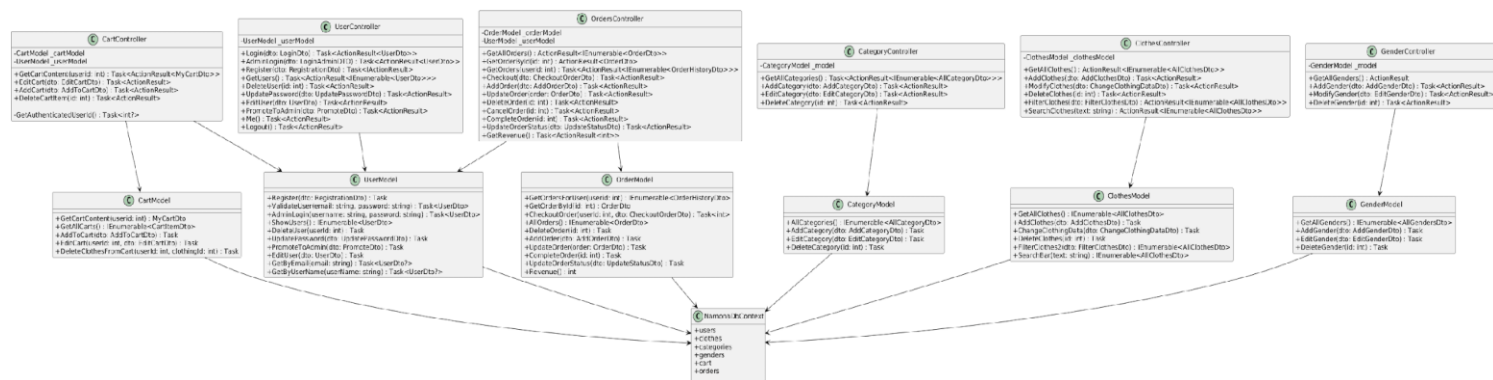
3.3.1 A backend réteges felépítése

A backend szerkezetében elkülönülnek az alábbi rétegek:

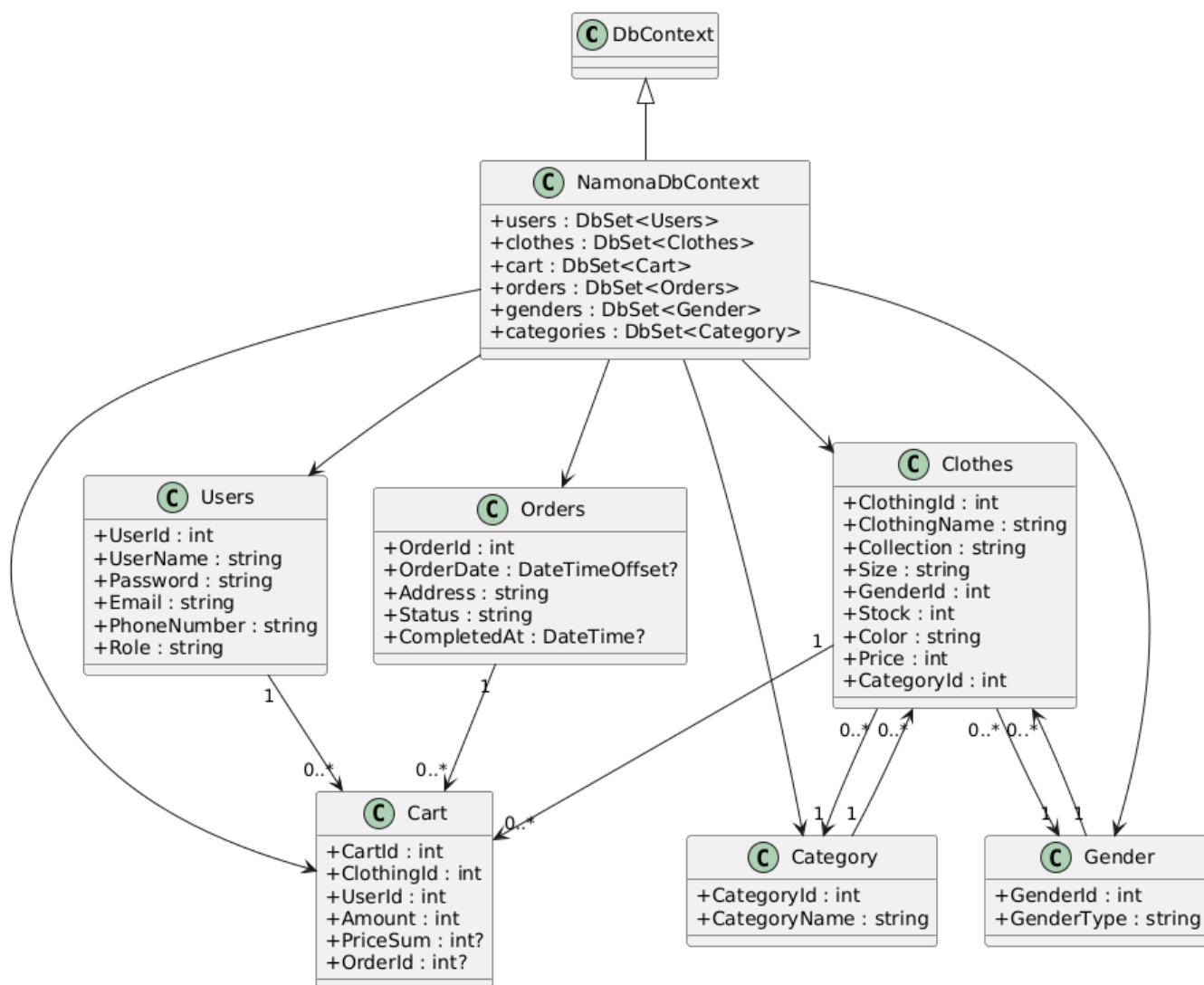
- **Controllers**
 - Ezek a HTTP végpontokat megvalósító osztályok. Feladatuk a kérés fogadása, a megfelelő model metódus meghívása és a válasz visszaadása.
- **Model**
 - A business logic réteg, amely az üzleti szabályokat, lekérdezéseket és módosításokat végzi. A controller nem közvetlenül az adatbázissal dolgozik, hanem ezeket az osztályokat használja.
- **Persistence**
 - A perzisztens entitásokat és a DbContext osztályt tartalmazó réteg.
- **DTO**
 - A Data Transfer Object osztályokat tartalmazó réteg, amelyek az API bemeneti és kimeneti adatszerkezeteket határozzák meg.

Ez a felépítés szakmailag kedvező, mert javítja a karbantarthatóságot, az átláthatóságot és a tesztelhetőséget.

Backend osztálydiagram



Adatbázis osztálydiagrammja :



3.3.2 Főbb backend model osztályok

A Program.cs alapján a következő főbb model osztályok vannak regisztrálva dependency injection segítségével:

- UserModel
- CategoryModel
- ClothesModel
- CartModel
- GenderModel
- OrderModel

Ez azt jelenti, hogy a rendszer funkcionális logikája legalább öt nagy domainrészre bontva jelenik meg:

- felhasználókezelés és autentikáció
- ruházati termékek kezelése
- kategóriák és nemek (gender) kezelése
- kosárkezelés
- rendelések és checkout folyamat

A rendszer központi szereplője az **OrderModel**, mivel ez kezeli a teljes vásárlási folyamatot a kosártól kezdve a rendelés létrehozásán át egészen a státuszkezelésig és bevételszámításig.

Emellett a **ClothesModel** tartalmazza a termékekkel kapcsolatos legfontosabb műveleteket (CRUD, szűrés, keresés), míg a **CartModel** a felhasználói kosár állapotát kezeli. A **UserModel** felel a regisztrációért, bejelentkezésért és jogosultságkezelésért.

3.3.3 UserController – Felhasználókezelés

Base route:

- /api/user
-

Login

Végpont:

- POST /api/user/login

Bemenet (LoginDto):

- Email (string)
- Password (string)

Kimenet: **UserDto**

- UserId (int) – a felhasználó egyedi azonosítója
- UserName (string) – felhasználónév
- Email (string) – e-mail cím
- Phone (string) – telefonszám (ha van)
- Role (string) – jogosultsági szerep (User/Admin)

Hibák:

- 401 Unauthorized – hibás email vagy jelszó
 - 400 BadRequest – egyéb hiba
-

Admin login

Végpont:

- POST /api/user/admin/login

Bemenet (LoginAdminDTO):

- UserName (string)
- Password (string)

Kimenet: **UserDto**

- UserId
- UserName
- Email
- Phone
- Role

Hibák:

- 401 Unauthorized
 - 404 NotFound
 - 400 BadRequest
-

Regisztráció

Végpont:

- POST /api/user/register

Bemenet (RegistrationDto):

- Email (string)
- Password (string)
- UserName (string)

Kimenet:

- 201 Created – nincs DTO visszatérési adat
 - 409 Conflict – már létező user
 - 406 NotAcceptable – hibás adatformátum
 - 400 BadRequest
-

Összes felhasználó (Admin)

Végpont:

- GET /api/user/ShowUsers

Kimenet: **List<UserDto>**

Minden elem:

- UserId
 - UserName
 - Email
 - Phone
 - Role
-

Felhasználó törlése

Végpont:

- DELETE /api/user/DeleteUser?id=int

Bemenet:

- id (int) – UserId

Kimenet:

- 200 OK – sikeres törlés
 - 404 NotFound – nem létező user
 - 400 BadRequest
-

Jelszó módosítás

Végpont:

- PUT /api/user/password

Bemenet (UpdatePasswordDto):

- UserId (int)
- Password (string)

Kimenet:

- 200 OK
 - 406 NotAcceptable
 - 400 BadRequest
-

User szerkesztés

Végpont:

- PUT /api/user/EditUser

Bemenet (UserDto):

- UserId

- UserName
- Email
- Phone
- Role

Kimenet:

- 200 OK
 - 406 NotAcceptable
 - 400 BadRequest
-

Promote user

Végpont:

- PUT /api/user/promote

Bemenet (PromoteDto):

- UserId (int)
- Role (string)

Kimenet:

- 200 OK
 - 404 NotFound
 - 406 NotAcceptable
 - 400 BadRequest
-

Saját profil

Végpont:

- GET /api/user/me

Bemenet:

- nincs

Kimenet: **UserDto**

- UserId
- UserName
- Email
- Phone
- Role

Hibák:

- 401 Unauthorized – ha nincs bejelentkezve
-

Logout

Végpont:

- POST /api/user/logout

Kimenet:

- 200 OK – nincs DTO
-

3.3.4 ClothesController – Ruhák kezelése

Base route:

- /api/clothes
-

Összes ruha

Végpont:

- GET /api/clothes/GetAllClothes

Kimenet: **List<AllClothesDto>**

- ClothingId (int)
 - ClothingName (string)
 - Collection (string)
 - Size (string)
 - Stock (int)
 - Color (string)
 - Price (int)
 - GenderId (int)
 - GenderType (string)
 - CategoryId (int)
 - CategoryName (string)
-

Ruha hozzáadás

Végpont:

- POST /api/clothes/AddClothes

Bemenet (AddClothesDto):

- ClothingName
- Collection
- CategoryName
- Size
- GenderName
- Stock
- Color
- Price

Kimenet:

- 201 Created
- 409 Conflict
- 406 NotAcceptable
- 404 NotFound
- 400 BadRequest

Ruha módosítás

Végpont:

- PUT /api/clothes/modify

Bemenet (ChangeClothingDataDto):

- ClothingId
- ClothingName
- Collection
- Size
- Stock
- Color
- Price
- CategoryId
- GenderId

Kimenet:

- 200 OK
- 404 NotFound
- 409 Conflict
- 406 NotAcceptable
- 400 BadRequest

Ruha törlés

Végpont:

- DELETE /api/clothes/remove?id=int

Bemenet:

- id (int)

Kimenet:

- 200 OK
 - 404 NotFound
 - 400 BadRequest
-

Szűrés

Végpont:

- GET /api/clothes/FilterClothes

Bemenet (FilterClothesDto):

- Category (string?)
- Collection (string?)
- Gender (string?)
- Minprice (int?)
- Maxprice (int?)

Kimenet: **List<AllClothesDto>**

- ClothingId
 - ClothingName
 - Collection
 - Size
 - Stock
 - Color
 - Price
 - GenderId
 - GenderType
 - CategoryId
 - CategoryName
-

Keresés

Végpont:

- GET /api/clothes/SearchClothes?text=string

Bemenet:

- text (string)

Kimenet: **List<AllClothesDto>**

- ClothingId
 - ClothingName
 - Collection
 - Size
 - Stock
 - Color
 - Price
 - GenderId
 - GenderType
 - CategoryId
 - CategoryName
-

3.3.5 CategoryController – Kategóriák

Base route:

- /api/category
-

Összes kategória

Végpont:

- GET /api/category/GetAllCategories

Kimenet: **List<AllCategoryDto>**

- Id (int)
 - CategoryName (string)
-

Kategória hozzáadás

Végpont:

- POST /api/category/AddCategory

Bemenet:

- CategoryName (string)

Kimenet:

- 201 Created
 - 409 Conflict
 - 406 NotAcceptable
 - 400 BadRequest
-

Kategória módosítás

Végpont:

- PUT /api/category/EditCategory

Bemenet (EditCategoryDto):

- Id
- CategoryName

Kimenet:

- 200 OK
 - 409 Conflict
 - 406 NotAcceptable
 - 400 BadRequest
-

Kategória törlés

Végpont:

- DELETE /api/category/DeleteCategory?id=int

Kimenet:

- 200 OK
 - 404 NotFound
 - 400 BadRequest
-

3.3.6 GenderController – Nemek

Base route:

- /api/gender
-

Összes gender

Végpont:

- GET /api/gender/AllGenders

Kimenet: **List<AllGendersDto>**

- Id (int)
 - Type (string)
-

Gender hozzáadás

Végpont:

- POST /api/gender/AddGender

Bemenet:

- Id (int)
- Type (string)

Kimenet:

- 201 Created
 - 406 NotAcceptable
 - 400 BadRequest
-

Gender módosítás

Végpont:

- PUT /api/gender/ModifyGender

Bemenet:

- Id
- Type

Kimenet:

- 200 OK
- 404 NotFound
- 406 NotAcceptable
- 400 BadRequest

Gender törlés

Végpont:

- DELETE /api/gender/DeleteGender?id=int

Kimenet:

- 200 OK
 - 404 NotFound
 - 400 BadRequest
-

3.3.7 CartController – Kosár

Base route:

- /api/cart
-

Kosár tartalma

Végpont:

- GET /api/cart/CartContent

Kimenet: **MyCartDto**

- UserId (int)
- Carts (List<CartItemDto>)

CartItemDto:

- ClothingId
 - ClothingName
 - Collection
 - CategoryId
 - CategoryName
 - GenderId
 - GenderName
 - Size
 - Stock
 - Amount
 - Color
 - Price
 - PriceSum
-

Kosár módosítás

Végpont:

- PUT /api/cart/EditCart

Bemenet:

- ClothingId
- Amount

Kimenet:

- 200 OK
 - 404 NotFound
 - 406 NotAcceptable
 - 400 BadRequest
-

Kosárhoz adás

Végpont:

- POST /api/cart/addCart

Bemenet:

- ClothingId
- UserId
- Amount

Kimenet:

- 201 Created
 - 404 NotFound
 - 406 NotAcceptable
 - 400 BadRequest
-

Kosár elem törlés

Végpont:

- DELETE /api/cart/DeleteCartItem?id=int

Kimenet:

- 200 OK
 - 404 NotFound
 - 400 BadRequest
-

3.3.8 OrdersController – Rendelések

Base route:

- /api/orders
-

Összes rendelés

Végpont:

- GET /api/orders/AllOrders

Kimenet: **List<OrderDto>**

- OrderId
 - UserName
 - OrderDate
 - Address
 - Status
 - CompletedAt
-

Saját rendelések

Végpont:

- GET /api/orders/Orders

Kimenet: **List<OrderHistoryDto>**

- OrderId
 - OrderDate
 - Status
 - Items (List<CartItemDto>)
-

Checkout

Végpont:

- POST /api/orders/Checkout

Bemenet:

- Address (string)

Kimenet:

- orderId (int)
-

Rendelés módosítás

Végpont:

- PUT /api/orders/UpdateOrder

Bemenet:

- OrderDto

Kimenet:

- 200 OK
 - 406 NotAcceptable
-

Bevétel

Végpont:

- GET /api/orders/GetRevenue

Kimenet:

- int (teljes bevétel)
-

3.3.9 Rendelésekhez kapcsolódó funkciók és végpontok

Rendelések lekérdezése (User)

Végpont:

- GET /api/Orders/Orders?userid=...

Leírás:

A végpont az aktuálisan bejelentkezett felhasználó rendeléseit adja vissza.

Bemenet:

- userid (int, query param) → a felhasználó azonosítója (de a backend ezt felülírja az autentikáció alapján)

Kimenet:

- List<CartItemDto>

CartItemDto tartalma:

- CartId (int)
 - UserId (int)
 - ClothingId (int)
 - ClothingName (string)
 - Collection (string)
 - CategoryId (int)
 - CategoryName (string)
 - GenderId (int)
 - GenderName (string)
 - Size (string)
 - Stock (int)
 - Amount (int)
 - Color (string)
 - Price (int)
 - PriceSum (int?)
-

Checkout

Végpont:

- POST /api/Orders/Checkout

Leírás:

A kosár tartalmából rendelést hoz létre.

Bemenet (DTO):

CheckoutOrderDto:

- Address (string)

Kimenet:

- 200 OK → { **orderId: int** }
- orderId (int): az újonnan létrejött rendelés azonosítója

Összes rendelés (Admin)

Végpont:

- GET /api/Orders/AllOrders

Leírás:

Az összes rendelés lekérdezése admin számára.

Kimenet:

- List<OrderDto>

OrderDto tartalma:

- OrderId (int)
 - UserName (string)
 - OrderDate (DateTimeOffset?)
 - Address (string)
 - Status (string)
 - CompletedAt (DateTimeOffset?)
-

Rendelés lekérdezése azonosító alapján (Admin)

Végpont:

- GET /api/Orders/GetOrderById?id=...

Leírás:

Egy konkrét rendelés lekérdezése ID alapján.

Bemenet:

- id (int, query)

Kimenet:

- OrderDto

OrderDto tartalma:

- OrderId (int)
 - UserName (string)
 - OrderDate (DateTimeOffset?)
 - Address (string)
 - Status (string)
 - CompletedAt (DateTimeOffset?)
-

Rendelés hozzáadása (Admin)

Végpont:

- POST /api/Orders/AddOrder

Leírás:

Új rendelés manuális létrehozása admin által.

Bemenet (DTO):

AddOrderDto:

- OrderDate (DateTimeOffset)
- Address (string)
- Status (string)
- CompletedAt (DateTimeOffset?)

Kimenet:

- 201 Created (string üzenet: „Order successfully added”)
-

Rendelés módosítása (Admin)

Végpont:

- PUT /api/Orders/UpdateOrder

Leírás:

Egy meglévő rendelés módosítása.

Bemenet (DTO):

OrderDto:

- OrderId (int)
- UserName (string)
- OrderDate (DateTimeOffset?)
- Address (string)
- Status (string)
- CompletedAt (DateTimeOffset?)

Kimenet:

- 200 OK (string: „Order successfully updated”)
-

Rendelés törlése

Végpont:

- DELETE /api/Orders/DeleteOrder?id=...

Leírás:

Rendelés végleges törlése.

Bemenet:

- id (int, query)

Kimenet:

- 200 OK (string: „Order successfully deleted”)
 - 404 NotFound (ha nincs ilyen rendelés)
-

Rendelés lemondása (User)

Végpont:

- DELETE /api/Orders/cancel?id=...

Leírás:

A felhasználó saját rendelését törli/lemondja.

Bemenet:

- id (int, query)

Kimenet:

- 200 OK (string: „Order cancelled”)
 - 404 NotFound
-

Rendelés teljesítése (Admin)

Végpont:

- PUT /api/Orders/Complete?id=...

Leírás:

A rendelés státuszát „Completed”-re állítja.

Bemenet:

- id (int, query)

Kimenet:

- 200 OK (string: „Order completed”)
 - 404 NotFound
-

Státusz módosítása (Admin)

Végpont:

- PUT /api/Orders/status

Leírás:

Rendelés státuszának módosítása tetszőleges értékre.

Bemenet (DTO):

UpdateStatusDto:

- OrderId (int)
- Status (string)

Kimenet:

- 200 OK (string: „Order status updated”)
- 406 NotAcceptable (ha hibás adat)
- 400 BadRequest

Bevétel lekérdezése (Admin)

Végpont:

- GET /api/Orders/GetRevenue

Leírás:

A rendszer teljes bevételének lekérdezése.

Bemenet:

- nincs

Kimenet:

- int (összes bevétel értéke)
- 200 OK → számérték
- 400 BadRequest (hiba esetén)

3.4 Webes frontend fejlesztői leírása - Namona E-commerce Platform

A webes frontend HTML, CSS, JavaScript technológiákból épül fel. A kliensoldali logika fetch hívásokkal kommunikál a backend API-val (ASP.NET Core).

3.4.1. Oldalszerkezet és főbb nézetek

A Frontend több különálló HTML oldalt tartalmaz. Ezek együtt alkotják a felhasználói nézetrendszert.

Főbb oldalak:

Namona.html - Főoldal

- A platform fő oldala. Innen lehet navigálni a termékkategóriákra (Hoodies, T-shirts, Pants), keresni és szűrni a termékeket.

Hoodies.html, Tshirts.html, Pants.html - Kategória oldalak

- Az egyes termékkategóriák bemutatása flip-card animációval a termékek elől- és hátulnézetével.

HoodieProduct1-6.html, TshirtProduct1-6.html, PantsProduct1-6.html - Termékdetail oldalak

- Az egyes termékek részletes adatainak megjelenítésére szolgáló oldalak (ár, szín, méret, készlet).

NamonaCart.html - Kosár oldal

- A kosár tartalmának megjelenítésére, szűrésére és a rendelés véglegesítésére szolgáló oldal. Tartalmaz search és filter funkciókat.

SavedProducts.html - Mentett termékek

- A felhasználó által mentett/lájkolt termékek megjelenítésére szolgáló oldal.

MyOrders.html - Moje rendelések

- A felhasználó aktív és lezárt rendeléseinek megjelenítésére szolgáló oldal.

NamonaLogin.html - Bejelentkezés és regisztráció

- A felhasználói bejelentkezési és regisztrációs felület. Támogatja a normál felhasználó és admin bejelentkezést.

AdminDashboard.html - Admin felület

- Az adminisztrátor számára a termékek, kategorák és rendelések kezelésére szolgáló oldal.

AboutUs.html - Rólunk

- Információs oldal a platform működéséről.

3.4.2 Közös vizuális megjelenés

1. Közös Namona.css stíluslap
2. Közös AdminDashboard.css az admin felülethez
3. Flip-card animációk a termékek megjelenítésére
4. Közös navigációs és layout elemek (header, sidebar, profil menü)

A közös képi elemek közé tartoznak:

- Namona logo
- Háttérképek
- Termékkép placeholder-ek
- Kosár és profil ikonok

3.4.3 A JavaScript klienslogika szerepe

Script.js egy nagyon jelentős, központosított fájl, amely szinte minden HTML-hez társul.

A Script.js a frontend szinte minden fontos funkcióját tartalmazza:

- API hívások és URL beállítások
- Termékek és kategóriák betöltése
- Kosárkezelés
- Rendeléskezelés
- Kliensoldali profilkezelés
- Bejelentkezési és regisztrációs logika
- Oldal-inicializációs műveletek
- Globális keresési és szűrési logika
- Mentett termékek kezelése (localStorage)

3.4.4 DTO interfészek és adatstruktúrák a frontendben

A backend az alábbi típusú adatokat küldi:

- Clothes - Termékek
- clothingId
- clothingName
- categoryName
- price
- color
- size
- stock
- imagePath
- description
- CartItem - Kosár tételek
- clothingId
- clothingName
- categoryName
- size
- color
- amount
- price
- priceSum
- Order - Rendelések
- orderId
- userId
- orderDate
- status
- items[]
- CurrentUser - Bejelentkezett felhasználó
- userId
- email
- userName
- role

3.4.5 Termékek megjelenítése és szűrése

Termékek betöltése

Az `getAllClothes()` függvény a `/api/Clothes/GetAllClothes` végpontot hívja meg. A kapott adatokból a `loadCategoryProducts()` függvény termékkartyákat hoz létre flip-card animációval.

A megjelenített információk:

- Termék neve
- Kategória (Hoodie, T-shirt, Pants)
- Elöl- és hátsúnézet képek (flip-card)
- Ár
- Termékdetail link
- A termékkártyák dinamikusan épülnek fel DOM létrehozással.
- Képek megjelenítése

A frontend kétféle forrásból képes képeket kezelni:

- Backendből érkező kép útvonalakból (`imagePath`)
- Fallback-ként statikus helyi képfájlokból
- Szűrők és keresés

A kosár oldalon többféle szűr implementálva van:

- Kategória szerinti szűrés (Hoodie, T-shirt, Pants)
- Méret szerinti szűrés (S, M, L, XL, stb.)
- Szín szerinti szűrés
- Szabad szöveges keresés (termék neve, kategória alapján)
- Header search - gyors keresés
- Side menu filter - részletes szűrőpanel
- A szűrési logika a `applyFiltersAndRender()` függvényben valósul meg, amely kombinálva mutatja az összes aktív szűrőt.

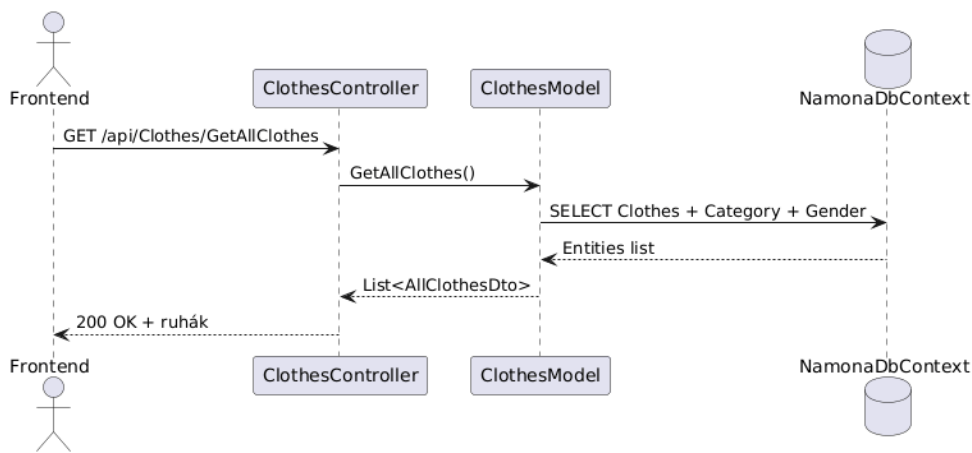
3.4.6 Kategória oldalak

A Hoodies.html, Tshirts.html, Pants.html oldalak loadCategoryProducts() függvénye:

- Lekéri az összes terméket a /api/Clothes/GetAllClothes végpontról
- Szűri azokat a kategória alapján (normalizáció: "hoodie", "tshirt", "pants")
- Visz flip-card formájában megjeleníti őket
- Dinamikusan generálja a product-detail link-eket cloth clothing ID-vel

3.4.7 Termék detail oldalak

A HoodieProduct1.html, PantsProduct1.html, stb. oldalak célja a termék részletes adatainak megjelenítése.

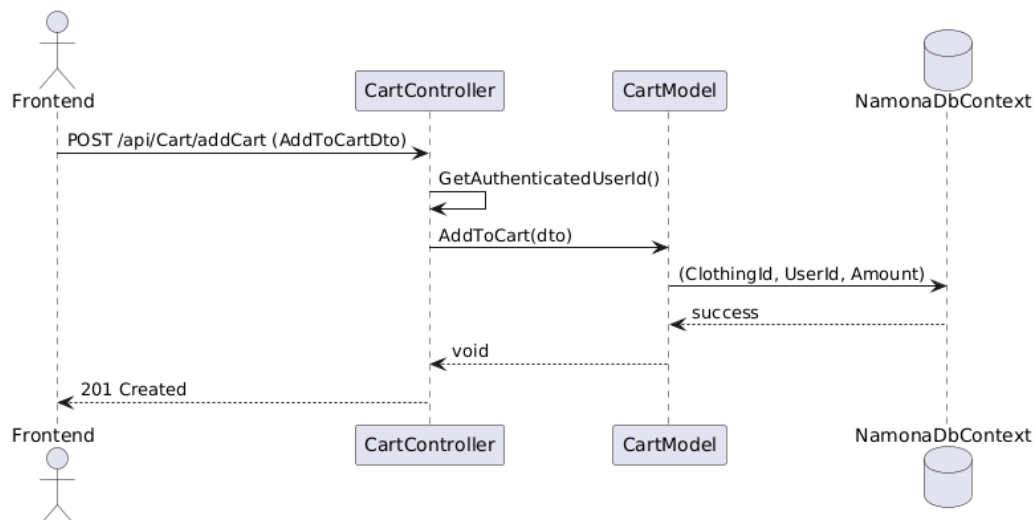


A loadProduct() függvény:

- Lekéri a termék ID-t az URL-ből (?id=clothingId)
- Megkeresi az adott terméket a /api/Clothes/AllClothes végpontból
- Megjeleníti az ár, szín, méret, készlet adatokat
- Támogatja a "Kosárba helyezés" és "Mentés" (♥) funkciókat

3.4.8 Kosárkezelés

Kosár adattárolása



A kosár adatait az API tárolja (/api/Cart/), és a frontend a `cartViewState` globális objektumban kezeli az aktuális szűrő és keresési állapotot.

A `CartItem` szerkezet a következő típusú adatokat tartalmazza:

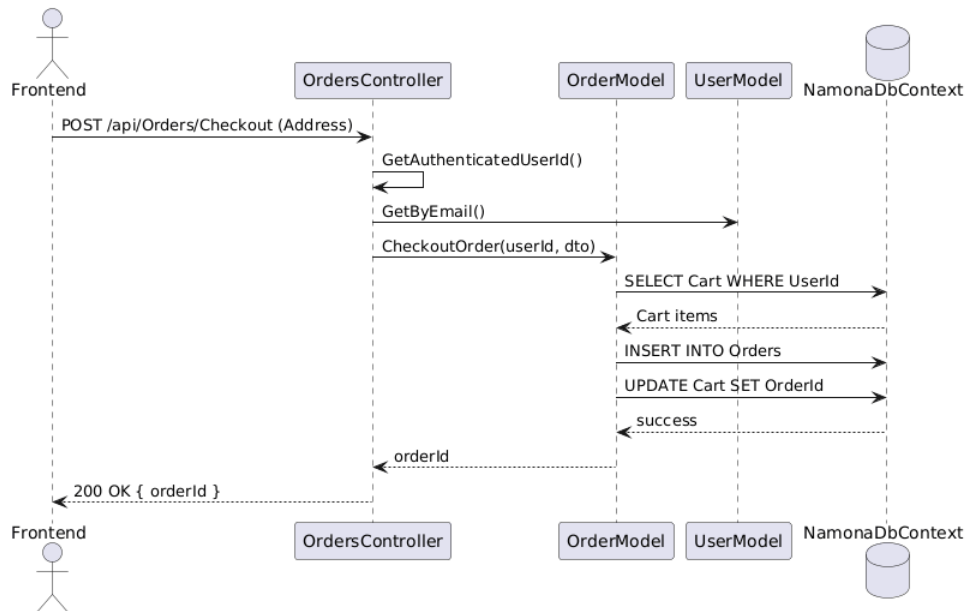
- Termék azonosító és neve
- Kategória, méret, szín
- Mennyiség
- Egységár és összeg
- Kosár funkcionalitása

A `loadCart()` és `renderCartItem()` függvények:

- A kosár tartalmának megjelenítése
- Az összesítés kiszámítása
- Mennyiség módosítása (+ / - gombokkal)
- Tétel törlése
- Szűrés és keresés a kosár tartalomban
- A kosárban lévő termékek valós időben szűrhetők kategória, méret és szín alapján.

3.4.9 Rendeléskezelés

Rendelés véglegesítése



A placeOrder() függvény:

- Ellenőrzi, hogy a felhasználó be van-e jelentkezve
- POST kérést küld a /api/Orders/Checkout végpontra
- Sikeres rendelés után üríti a kosarat
- Üzenettel tájékoztatja a felhasználót
- Rendelések megtekintése

A MyOrders.html oldal loadOrders() függvénye:

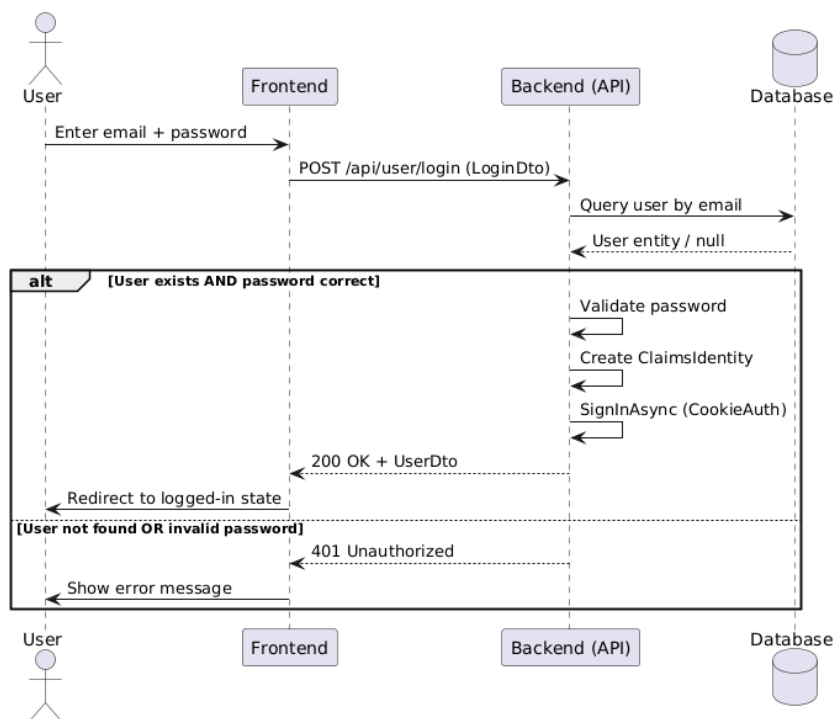
- Lekéri a felhasználó rendeléseit a /api/Orders/Orders végpontról
- Megjeleníti a rendelés ID-jét, dátumát, státuszát
- Listázza az egyes tételeket és az összárát

3.4.10 Mentett termékek

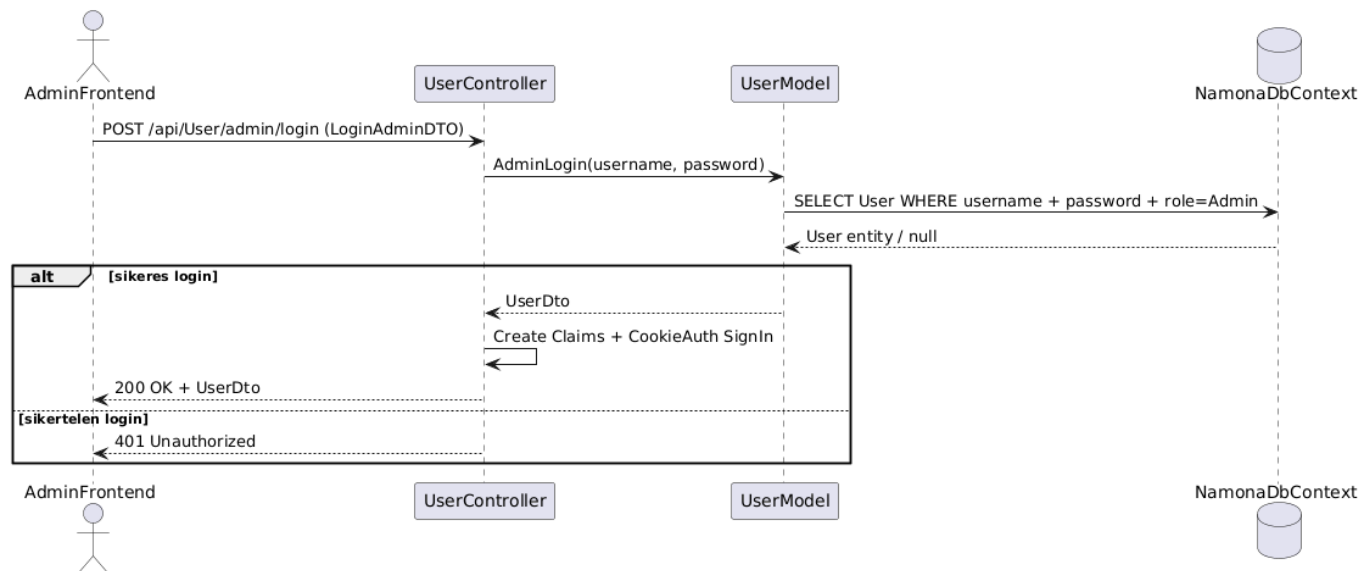
A **SavedProducts.html** oldal a **getSavedProducts()** és **loadSavedProducts()** függvényekhez:

1. A mentett termékek a **localStorage**-ben tárolódnak
2. Az "❤️ Saved" gomb megjelenik a termékdetail oldalakon
3. A felhasználó bármikor eltávolíthat termékeket a mentett listáról
4. Ez a megoldás lehetővé teszi:
5. Offline elérhetőséget (**localStorage**)
6. Gyors hozzáférést kedvenc termékekhez

3.4.11 Bejelentkezés és regisztráció



Admin:



A NamonaLogin.html oldal két fő folyamatot támogat:

- Normál felhasználó bejelentkezés
- Admin bejelentkezés
- Regisztráció
- Bejelentkezés

A loginUser() függvény:

- Összegyűjti az email és jelszó adatokat
- POST kérést küld a /api/User/login végpontra
- Sikeres bejelentkezés után eltárolja a felhasználó adatait (localStorage)
- Átírnyít a főoldalra
- A backend cookie alapú hitelesítést használ
- Admin bejelentkezés

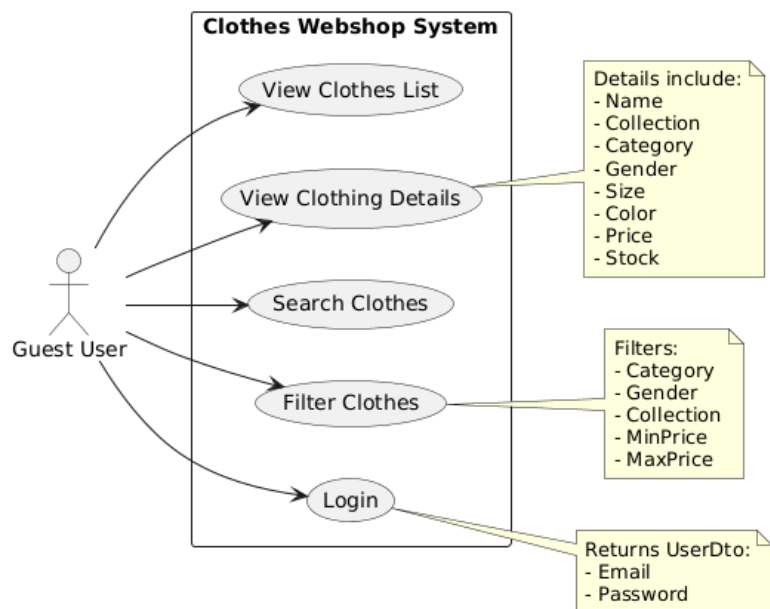
A handleAdminLogin() függvény:

1. Külön adminisztratív hitelesítési végpontot használ (/api/User/admin/login)
2. Sikeres bejelentkezés után átírnyít az AdminDashboard.html-re
3. Regisztráció

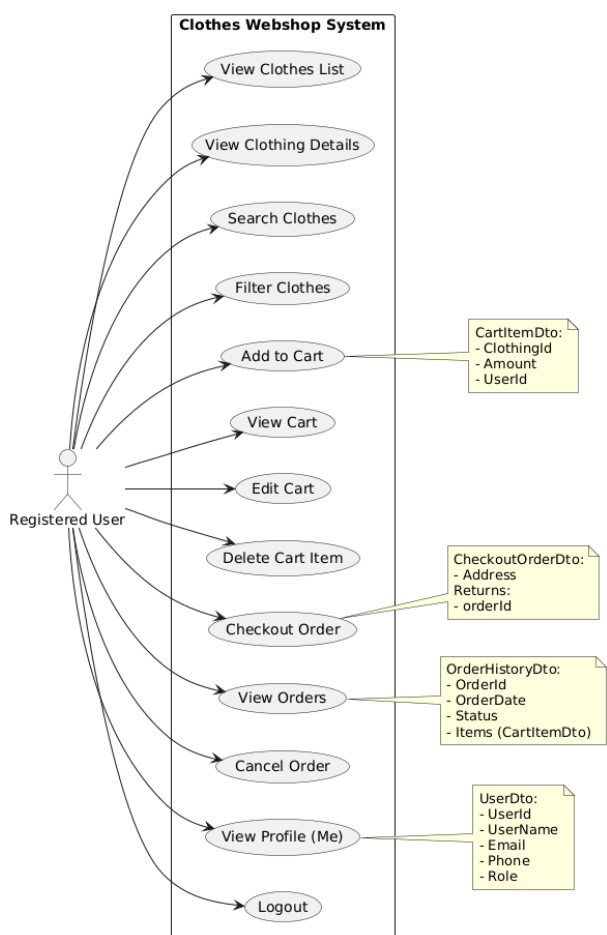
A registerUser() függvény:

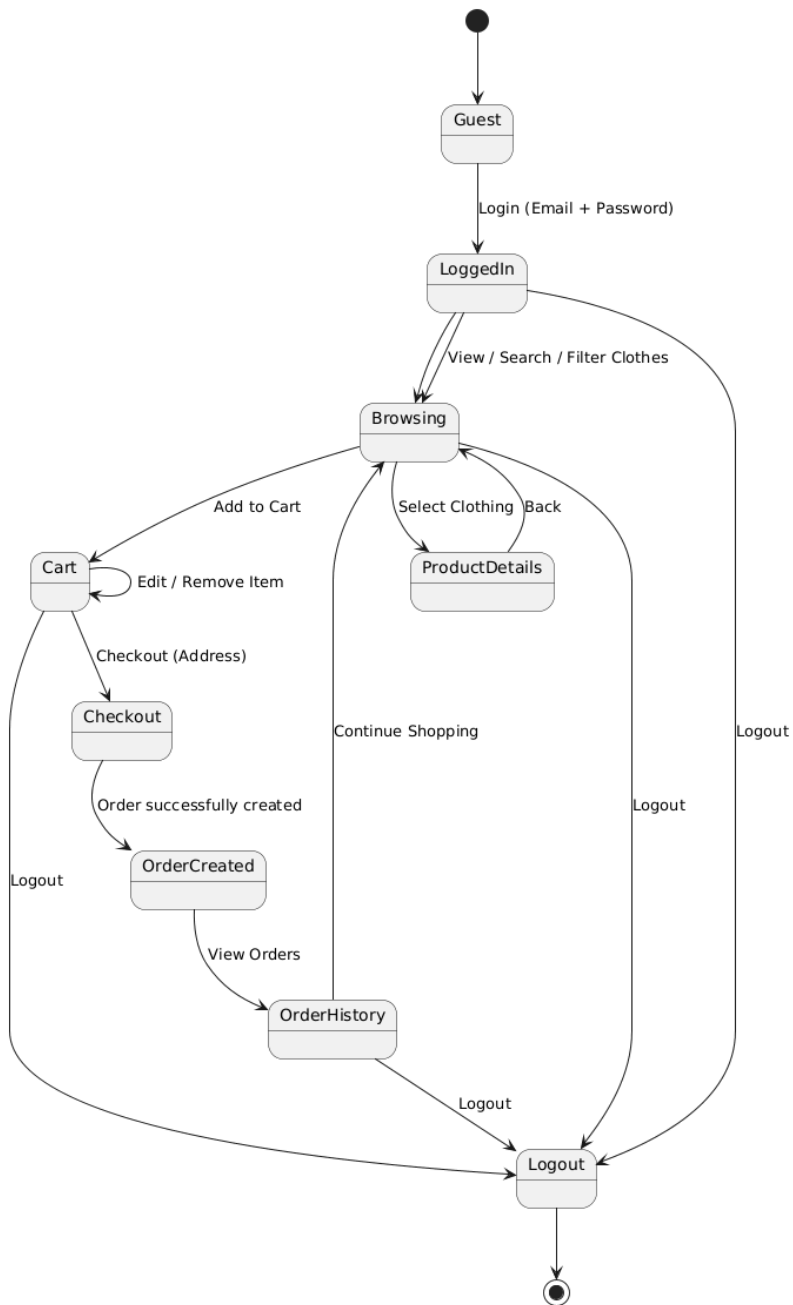
- Ellenőrzi a jelszavak egyezését
- POST kérést küld a /api/user/register végpontra
- Sikeres regisztráció után tájékoztat az eredményről

Bejelentkezés nélküli funkciók:



Bejelentkezve:





3.4.12 Profil kezelés

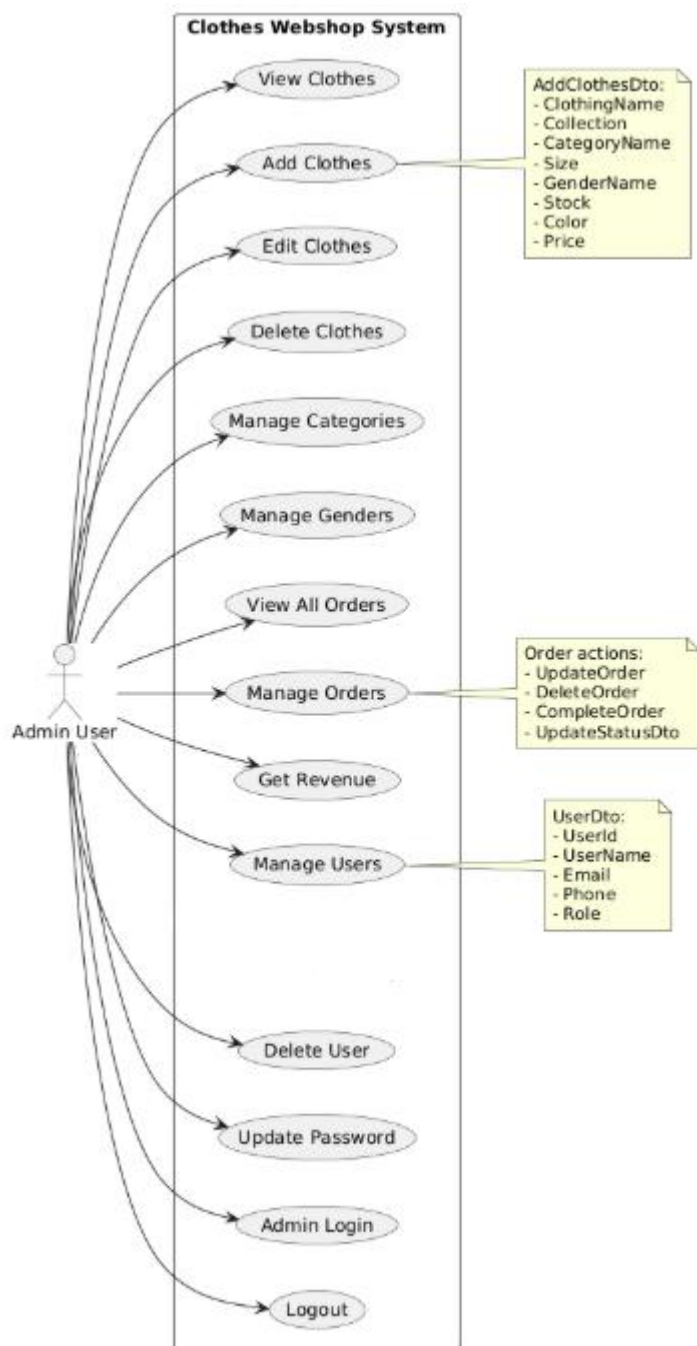
Az `initEventListeners()` függvény a `checkLoggedInUser()` hívásán keresztül:

- Betölti az aktuális felhasználó adatait a `localStorage`-ból
- Dinamikusan frissíti a profil menüt (nema mellett "Logout" gomb)
- Megjeleníti az admin linkeket, ha az user Admin szerepű
- A profil menü tartalmazza:
- Bejelentkezés / Regisztráció linkek (nem bejelentkezett felhasználó)

- Logout gomb (bejelentkezett felhasználó)
- Admin Dashboard link (admin felhasználó)
- Mentett termékek, Rendeléseim, Rólunk linkek

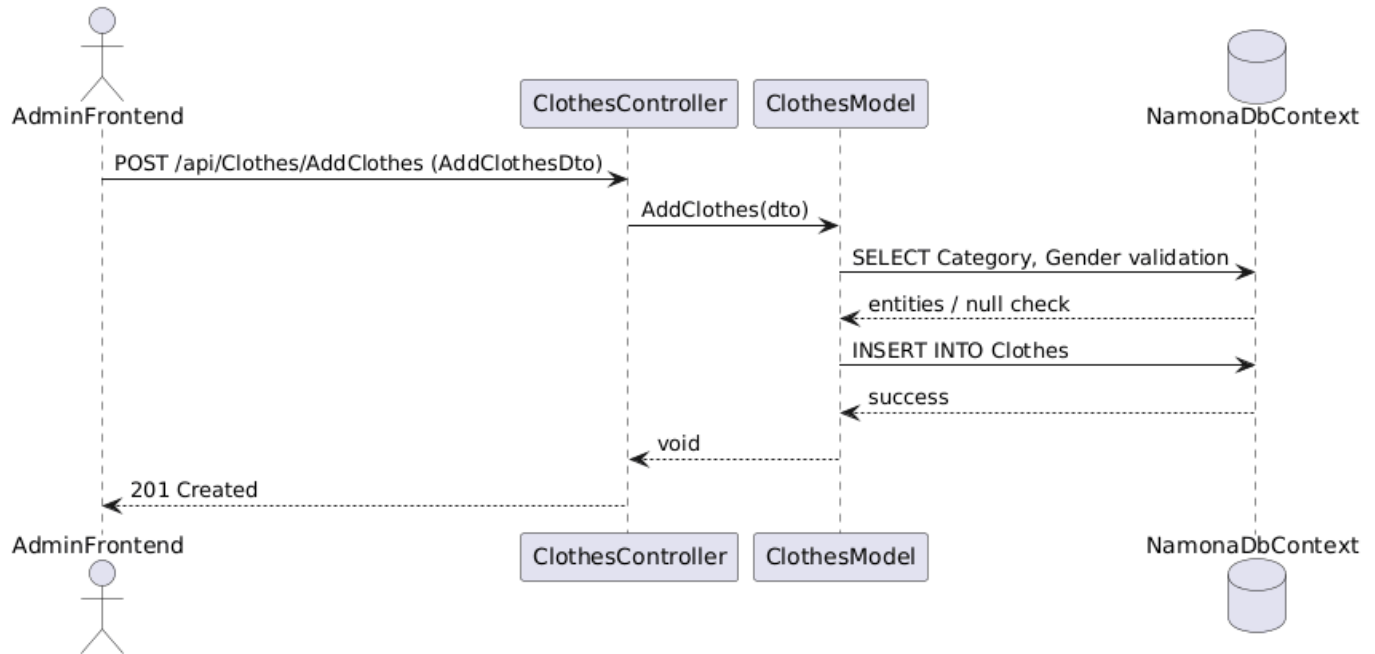
3.4.13 Admin frontend

A projekt admin funkcionalitást is tartalmaz, amelyet az AdminDashboard.html valósít meg.



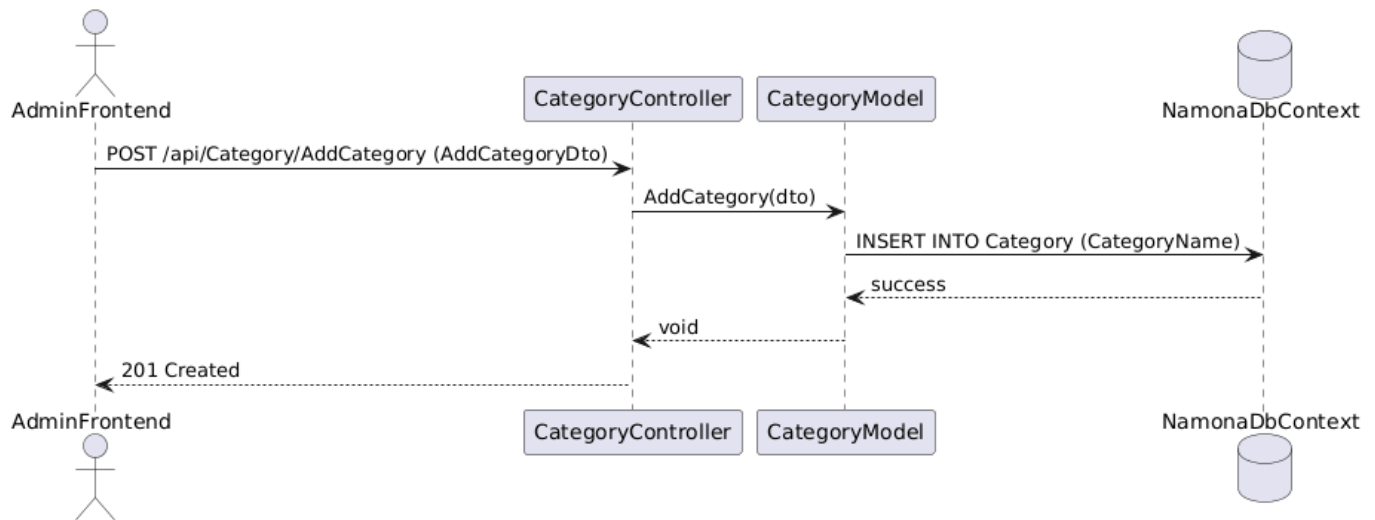
Fő admin műveletek:

- Termékek (Clothes) kezelése (létrehozás, módosítás, törlés)



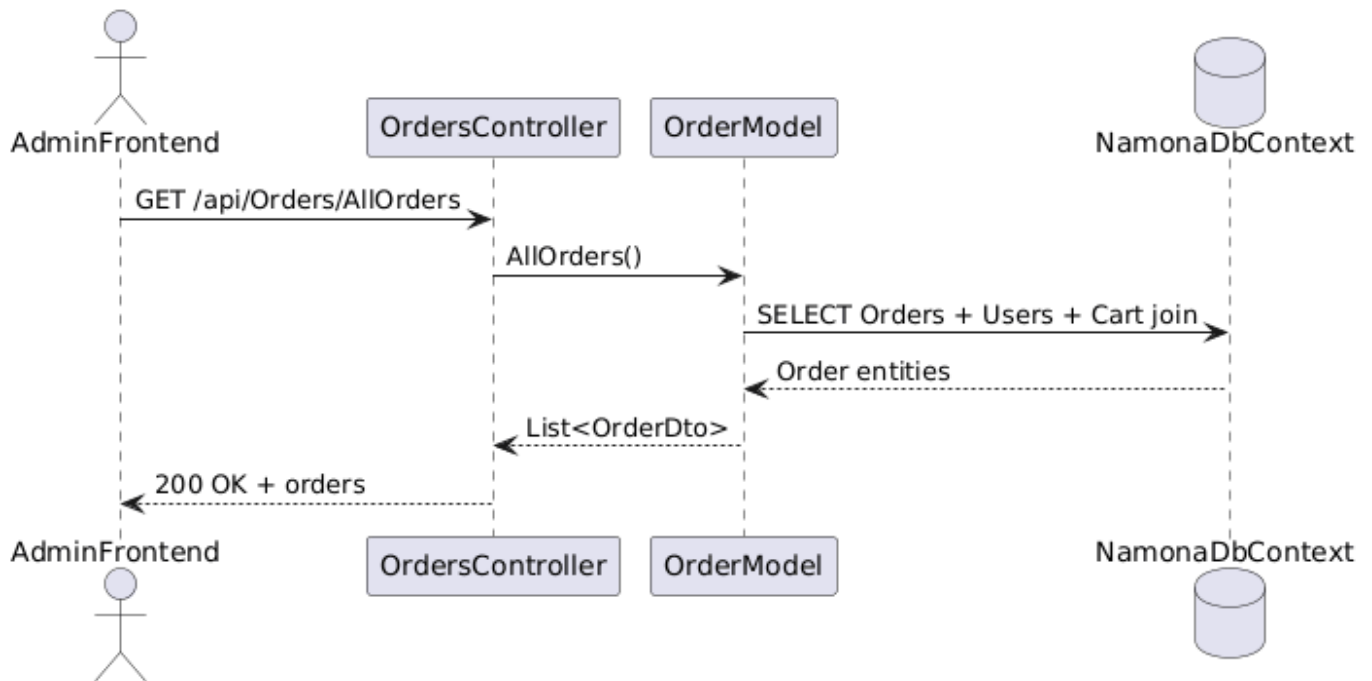
•

- Kategóriák kezelése

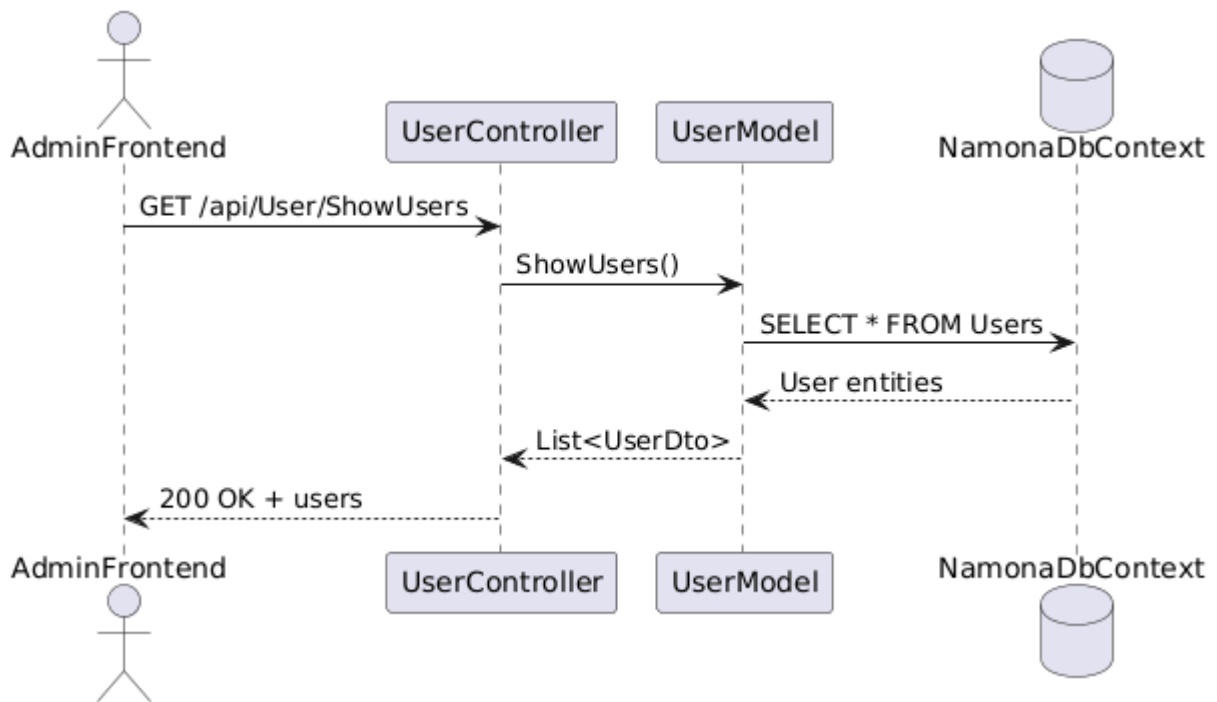


•

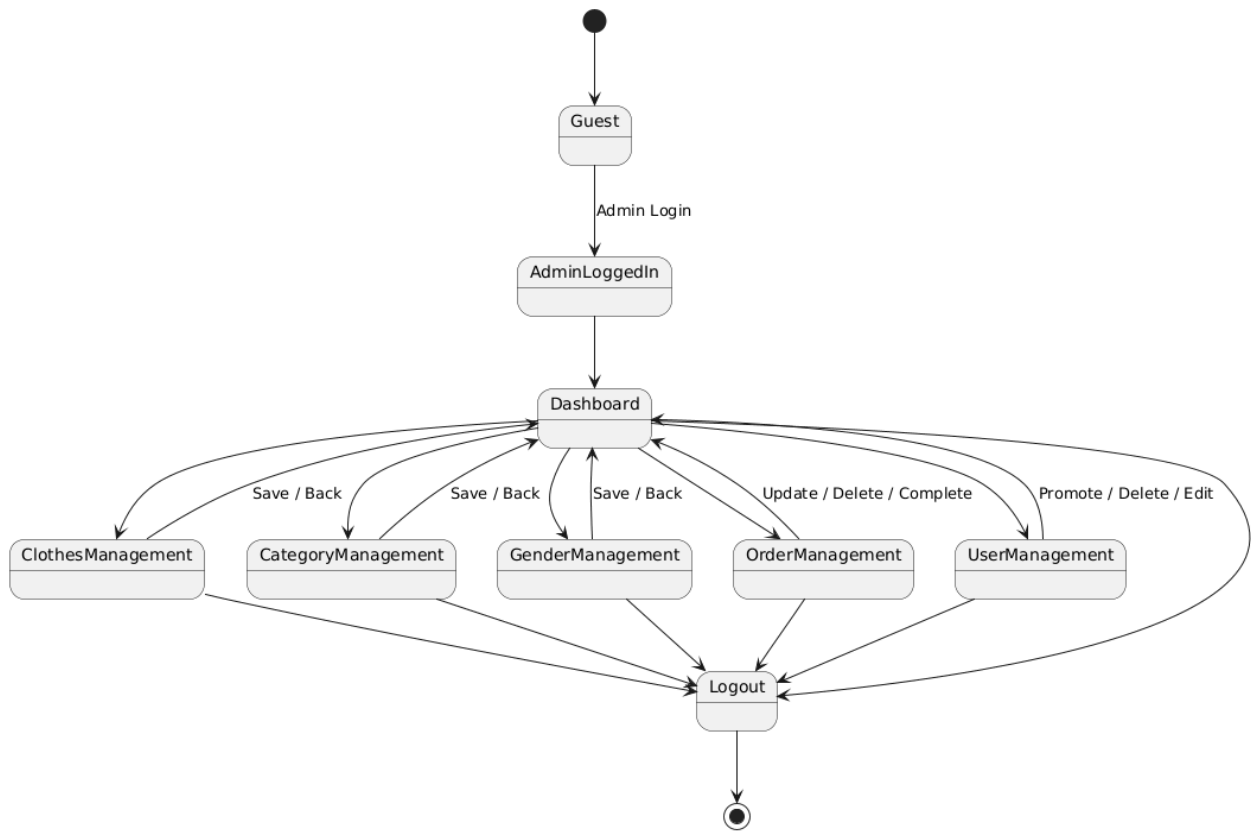
- Rendelések áttekintése és státuszkezelése



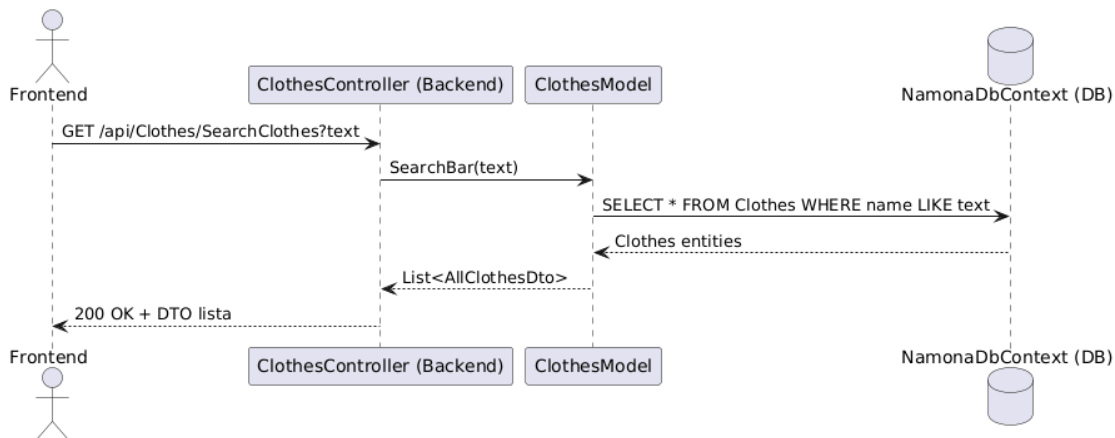
-
- Felhasználók kezelése



-
- Értékesítési statisztikák (kiterjeszthető)
- Admin hitelesítés
- A kliensoldali logika ellenőrzi, hogy az adott felhasználónak Admin szerepe van-e. Ha nem, a rendszer átirányítja a login oldalra.



3.4.14 Globális keresési és szűrési rendszer



A Script.js tartalmazza az `initGlobalSearchUi()` és `initGlobalSideFilter()` függvényeket, amelyek:

- Header Search: Valós idejű keresés az összes terméken
- Side Menu Filter: Kategória, szín, méret szerinti szűrés
- Cache rendszer: 30 másodperces cache a `/api/Clothes/GetAllClothes` végponthoz
- Ez biztosítja a gyors és hatékony termék keresést az egész oldalon.

3.4.15 Event handling és inicializáció

A DOMContentLoaded event handler az oldal típusának alapján inicializálja az szükséges függvényeket:

- Termékdetail oldalakon: loadClothes() és loadProduct()
- Kategória oldalakon: loadCategoryProducts()
- Kosár oldalakon: initializeCartPage() és wireSearchInputs()
- Rendelések oldalakon: loadOrders()
- Mentett termékek oldalakon: loadSavedProducts()

3.5 Avalonia fejlesztői leírása

A Namona desktop és többplatformos adminisztrációs kliens Avalonia UI technológiára épül. Az alkalmazás MVVM architektúrát használ, ahol a Views réteg XAML felületeket tartalmaz, a ViewModels kezeli az UI logikát és állapotot, míg a Model réteg végzi a backend API kommunikációt.

A projekt egy ruházati webshop adminisztrációs rendszerét valósítja meg. Az admin felület segítségével kezelhetők:

- ruházati termékek
- kategóriák
- rendelések
- felhasználók
- dashboard statisztikák

A rendszer több platformra is elkészült:

- Desktop

A közös logika a NamonaAvalonia projektben található.

3.5.1 Alkalmazott technológiák

A kliens a következő technológiákat használja:

- .NET 8
- Avalonia UI
- Avalonia Desktop
- Avalonia Android
- Avalonia iOS
- Avalonia Browser
- CommunityToolkit.Mvvm
- XAML alapú UI
- MVVM architektúra
- HTTP REST API kommunikáció

Továbbá:

- modern többplatformos .NET környezetben fut
- platformfüggetlen UI technológiát használ
- ugyanazokat a ViewModel és Model osztályokat használja minden platformon
- DTO objektumokkal kommunikál a backend API-val
- XAML alapú deklaratív felületet alkalmaz

3.5.2 Az alkalmazás belépési pontja

Desktop környezetben az alkalmazás a `NamonaAvalonia.Desktop` projektből indul.

Az Avalonia szabványos `BuildAvaloniaApp()` inicializációs folyamatot használja, amely az `App.axaml.cs` osztályt tölti be.

A projekt támogatja:

- `IClassicDesktopStyleApplicationLifetime` módot desktopon
- `ISingleViewApplicationLifetime` módot mobil és browser platformokon

3.5.3 Az `App.axaml.cs` működése

Az `App.axaml.cs` végzi az alkalmazás inicializációját.

A főbb lépések:

1. létrejön egy `ClientModel` objektum,
2. beállításra kerül a backend URL,
3. létrejön a `LoginViewModel`,
4. inicializálódik a `MainWindow` vagy `MainView`,
5. a `LoginViewModel` kerül `DataContext`ként beállításra,
6. sikeres bejelentkezés után betöltődik az `AdminPanel`,
7. az `AdminPanelVM` lesz az admin felület `ViewModel`-je.

A backend URL:

- `http://localhost:5222/`

A `SuccessLogin` esemény sikeres belépés után lecseréli a login nézetet az adminisztrációs panelre.

Az alkalmazás tartalmaz egy `DisableAvaloniaDataAnnotationValidation()` metódust is, amely kikapcsolja az Avalonia alapértelmezett `DataAnnotations` validációját a `CommunityToolkit` validációs rendszerrel való ütközések elkerülése érdekében.

3.5.4 MVVM architektúra

Az alkalmazás klasszikus MVVM mintát követ.

Models

A Model réteg végzi:

- a backend API kommunikációt
- HTTP kérések küldését
- DTO objektumok kezelését
- autentikációt
- CRUD műveleteket

A fő modell:

- ClientModel

A projekt számos DTO osztályt használ, például:

- AddClothesDto
- AddCategoryDto
- LoginDto
- RegistrationDto
- OrderDto
- UserDto
- CartDto
- PromoteDto

ViewModels

A ViewModels réteg kezeli:

- a UI állapotot
- az adatbetöltést
- az eseményeket
- a parancsokat
- az adminisztrációs logikát

Főbb ViewModel osztályok:

- LoginViewModel
- AdminPanelVM
- DashboardViewModel
- ClothesPanelViewModel
- ClothesHandlerPanelViewModel
- AddClothesHandlePanelViewModel

- CategoryPanelViewModel
- UserPanelViewModel
- UserHandlerPanelViewModel
- OrderPanelViewModel
- OrderHandlerPanelViewModel

Views

A Views réteg XAML alapú felületeket tartalmaz.

Főbb nézetek:

- MainWindow
- MainView
- LoginWindow
- AdminPanel
- DashboardPanel
- ClothesPanel
- ClothesHandlerPanel
- AddClothesPanel
- CategoryPanel
- UserPanel
- UserHandlerPanel
- OrderPanel
- OrderHandlerPanel

3.5.5 A ClientModel szerepe

3.5.5.1 API kommunikáció

A ClientModel a rendszer központi API-kommunikációs rétege.

Feladata:

- HTTP kérések küldése a backend felé
- JSON válaszok feldolgozása
- autentikáció kezelése
- adminisztratív műveletek végrehajtása
- DTO objektumok kezelése

A fontosabb függvények és szerepük:

Autentikációs függvények

- `LoginAsync()` → felhasználói bejelentkezés kezelése
- `RegisterAsync()` → új felhasználó regisztrációja

Ruházati termék kezelő függvények

- `GetClothesAsync()` → ruházati termékek lekérése
- `AddClothesAsync()` → új ruházati termék létrehozása
- `UpdateClothesAsync()` → termék adatainak módosítása
- `DeleteClothesAsync()` → termék törlése

Kategóriakezelő függvények

- `GetCategoriesAsync()` → kategóriák lekérése
- `AddCategoryAsync()` → új kategória létrehozása
- `UpdateCategoryAsync()` → kategória módosítása
- `DeleteCategoryAsync()` → kategória törlése

Felhasználókezelő függvények

- `GetUsersAsync()` → felhasználók lekérése
- `PromoteUserAsync()` → jogosultság vagy szerepkör módosítása

Rendeléskezelő függvények

- `GetOrdersAsync()` → rendelések lekérése
- `UpdateOrderStatusAsync()` → rendelési státusz módosítása

Kosárkezelő függvények

- `GetCartAsync()` → kosár adatok lekérése

A modell funkcionalitásai:

- bejelentkezés
- regisztráció
- ruházati termékek kezelése
- kategóriák kezelése
- felhasználók kezelése
- rendelések kezelése
- kosár műveletek

- státusz módosítások
 - jogosultságkezelés
-

3.5.6 Az AdminPanelVM szerepe

3.5.6.1 Admin panel működése

A ClientModel a rendszer központi API-kommunikációs rétege.

Feladata:

- HTTP kérések küldése a backend felé
- JSON válaszok feldolgozása
- autentikáció kezelése
- adminisztratív műveletek végrehajtása
- DTO objektumok kezelése

A modell funkcionálisai:

- bejelentkezés
 - regisztráció
 - ruházati termékek kezelése
 - kategóriák kezelése
 - felhasználók kezelése
 - rendelések kezelése
 - kosár műveletek
 - státusz módosítások
 - jogosultságkezelés
-

3.5.6.2 Az AdminPanelVM

Az AdminPanelVM a teljes admin felület központi ViewModelje.

Feladata:

- adminisztrációs panelek kezelése
- navigáció vezérlése
- adatok betöltése
- CRUD műveletek koordinálása
- backend kommunikáció kezelése

Az admin felület külön paneljei:

- Dashboard
- Ruházati termékek kezelése
- Kategóriakezelés
- Felhasználókezelés
- Rendeléskezelés

A rendszer dinamikusan váltja a különböző paneleket.

3.5.7 Inicializáció bejelentkezés után

Sikeres bejelentkezés után a LoginViewModel SuccessLogin eseményt vált ki.

Ekkor:

1. létrejön az AdminPanel,
2. létrejön az AdminPanelVM,
3. a login felület lecserélődik az admin felületre,
4. betöltődnek az adminisztrációs adatok.

Desktop platformon a MainWindow tartalma cserélődik le, míg mobil és browser környezetben a MainView kerül lecserélésre.

3.5.8 Adatok kezelése az alkalmazásban

3.5.8.1 Ruházati termékek kezelése

A ClothesPanel és ClothesHandlerPanel feladata a ruházati termékek adminisztrációja.

A fontosabb függvények:

- LoadClothesAsync() → ruházati termékek betöltése
- AddClothesAsync() → új termék létrehozása
- UpdateClothesAsync() → meglévő termék módosítása
- DeleteClothesAsync() → termék törlése
- OpenAddClothesPanel() → új termék panel megnyitása
- OpenEditClothesPanel() → szerkesztő panel megnyitása
- RefreshClothesAsync() → lista frissítése

A rendszer támogatja:

- termékek listázását
- új termék létrehozását
- termék módosítását
- termék törlését
- kategóriák hozzárendelését
- nemek szerinti besorolást
- készletkezelést
- ár kezelését

Az AddClothesHandlePanelViewModel kezeli az új termékek létrehozását.

A ClothesPanel és ClothesHandlerPanel feladata a ruházati termékek adminisztrációja.

A rendszer támogatja:

- termékek listázását
- új termék létrehozását
- termék módosítását
- termék törlését
- kategóriák hozzárendelését
- nemek szerinti besorolást
- készletkezelést
- ár kezelését

Az AddClothesHandlePanelViewModel kezeli az új termékek létrehozását.

3.5.8.2 Kategóriakezelés

A CategoryPanel segítségével kezelhetők a ruházati kategóriák.

A fontosabb függvények:

- LoadCategoriesAsync() → kategóriák betöltése
- AddCategoryAsync() → új kategória létrehozása
- UpdateCategoryAsync() → kategória módosítása
- DeleteCategoryAsync() → kategória törlése
- RefreshCategoriesAsync() → kategórialista frissítése

A támogatott műveletek:

- kategóriák listázása
- új kategória létrehozása
- kategória módosítása
- kategória törlése

A CategoryPanel segítségével kezelhetők a ruházati kategóriák.

A támogatott műveletek:

- kategóriák listázása
 - új kategória létrehozása
 - kategória módosítása
 - kategória törlése
-

3.5.8.3 Felhasználókezelés

A UserPanel és UserHandlerPanel biztosítja a felhasználók adminisztrációját.

A fontosabb függvények:

- LoadUsersAsync() → felhasználók betöltése
- PromoteUserAsync() → szerepkör módosítása
- DeleteUserAsync() → felhasználó törlése
- RefreshUsersAsync() → felhasználói lista frissítése
- SearchUsersAsync() → felhasználók keresése

A rendszer támogatja:

- felhasználók listázását
- jogosultságkezelést
- szerepkör módosítást
- admin státusz kezelését
- felhasználói adatok kezelését

A PromoteDto objektum szerepkör módosításra szolgál.

A UserPanel és UserHandlerPanel biztosítja a felhasználók adminisztrációját.

A rendszer támogatja:

- felhasználók listázását
- jogosultságkezelést
- szerepkör módosítást
- admin státusz kezelését
- felhasználói adatok kezelését

A PromoteDto objektum szerepkör módosításra szolgál.

3.5.8.4 Rendeléskezelés

Az OrderPanel és OrderHandlerPanel kezeli a webshop rendeléseit.

A fontosabb függvények:

- LoadOrdersAsync() → rendelések betöltése
- UpdateOrderStatusAsync() → rendelési státusz módosítása
- DeleteOrderAsync() → rendelés törlése
- RefreshOrdersAsync() → rendelési lista frissítése
- OpenOrderDetails() → rendelés részleteinek megjelenítése

A rendszer képes:

- rendelések megjelenítésére
- rendelési adatok kezelésére
- státusz módosítására
- rendelési információk adminisztrációjára

A rendelési adatok OrderDto objektumokban kerülnek tárolásra.

Az OrderPanel és OrderHandlerPanel kezeli a webshop rendeléseit.

A rendszer képes:

- rendelések megjelenítésére
- rendelési adatok kezelésére
- státusz módosítására
- rendelési információk adminisztrációjára

A rendelési adatok OrderDto objektumokban kerülnek tárolásra.

3.5.8.5 Dashboard rendszer

A DashboardPanel az admin felület kezdőoldala.

A fontosabb függvények:

- LoadDashboardDataAsync() → dashboard adatok betöltése
- RefreshDashboardAsync() → statisztikák frissítése
- LoadStatisticsAsync() → admin statisztikák lekérése

Feladata:

- összesített adatok megjelenítése
- admin statisztikák kezelése
- rendszerállapot megjelenítése

A DashboardPanel az admin felület kezdőoldala.

Feladata:

- összesített adatok megjelenítése
- admin statisztikák kezelése
- rendszerállapot megjelenítése

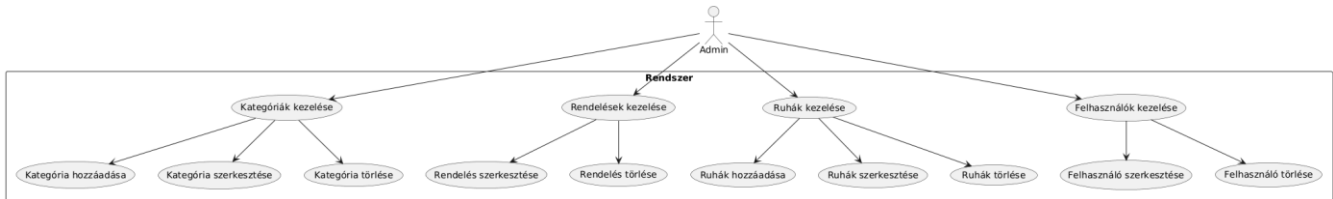
3.5.8.6 UI állapotkezelés

A ViewModel osztályok kezelik:

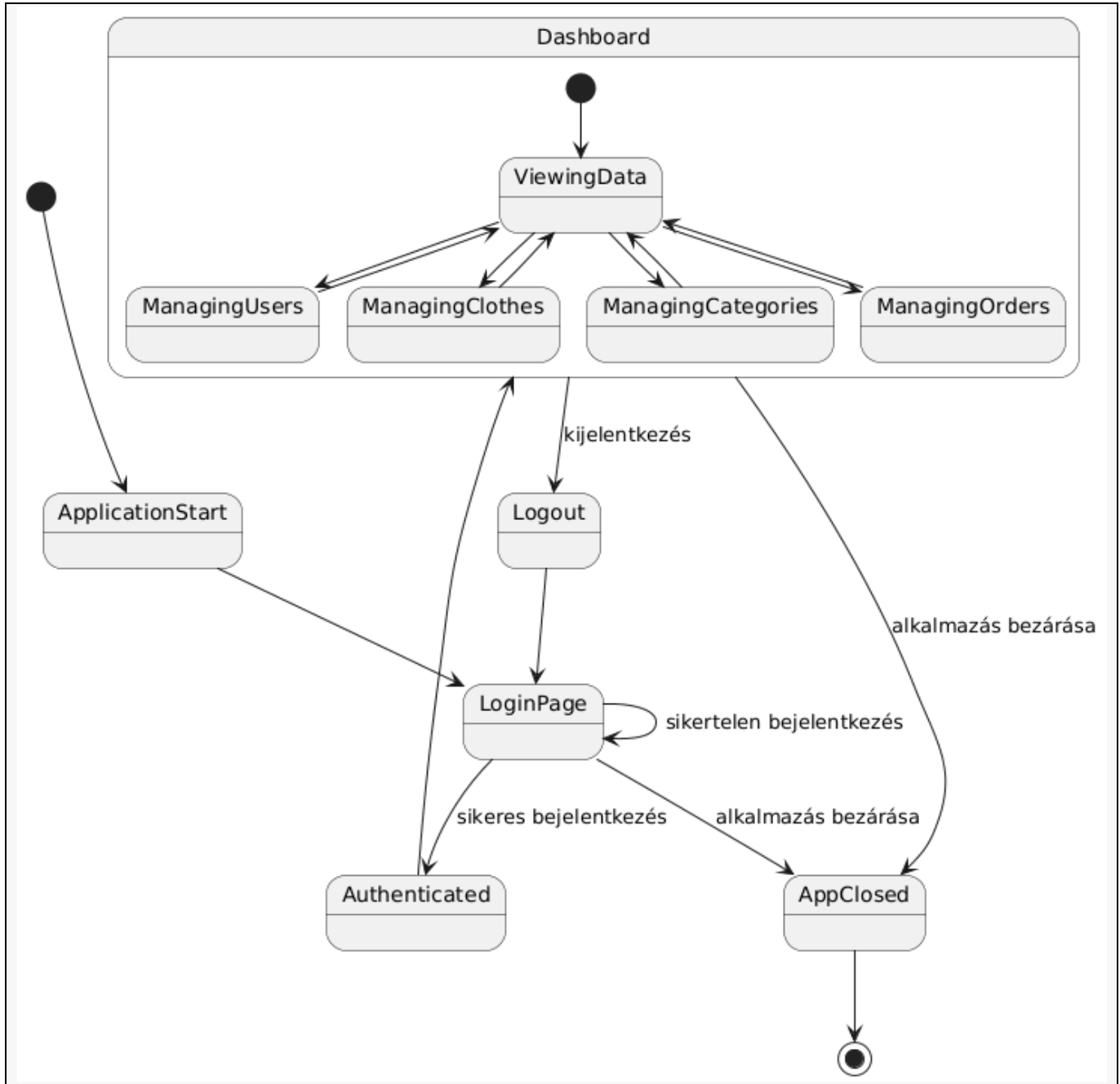
- panelek megnyitását
- navigációt
- adatfrissítést
- eseménykezelést
- felhasználói interakciókat

A felület dinamikusan tölti be az aktuálisan kiválasztott

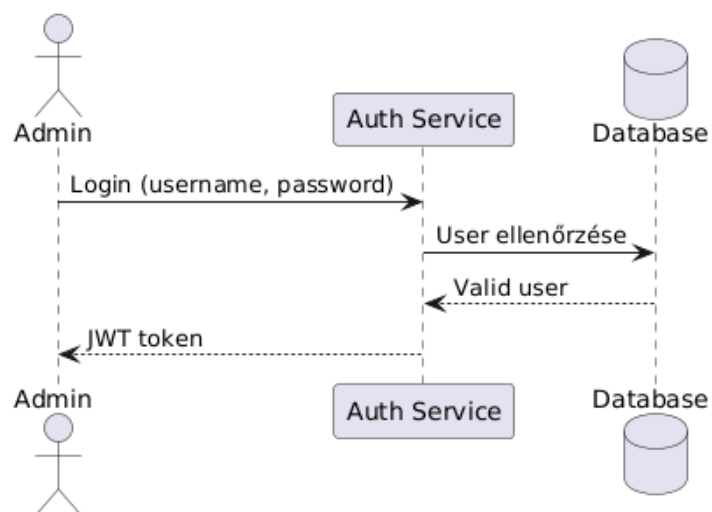
Avalonia use-case diagram

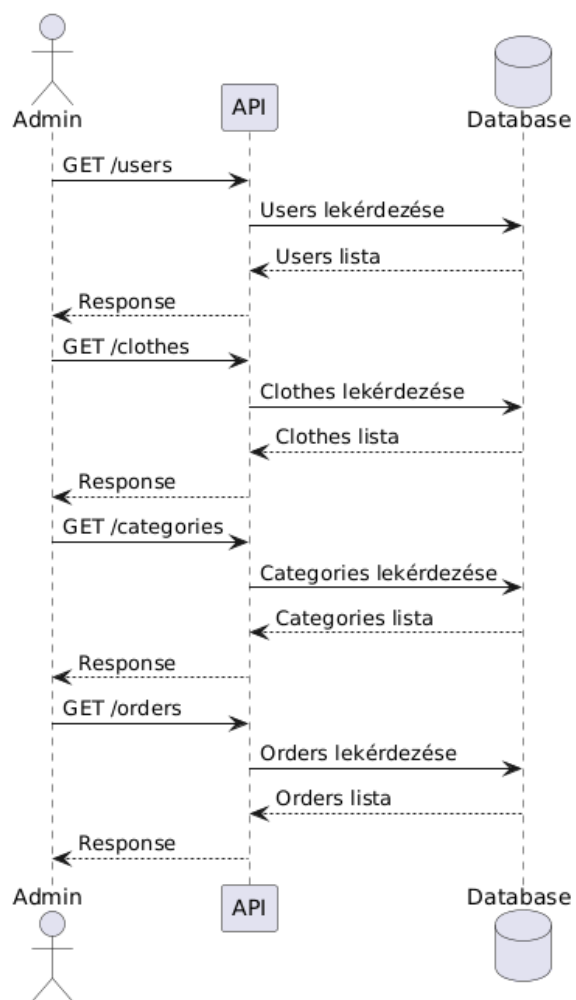


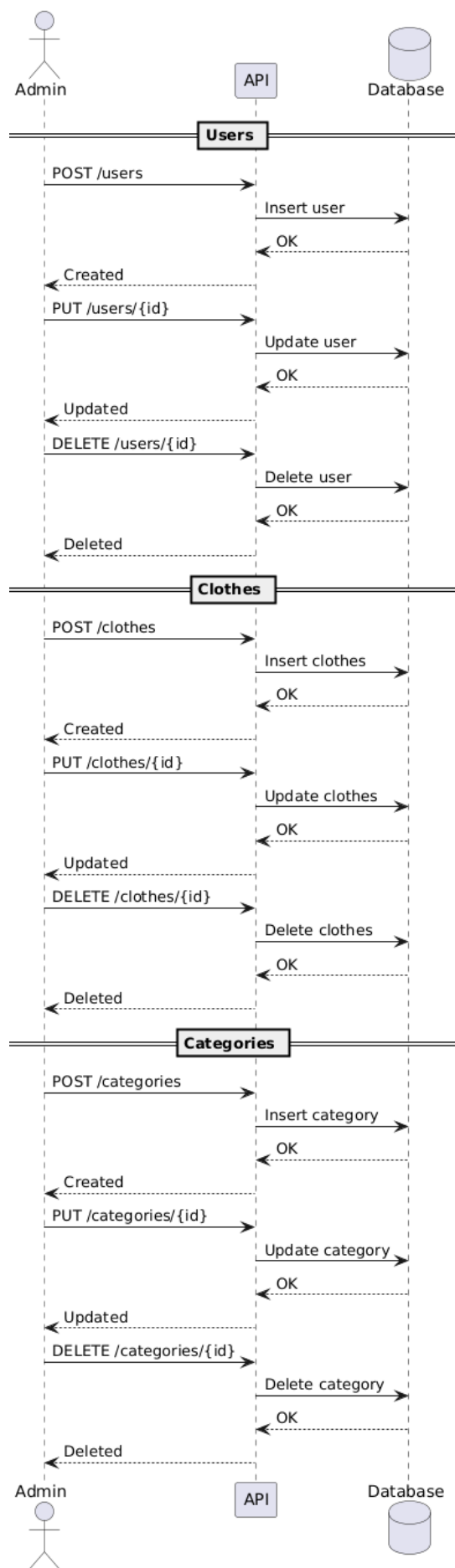
Avalonia állapotdiagram

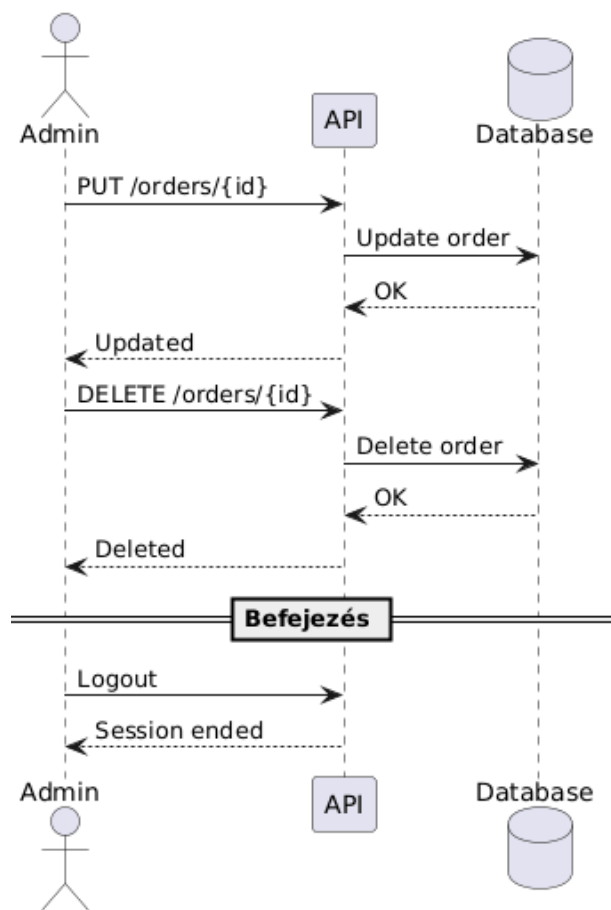


Avalonia szekvencia diagram

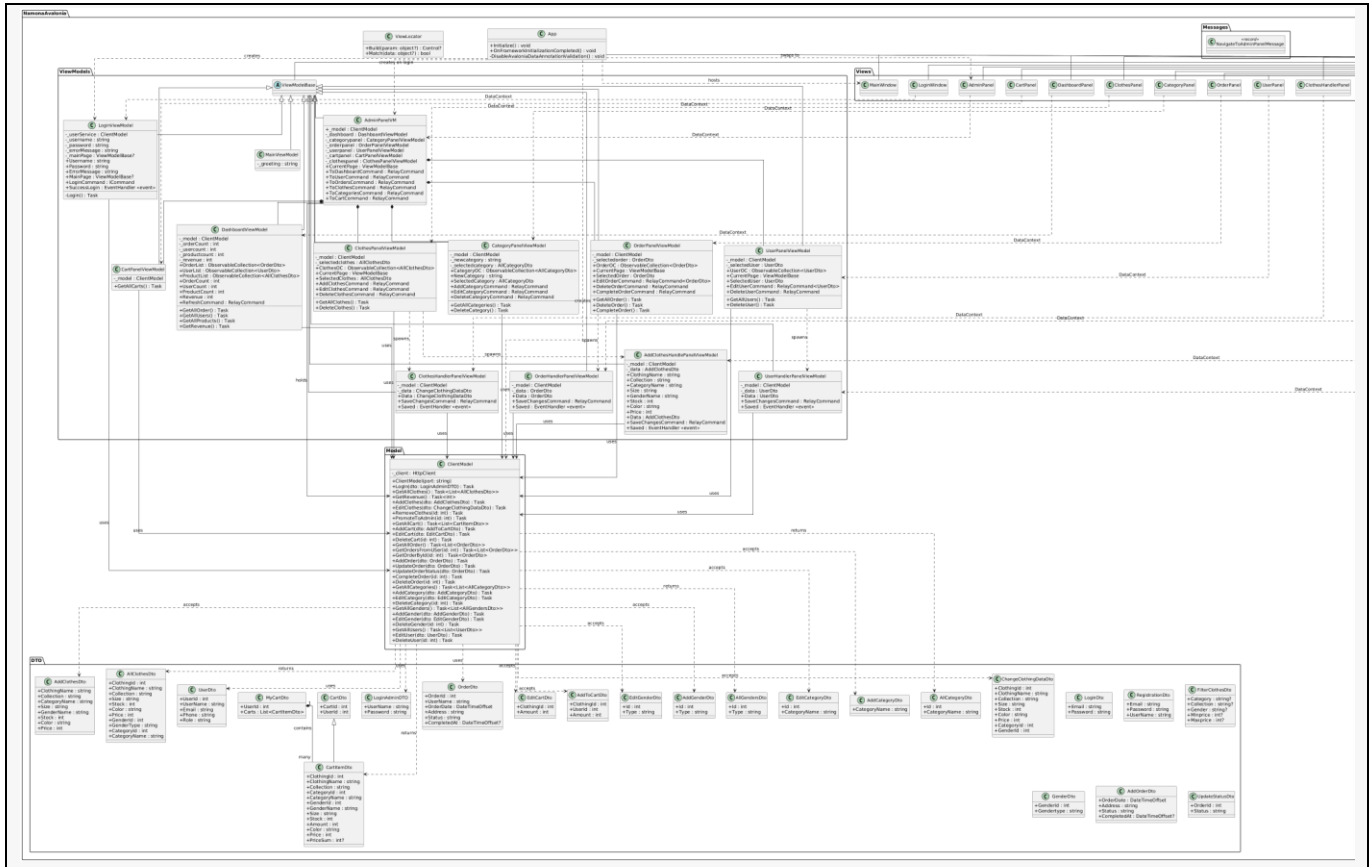








Avalonia osztálydiagram



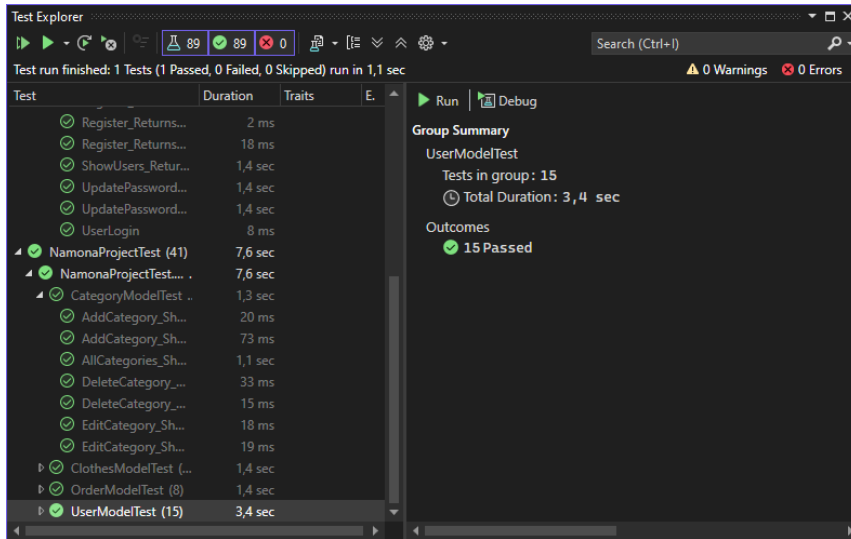
4. Tesztelés

4.1 Tesztkörnyezet

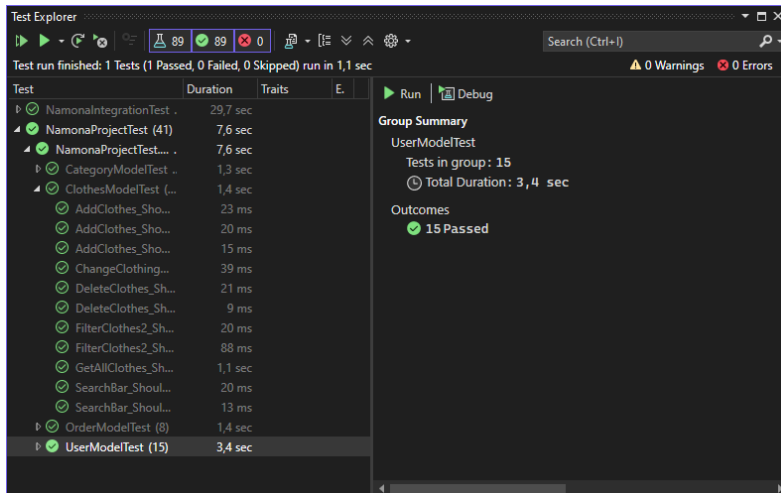
A projekt unit teszteket és integrációs teszteket is tartalmaz. A unit tesztek a model réteg metódusait ellenőrzik, az integrációs tesztek pedig a controller végpontokat HTTP kliensen keresztül vizsgálják. A tesztprojektek xUnit keretrendszerre épülnek.

Tesztprojekt	Tartalom
NamonaProjectTest	ClothesModelTest, CartModelTest, OrderModelTest, CategoryModelTest, GenderModelTest, UserModelTest
NamonaIntegrationTest	ClothesControllerIntegrationTest, CartControllerIntegrationTest, OrderControllerIntegrationTest, GenderControllerIntegrationTest, CategoryControllerIntegrationTest, UserControllerIntegrationTest

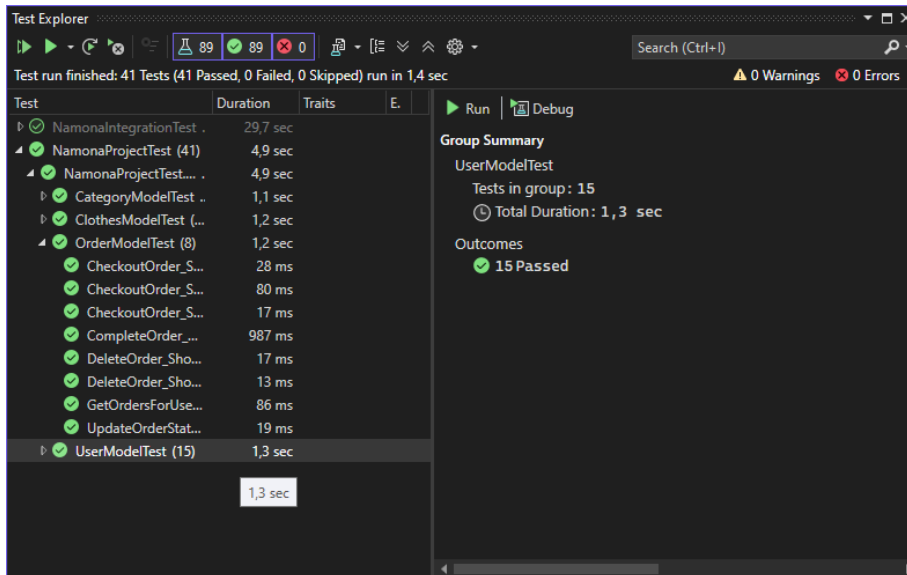
CategoryModelTest:



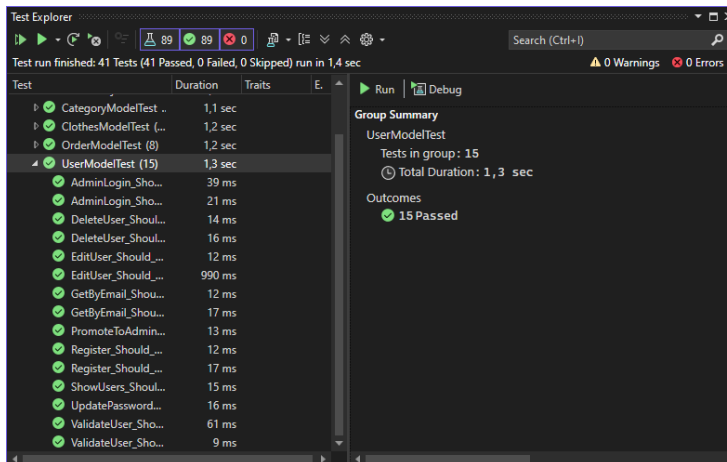
Clothes Unit Test



Orders Unit Test



Users Unit Tests



4.2 UnitTest projekt

A NamonaProjectTest .NET 8 alapú tesztprojekt.

A UnitTest következő tesztelési technológiákat használja:

- xUnit,
- Microsoft.NET.Test.Sdk,
- coverlet.collector,
- Entity Framework Core InMemory,
- Entity Framework Core Sqlite.

Az UnitTest célja elsősorban a model osztályok működésének tesztelése.

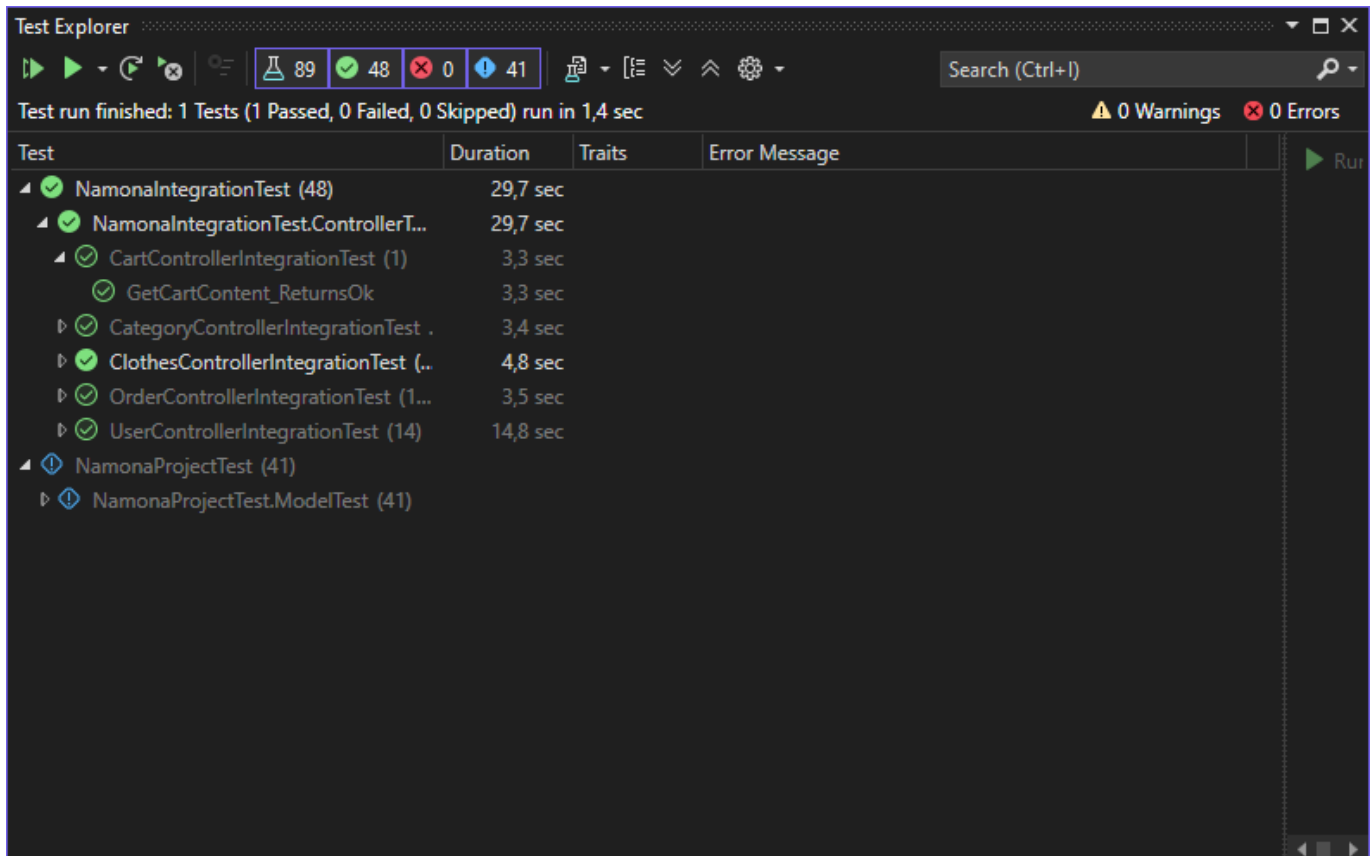
4.3 Tesztadatbázis és seedelés

Az UnitTestekhez külön DbContextFactory és DbSeeder is tartozik:

- DbContextFactory.cs
- DbSeeder.cs

4.4 Integrációs tesztelés :

Cart Integrációs teszt:



Category Integrációs teszt

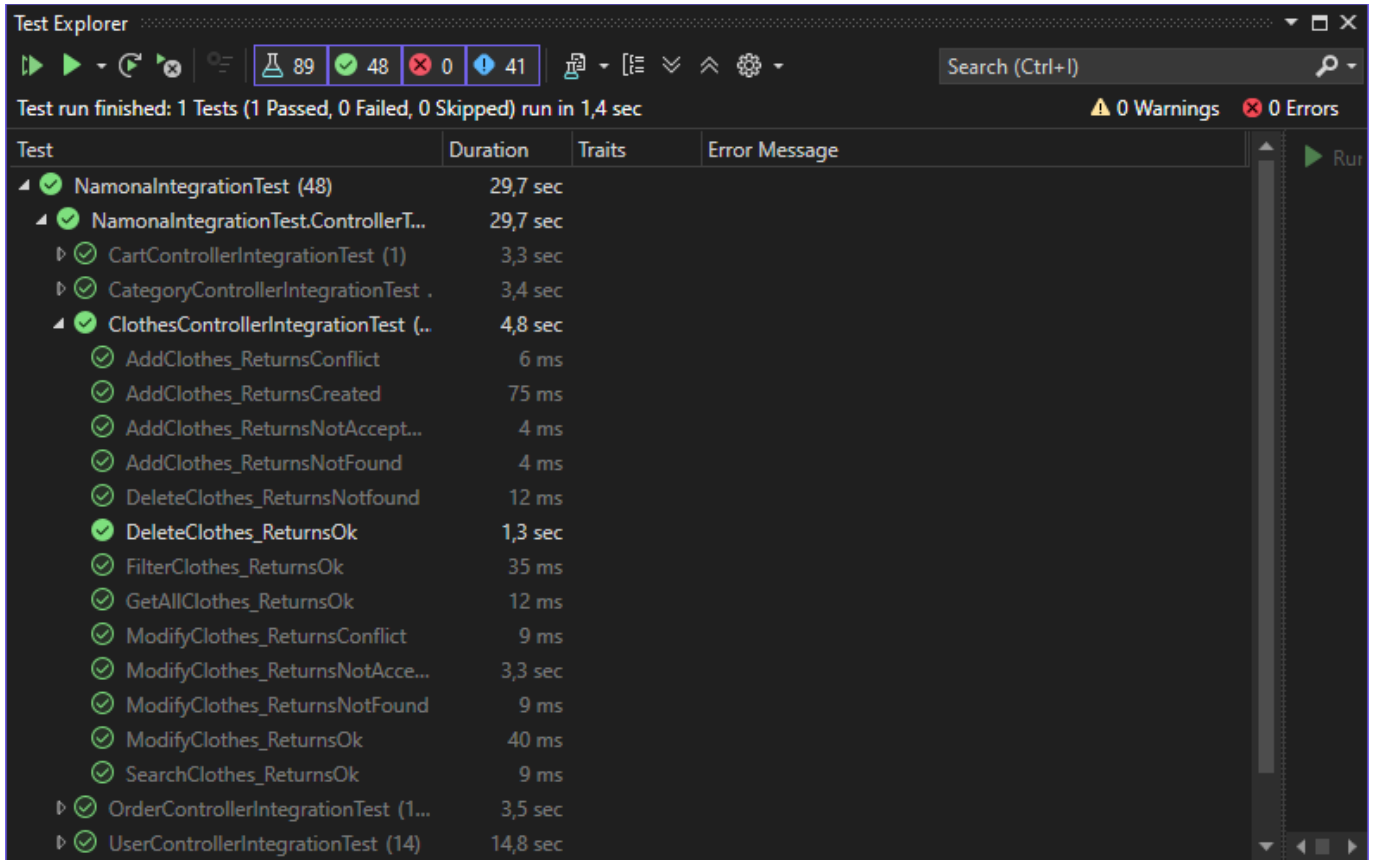
Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 1,4 sec

0 Warnings 0 Errors

Test	Duration	Traits	Error Message
✓ NamonaIntegrationTest (48)	29,7 sec		
✓ NamonaIntegrationTest.ControllerT...	29,7 sec		
✓ CartControllerIntegrationTest (1)	3,3 sec		
✓ CategoryControllerIntegrationTest .	3,4 sec		
✓ AddCategory_ReturnConflict	8 ms		
✓ AddCategory_ReturnCreated	6 ms		
✓ AddCategory_ReturnNotAccept...	18 ms		
✓ DeleteCategory_ReturnNotFound	6 ms		
✓ DeleteCategory_ReturnOk	21 ms		
✓ GetAllCategories_ReturnsOk	41 ms		
✓ ModifyCategory_ReturnsConflict	8 ms		
✓ ModifyCategory_ReturnsNotAc...	4 ms		
✓ ModifyCategory_ReturnsOk	3,3 sec		
✓ ClothesControllerIntegrationTest (..	4,8 sec		
✓ OrderControllerIntegrationTest (1...	3,5 sec		
✓ UserControllerIntegrationTest (14)	14,8 sec		
⚡ NamonaProjectTest (41)			
⚡ NamonaProjectTest.ModelTest (41)			

Clothes Integrációs teszt



The screenshot shows the Test Explorer window in Visual Studio. The top bar indicates the test run status: 89 tests passed, 48 tests failed, 0 tests skipped, and 41 tests were in progress. The test run finished in 1.4 seconds with 0 warnings and 0 errors. The table below lists the tests and their durations.

Test	Duration	Traits	Error Message
✓ NamonaIntegrationTest (48)	29,7 sec		
✓ NamonaIntegrationTest.ControllerT...	29,7 sec		
✓ CartControllerIntegrationTest (1)	3,3 sec		
✓ CategoryControllerIntegrationTest .	3,4 sec		
✓ ClothesControllerIntegrationTest (..	4,8 sec		
✓ AddClothes_ReturnsConflict	6 ms		
✓ AddClothes_ReturnsCreated	75 ms		
✓ AddClothes_ReturnsNotAccept...	4 ms		
✓ AddClothes_ReturnsNotFound	4 ms		
✓ DeleteClothes_ReturnsNotFound	12 ms		
✓ DeleteClothes_ReturnsOk	1,3 sec		
✓ FilterClothes_ReturnsOk	35 ms		
✓ GetAllClothes_ReturnsOk	12 ms		
✓ ModifyClothes_ReturnsConflict	9 ms		
✓ ModifyClothes_ReturnsNotAcce...	3,3 sec		
✓ ModifyClothes_ReturnsNotFound	9 ms		
✓ ModifyClothes_ReturnsOk	40 ms		
✓ SearchClothes_ReturnsOk	9 ms		
✓ OrderControllerIntegrationTest (1...	3,5 sec		
✓ UserControllerIntegrationTest (14)	14,8 sec		

Order Integrációs teszt

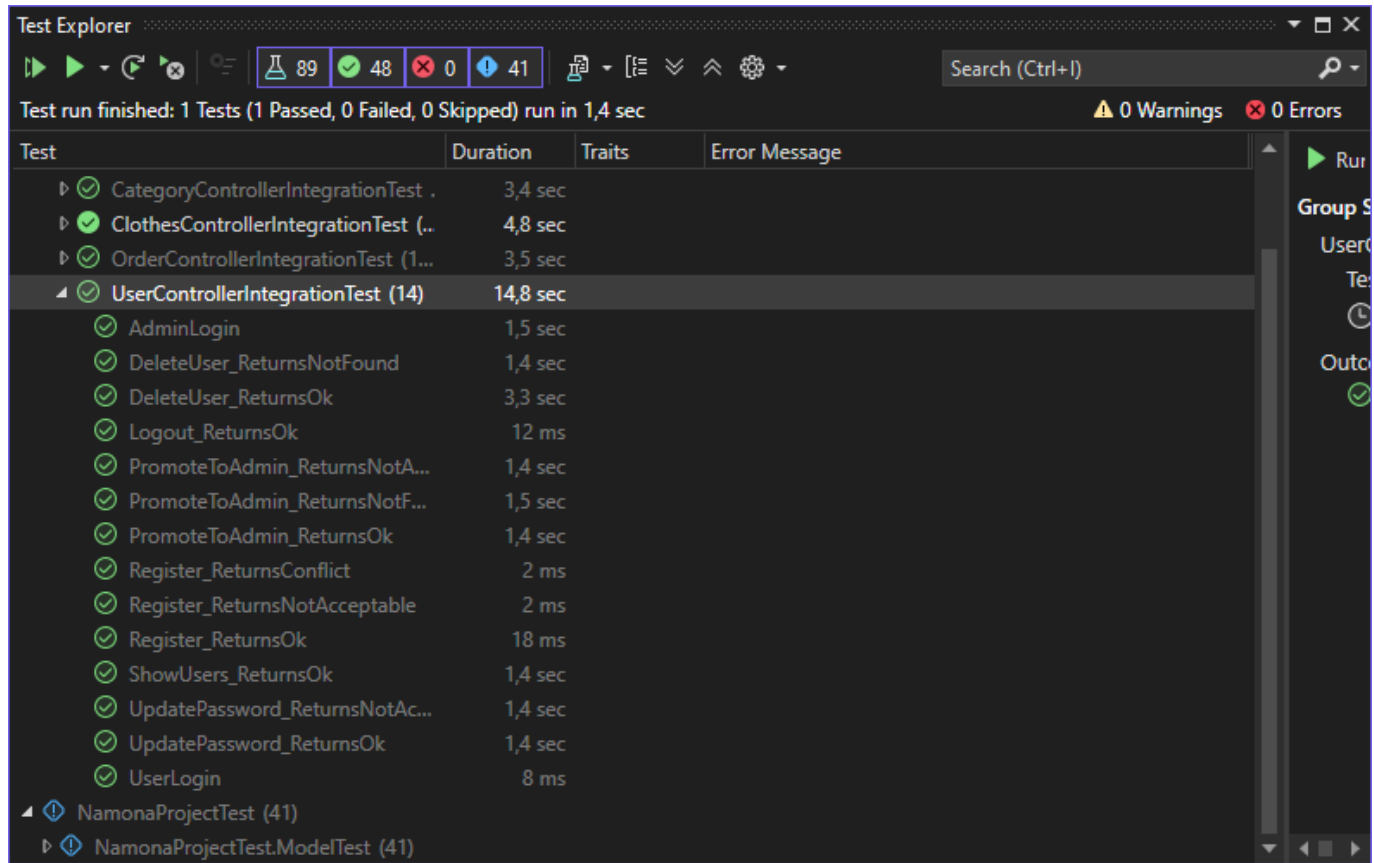
Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 1,4 sec

0 Warnings 0 Errors

Test	Duration	Traits	Error Message
✓ NamonaIntegrationTest (48)	29,7 sec		
✓ NamonaIntegrationTest.ControllerT...	29,7 sec		
✓ CartControllerIntegrationTest (1)	3,3 sec		
✓ CategoryControllerIntegrationTest .	3,4 sec		
✓ ClothesControllerIntegrationTest (..	4,8 sec		
✓ OrderControllerIntegrationTest (1...	3,5 sec		
✓ CompleteOrder_ReturnsNotFou...	5 ms		
✓ DeleteOrderAdmin_ReturnsNot...	2 ms		
✓ EditOrder_ReturnsNotAccaptable	15 ms		
✓ EditOrder_ReturnsOk	3,3 sec		
✓ EditStatus_ReturnsNotAcceptable	19 ms		
✓ EditStatus_ReturnsOk	7 ms		
✓ GetAllOrders_ReturnsOk	35 ms		
✓ GetOrderByld_ReturnsNotFound	9 ms		
✓ GetOrderByld_ReturnsOk	9 ms		
✓ GetRevenue_ReturnsOk	19 ms		
✓ Orders_ReturnsOk	78 ms		
✓ UserControllerIntegrationTest (14)	14,8 sec		
⚡ NamonaProjectTest (41)			
⚡ NamonaProjectTest.ModelTest (41)			

User Integrációs teszt



4.5 Tesztesetek táblázata

Teszt neve	Tesztelt funkció	Bemeneti paraméter	Elvárt eredmény	Eredmény
UserModelTest.Register_Should_Add_New_User	Regisztráció	Email: newuser@ password123	Új user létrejön, Role = User, jelszó hash-elve	Átment
UserModelTest.ValidateUser_Should_Return_UserDto_When_Correct	Bejelentkezés helyes adatokkal	Email + helyes jelszó	UserDto visszatér	Átment
UserModelTest.ValidateUser_Should_Throw_When_Invalid	Hibás login	Nem létező email / jelszó	InvalidOperationException	Átment

UserModelTest.ShowUsers_Should_Return_Users	Felhasználók listázása	nincs	Nem üres UserDto lista	Átment
UserModelTest.AdminLogin_Should_Return_Admin_When_Correct	Admin bejelentkezés	Admin user + helyes jelszó	Admin UserDto visszatér	Átment
UserModelTest.AdminLogin_Should_Throw_When_Not_Admin	Nem admin login	UserId	User törlődik DB-ből	Átment
UserModelTest.DeleteUser_Should_Remove_User	Felhasználó törlése	userId=1	User törlődik	Átment
UserModelTest.DeleteUser_Should_Throw_When_NotFound	Hibás törlés	nem létező ID	InvalidOperationException	Átment
UserModelTest.UpdatePassword_Should_Change_Password	Jelszó módosítása	UserId + új jelszó	Jelszó frissül hash-elve	Átment
UserModelTest.GetByEmail_Should_Return_Null_When_NotFound	Email keresés	UserId + Role=admin	Role = Admin	Átment
UserModelTest.GetByEmail_Should_Return_User	Email keresés	nem létező email	null	Átment
UserModelTest.EditUser_Should_Update_User	User módosítás	UserDto	Adatok frissülnek	Átment
UserModelTest.EditUser_Should_Throw_When_Invalid_Data	Hibás szerkesztés	üres mezők	InvalidDataException	Átment
ClothesModelTest.GetAllClothes_Should_Return_Data	Összes ruha lekérdezése	nincs	Nem üres AllClothesDto lista, minden elem tartalmaz nevet,	Átment

			kategóriát, gender típust	
ClothesModelTest.AddClothes_Should_Add_New_Item	Új ruha hozzáadása	AddClothesDto (valid kategória + gender)	Új Clothes rekord létrejön az adatbázisban	Átment
ClothesModelTest.AddClothes_Should_Throw_When_Category_Invalid	Hibás kategória kezelése	CategoryName = INVALID	KeyNotFoundException	Átment
ClothesModelTest.AddClothes_Should_Throw_When_Invalid_Data	Hibás adatok kezelése	null mezők, negatív stock	nvalidDataException	Átment
ClothesModelTest.ChangeClothingData_Should_Update_Item	Ruha módosítása	ChangeClothingDataDto	Létező ruha adatai frissülnek	Átment
ClothesModelTest.DeleteClothes_Should_Remove_Item	Ruha törlése	ClothingId	Ruha törlődik az adatbázisból	Átment
ClothesModelTest.DeleteClothes_Should_Throw_When_NotFound	Hibás törlés	nem létező ID	KeyNotFoundException	Átment
ClothesModelTest.FilterClothes2_Should_Filter_By_Category	Szűrés kategória szerint	CategoryName	Csak adott kategóriájú ruhák	Átment
ClothesModelTest.FilterClothes2_Should_Filter_By_Price	Szűrés ár szerint	Minprice + Maxprice	Csak adott árintervallum	Átment
ClothesModelTest.SearchBar_Should_Return_Results	Keresés (találat esetén)	létező szöveg	Nem üres lista	Átment
ClothesModelTest.SearchBar_Should_Return_Empty_When_NoMatch	Keresés (nincs találat)	"zzzzzz_not_found"	Üres lista	Átment

CategoryModelTest.AllCategories_Should_Return_Data	Összes kategória lekérdezése	nincs	Nem üres AllCategoryDto lista, minden elem tartalmaz CategoryName és pozitív Id	Átment
CategoryModelTest.AddCategory_Should_Add_New_Category	Új kategória hozzáadása	AddCategoryDto (CategoryName = "TestCategory")	Új Category rekord létrejön az adatbázisban	Átment
CategoryModelTest.AddCategory_Should_Throw_When_Name_Null	Hibás adat kezelése	CategoryName = null	InvalidDataException	Átment
CategoryModelTest.EditCategory_Should_Update_Name	Kategória módosítása	EditCategoryDto (létező Id + új név)	Kategória neve frissül az adatbázisban	Átment
CategoryModelTest.EditCategory_Should_Throw_When_Name_Null	Hibás módosítás	CategoryName = null	InvalidDataException	Átment
CategoryModelTest.DeleteCategory_Should_Remove_Category	Kategória törlése	CategoryId	Kategória törlődik az adatbázisból	Átment
CategoryModelTest.DeleteCategory_Should_Throw_When_NotFound	Nem létező kategória törlése	999999 ID	KeyNotFoundException	Átment
OrderModelTest.CheckoutOrder_Should_Create_Order_And_Update_Stock	Checkout rendelés létrehozása és készlet csökkentése	UserId + Cart (ClothingId, Amount=1), Address="Test address"	Orders létrejön, Cart.OrderId kitöltődik, Clothing.Stock csökken	Átment
OrderModelTest.CheckoutOrder_Should_Throw_When_Cart_Is_Empty	Üres kosár esetén checkout hibakezelés	UserId=1, üres cart	InvalidOperationException	Átment

OrderModelTest.Checkout Order_Should_Throw_When_Invalid_Amount	Érvénytelen mennyiség kezelése	Cart.Amount > Clothing.Stock	InvalidDataException	Átment
OrderModelTest.DeleteOrder_Should_Remove_Order	Rendelés törlése	létező OrderId	Order törlődik adatbázisból	Átment
OrderModelTest.DeleteOrder_Should_Throw_When_Not_Found	Nem létező rendelés törlése	OrderId=999999	KeyNotFoundException	Átment
OrderModelTest.CompleteOrder_Should_Update_Status_And_Date	Rendelés teljesítése	OrderId	Status="Completed", CompletedAt kitöltve	Átment
OrderModelTest.GetOrdersForUser_Should_Return_Grouped_Orders	Felhasználó rendeléseinek lekérése	UserId	OrderHistory Dto	Átment
OrderModelTest.UpdateOrderStatus_Should_Update_Status	Státusz módosítása	OrderId + Status="Shipped"	Orders.Status frissül	Átment

Teszt neve	Tesztelt funkció	Bemeneti paraméter	Elvárt eredmény	Eredmény
UserControllerTests.RegisterUser	Regisztráció API	POST /api/user/regist	Sikeres státusz	Átment
UserControllerTests.LoginUser	Bejelentkezés API	POST /api/user/login	OK vagy Unauthorized ellenőrzés	Átment
UserControllerTests.AdminLogin	Admin bejelentkezés API	POST /api/User/admin/login	OK vagy Unauthorized ellenőrzés	Átment
UserControllerTests.Register_ReturnsNotAcceptable	Regisztráció validációs hiba	POST /api/User/register	406 Not Acceptable	Átment
UserControllerTests.Register_ReturnsConflict	Regisztráció duplikált email esetén	POST /api/User/register	409 Conflict	Átment
UserControllerTests.ShowUsers	Felhasználók listázása (admin)	GET /api/User/ShowUsers	OK	Átment
UserControllerTests.DeleteUser	Felhasználó törlése	DELETE /api/User/DeleteUser?id=...	OK	Átment
UserControllerTests.UpdatePassword	Jelszó módosítás	PUT /api/User/password	OK	Átment
UserControllerTests.UpdatePassword_ReturnsNotAcceptable	Érvénytelen jelszó módosítás	PUT /api/User/password	406	Átment

UserControllerTests.PromoteToAdmin	Felhasználó adminná léptetése	PUT /api/User/promote	OK	Átment
UserControllerTests.PromoteToAdmin_ReturnsNotAcceptable	Érvénytelen role megadás	PUT /api/User/promote	406	Átment
UserControllerTests.PromoteToAdmin_ReturnsNotFound	Nem létező szerepkör vagy user	PUT /api/User/promote	404	Átment
UserControllerTests.Logout	Kijelentkezés API	DELETE /api/admin/deleteuser	OK	Átment

5. Összegzés

A projekt során egy működő ruházati webáruház rendszer került megvalósításra, amely több egymáshoz kapcsolódó komponensből épül fel. Elkészült a központi ASP.NET Core alapú backend API, a PostgreSQL adatbázis, a webes felhasználói felület, valamint az Avalonia alapú asztali kliens alkalmazás is. A rendszer képes ruhák, kategóriák, nemek, kosár tartalmának, rendelések és felhasználók kezelésére, továbbá támogatja az online vásárlási és rendelési folyamatot.

A fejlesztés során jelentős tapasztalatot szereztünk a modern .NET alapú alkalmazásfejlesztés területén. Gyakorlatot szereztünk REST API tervezésében és implementálásában, adatbázis-tervezésben PostgreSQL környezetben, Entity Framework Core használatában, valamint a frontend és backend integrációjában. Emellett több kliens alkalmazás (web és desktop) közös backendhez való csatlakoztatása is fontos gyakorlati tapasztalatot jelentett. A projekt során fejlődünk a verziókezelés, hibakeresés, tesztelés és dokumentációkészítés területén is.

A projekt egyik legfontosabb tanulsága az volt, hogy egy összetett rendszer fejlesztésénél kiemelten fontos a megfelelő tervezés, a rétegek szétválasztása és a moduláris felépítés. Megtanultuk, hogy a backend, az adatbázis és a kliens oldali alkalmazások csak jól definiált API-kon keresztül képesek stabilan együttműködni. További fontos tapasztalat volt, hogy a folyamatos tesztelés és a hibák korai kezelése jelentősen növeli a fejlesztés hatékonyságát és a rendszer megbízhatóságát.

6. Köszönetnyilvánítás

Szeretnénk köszönetet mondani mindazoknak, akik szakmai tudásukkal és útmutatásukkal támogatták munkánkat a projekt megvalósítása során. A tanulmányaink és a fejlesztési folyamat alatt elsajátított ismeretek, valamint a kapott észrevételek és javaslatok nélkülözhetetlen szerepet játszottak a rendszer felépítésében és a feladat eredményes elvégzésében. Különösen Béltéki Anisszának és Boros Botondnak tartozunk köszönettel, hiszen az ő közreműködésüknek köszönhető szakmai tudásunk nagy része.

A tőlük kapott segítség nemcsak abban volt meghatározó, hogy a projektet sikeresen befejezzük, hanem abban is, hogy szakmailag fejlődjünk és gyakorlati tudásunkat elsajátítsuk.